Inverse problems, optimization & imaging

Vision Robotics – M2 SETI

Nicolas Lermé[1]

[1]Laboratoire SATIE
Avenue des sciences, Gif-sur-yvette
nicolas.lerme@universite-paris-saclay.fr

Last update: December 14, 2022



Outline

1. Motivations
2. Bayes justification
3. Elements of optimization
4. Applications
5. Conclusion

Outline

1 Motivations

2 Bayes justification

3 Elements of optimization

4 Applications

5 Conclusion

How computer vision differs from image processing?

- Both terms are often used interchangeably in many contexts.
- Both domains are in fact intertwined and have distinct goals.

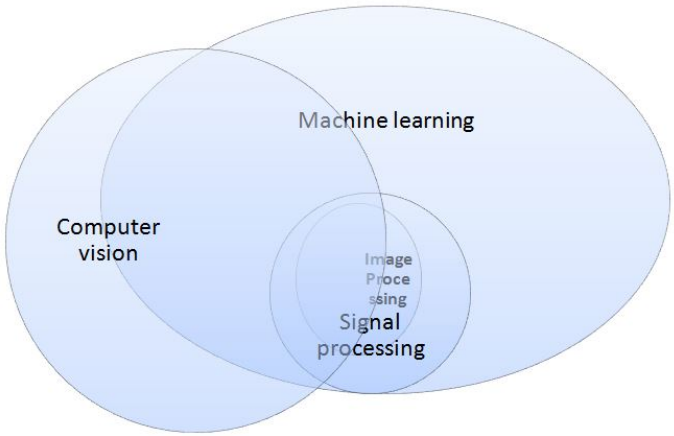


Figure 1: Venn diagram involving multiple domains.

Image processing

- **Input:** images.
- **Output:** images.
- **Methods:** filtering, contrast adjustment, etc.

Computer vision

- **Input:** images.
- **Output:** knowledge of the scene (stats about objects, people counting, etc).
- **Methods:** image processing, machine learning, etc.

Many images: A common equation

- A number of images come from an acquisition device,
- Despite the range of devices, any digital image $v : \mathbb{Z}^2 \to \mathbb{Z}$ can be written

$$v_{i,j} = \underbrace{Ar}_{\text{Quantification}} \Big(\underbrace{h*}_{\text{Averaging}} u_{i,j} \underbrace{\mathbf{1}_{[0,N]^2}}_{\text{Windowing}} \underbrace{(i,j)}_{\text{Sampling}} + \underbrace{b_{i,j}}_{\text{Noise}} \Big).$$

  - h is generally a translation invariant operator.
  - b is noise (e.g. assumed to be an additive white Gaussian noise).
  - Other degradations include geometrical distortions, partial image loss, etc.

A common notation

- Labeling problem aims at assigning a label $l \in \mathcal{L}$ to each site $p \in \mathcal{P}$ (mapping).
- The set of all possible labelings is the Cartesian product $\mathcal{F} = \underbrace{\mathcal{L} \times \ldots \times \mathcal{L}}_{\sharp\mathcal{P} \text{ times}} = \mathcal{L}^{\sharp\mathcal{P}}$.
- $\mathcal{F}$ is the space of configurations of admissible images.
- In an extreme case, we could even have $\mathcal{F} = \mathcal{L}_1 \times \mathcal{L}_2 \times \ldots \times \mathcal{L}_{\sharp\mathcal{P}}$.

Remarks

- Sites are image features on which we want to estimate some quantity.
- Labels are the quantity we want to estimate ($\mathcal{L}$ can be ordered).
- A label is assigned to every site but a label can be unassigned to any site.
- $\mathcal{L}$ can be discrete or continuous while $\mathcal{P}$ can be regular or irregular.

Examples of labeling problems

- Image segmentation: $\mathcal{P}$ is regular and $\mathcal{L}$ is discrete.
- Image restoration: $\mathcal{P}$ is regular and $\mathcal{L}$ is continuous.
- Object matching: $\mathcal{P}$ is irregular and $\mathcal{L}$ is discrete.
- Pose estimation: $\mathcal{P}$ is irregular and $\mathcal{L}$ is continuous.

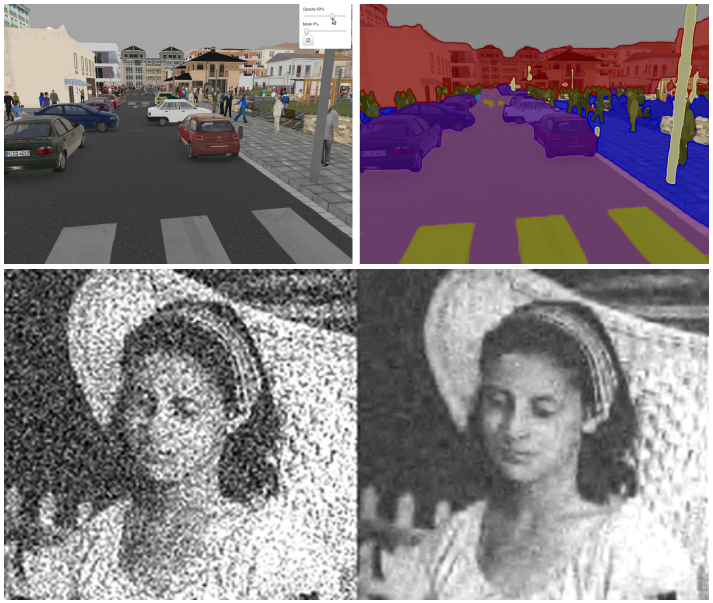


Figure 2: Illustration of common vision tasks: segmentation (top) and denoising (bottom).

Forward problem

- Deduce effects of a phenomenon from the knowledge of what causes it.
- "The same causes produce the same effects".

Inverse problem

- Determine what caused a phenomenon from the knowledge of its effects.
- "The same effects can from different causes" $\rightarrow$ much harder!
- Requires a good knowledge of the forward problem.

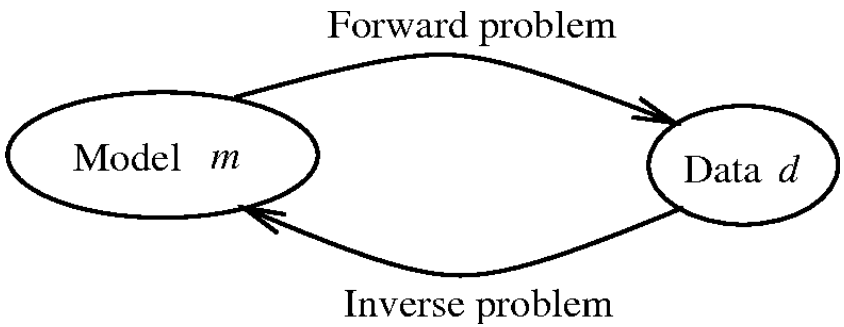


Figure 3: Forward and inverse problems.

Well-posed and ill-posed problems

In the sense of Hadamard (1923), a problem is said to be well-posed if

- The solution exists.
- The solution is unique.
- The solution continuously depend on input data.

Problems not well-posed are termed ill-posed. Inverse problems are often ill-posed.

Domains of application

- Medical imaging (echography, x-rays, scanners, etc.).
- Petroleum engineering (prospection by seismic/magnetic methods).
- Radar and submarine acoustics (determination of the shape of an obstacle).
- Autonomous vehicles (identify moving objects, estimate scene depth).

Energy minimization framework

A way to get a solution from an inverse problem is to reformulate it as an optimization problem. The optimal solution is chosen as the one yielding the minimum energy, i.e.

$$u^* \in \underset{u \in \mathcal{L}^{\mathcal{P}}}{\operatorname{argmin}} E(u) = \beta \underbrace{E_d(u)}_{\text{Data term}} + \underbrace{E_s(u)}_{\text{Reg. term}},$$

- u^* is called a minimizer of $E : \mathcal{L}^{\mathcal{P}} \to \mathbb{R}$.
- $E_d : \mathcal{L}^{\mathcal{P}} \to \mathbb{R}$ and $E_s : \mathcal{L}^{\mathcal{P}} \to \mathbb{R}$ depend on application and inputs.
- $\beta \in \mathbb{R}_{\geq 0}$ is a weighting parameter (regularizer).

Remarks

- $E(u)$ measures the quality of the solution u and can guide towards u^*.
- E must be designed s.t. a minimizer u^* provides expected outcome.
- Minimizing E is NP-hard in the general case.

Remarks about priors

- The shape of $E_s(u)$ determines the type of prior.
- Priors heavily depend on the problem and the targeted application.

Examples of priors

- **Everywhere smooth:** assigns low cost to everywhere smooth labelings:

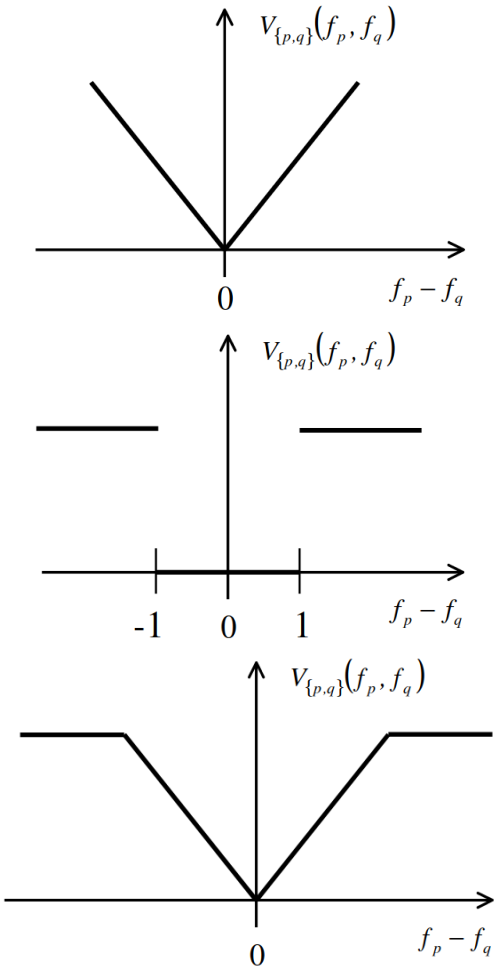
$$E_{p,q}(u_p, u_q) = |u_p - u_q|.$$

- **Piecewise constant:** assigns low cost to labelings with regions of constant labels:

$$E_{p,q}(u_p, u_q) = \mathbf{1}_{u_p \neq u_q}.$$

- **Piecewise smooth:** assigns low cost to labelings with regions of smoothly varying labels:

$$E_{p,q}(u_p, u_q) = \min\{|u_p - u_q|^\alpha, T\}, \quad \alpha \in \mathbb{R}_{\geq 0}, T \in \mathbb{R}_{\geq 0}.$$



(a) Prior shape

50	52	51	49
53	49	52	50
54	50	53	52
52	49	51	53

50	50	200	200
50	50	200	200
50	50	200	200
50	50	200	200

50	52	201	203
53	49	202	205
54	50	204	200
52	49	202	201

(b) Low cost labeling

50	52	201	203
53	49	202	205
54	50	204	200
52	49	202	201

50	52	51	49
53	49	52	50
54	50	53	52
52	49	51	53

150	82	150	49
63	249	30	250
254	98	53	95
72	149	151	53

(c) High cost labeling

Figure 4: Quantitative comparison of everywhere smooth (top row), piecewise constant (middle row) and piecewise smooth priors (bottom row) on toy examples (courtesy of [Vek99]).

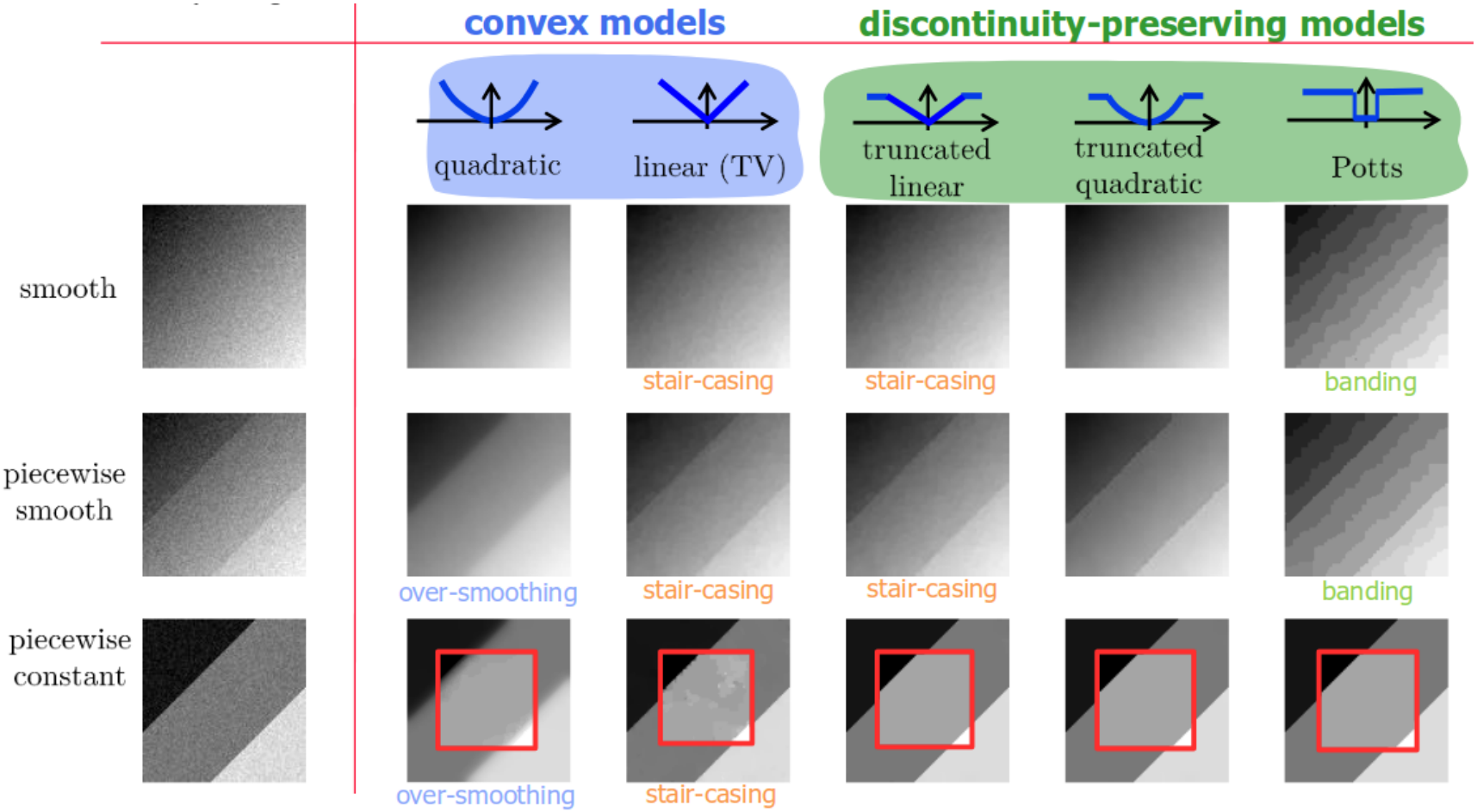


Figure 5: Qualitative comparison of pairwise potentials for image restoration on synthetic images.

Outline

1 Motivations

2 Bayes justification

3 Elements of optimization

4 Applications

5 Conclusion

Notations

- $\mathcal{P} \subset \mathbb{Z}^d$: set of sites ($d \in \mathbb{N}_{>0}$),
- $\mathcal{L}$: set of labels,
- $\mathcal{N} = \{\mathcal{N}_p \mid p \in \mathcal{P}\}$: neighboring relationships,
- $U = \{U_1, \ldots, U_{\sharp\mathcal{P}}\}$: random variables defined on $\mathcal{P}$ (U_p takes value $u_p \in \mathcal{L}$),
- $u = \{u_p \mid p \in \mathcal{P}\}$: a realization of the field U,
- Joint event $(U_1 = u_1, \ldots, U_{\sharp\mathcal{P}} = u_{\sharp\mathcal{P}})$ is abbreviated $U = u$.

Definition of an MRF

A field U is said to be an MRF if

- $P(u) > 0, \forall u \in \mathcal{L}^{\mathcal{P}}$ (positivity property),
- $P(u_p \mid u_{\mathcal{P}\setminus\{p\}}) = P(u_p | u_{\mathcal{N}_p})$ (Markovianity property).

Definition of cliques

- A clique C is a subset of $\mathcal{P}$ s.t. every pair of sites $(p, q) \in C$ are adjacent w.r.t. $\mathcal{N}$,
- The size of a clique C is the number of sites it contains,
- $\mathcal{C}_k$ denotes the set of cliques of size k and $\mathcal{C} = \cup_k \mathcal{C}_k$ the set of all cliques.

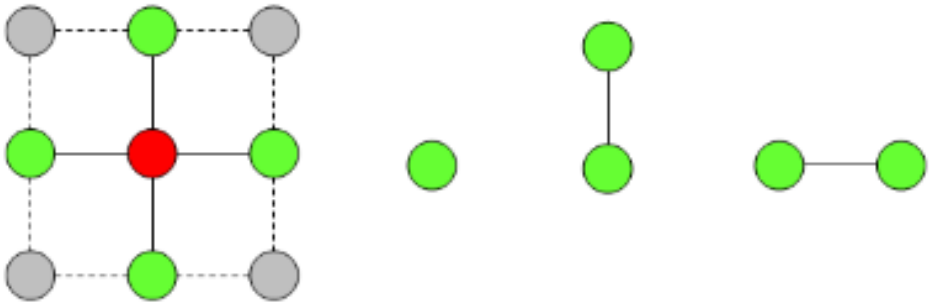


Figure 6: $\mathcal{C}_1$ cliques (green/red dots) and $\mathcal{C}_2$ cliques (red<->green relationships) in 4-connectivity.

Definition of an GRF

A field U is said to be a GRF on the sites $\mathcal{P}$ w.r.t. neighboring relationships $\mathcal{N}$ iff its configurations obey a Gibbs distribution of the form

$$P(u) = \frac{1}{Z} \exp\Big(-\beta \underbrace{\sum_{c \in \mathcal{C}} E_c(u)}_{E(u)} \Big), \quad \text{where}$$

- $Z = \sum_{u \in \mathcal{L}^{\mathcal{P}}} \exp\left(-E(u)\right)$ is a normalizing constant (partition function),
- $E_c(u)$ are clique potential functions (need to be defined),
- $\beta \in \mathbb{R}_{\geq 0}$ is a free parameter (inverse of the temperature in physics).

Remarks

- More probable labelings u are those with lower energies,
- The parameter β controls the sharpness of the Gibbs distribution.

Homogeneity and isotropy of GRFs

- **Homogeneity:** Functions $E_c(u)$ do not depend on the position of c,
- **Isotropy:** Functions $E_c(u)$ are independent of the orientation of c.

In words...

- An MRF is characterized by its local property (Markovianity),
- An GRF is characterized by its global property (Gibbs distribution).

Hammersley-Clifford theorem [HC71] (proof in [Bes74, KS80])

U is an MRF on $\mathcal{P}$ w.r.t. $\mathcal{N}$ iff U is an GRF on $\mathcal{P}$ w.r.t. $\mathcal{N}$.

$\rightarrow$ Joint probability $P(U = u)$ can be specified by defining the terms $E_c(u)$.

Bayes rule

The field U is generally not observable. It is common to estimate u based on empirical data d. A widespread approach is to estimate the MAP of U by maximizing

$$P(u|d) = \frac{P(d|u)P(u)}{P(d)} \propto P(d|u)P(u), \qquad \text{where,}$$

- $P(u|d)$: posterior probability,
- $P(u)$: prior probability,
- $P(d|u)$: likelihood,
- $P(d)$: evidence.

Prior probability $P(u)$

Since U is an MRF on $\mathcal{P}$ w.r.t $\mathcal{N}$, u follows a Gibbs distribution. By only considering cliques of size 2 (pairwise interactions), we have

$$P(u) \propto \exp\left(-\beta \sum_{(p,q)\in\mathcal{N}} E_{p,q}(u_p, u_q)\right).$$

Likelihood $P(d|u)$

Assuming conditional probabilities $P(d_p|u_p)$ mutually independent (this holds if d is corrupted with i.i.d. noise), we have

$$P(d|u) = \prod_{p\in\mathcal{P}} P(d_p|u_p) = \exp\left(-\sum_{p\in\mathcal{P}} -\log P(d_p|u_p)\right).$$

Maximum A Posteriori (MAP)

Given data d, the MAP estimate u^* of the field U is obtained by maximizing $P(u|d)$, i.e.

$$u^* \in \underset{u \in \mathcal{L}^{\mathcal{P}}}{\operatorname{argmax}} \left(\exp\left(-\beta \sum_{(p,q) \in \mathcal{N}} E_{p,q}(u_p, u_q) - \sum_{p \in \mathcal{P}} -\log P(d_p|u_p) \right) \right),$$

which is equivalent to (once negative logarithm is applied)

$$u^* \in \underset{u \in \mathcal{L}^{\mathcal{P}}}{\operatorname{argmin}} \left(\beta \sum_{(p,q) \in \mathcal{N}} E_{p,q}(u_p, u_q) + \sum_{p \in \mathcal{P}} -\log P(d_p|u_p) \right).$$

Remarks

- MAP estimator provides an estimate of the solution of an inverse problem,
- Bayes modeling provides a convenient way to model complex inverse problems,
- MAP is equivalent to maximum likelihood when $P(u)$ is “flat”.

Outline

1 Motivations

2 Bayes justification

3 Elements of optimization

4 Applications

5 Conclusion

What does “optimize” means?

Definition: Make the best or most effective use of a situation or resource.

Remarks

- Still an attractive area of research (500K results since 2015 with Google Scholar).
- Generally represents the last step when solving an inverse problem.
- Optimization algorithms must be selected with regard to the application.

General formulation

Given an Euclidean space X on $\mathbb{R}$ and a function $E : X \to \mathbb{R}$, let us consider

$$\min \ E(x), \quad \text{s.t.} \quad x \in X. \tag{1}$$

- **Input data:** generally a set of functions.
- **Variables:** their number is generally proportional to the image size.
- **Objective function:** E to be minimized.
- **Constraints:** a set of equations and/or inequalities (eventually empty).

Remarks

- E is generally assumed to be (twice) continously differentiable over X.
- The strategy for solving (1) both depend on the form of E and X.
- In what follows, we provide X with $\langle ., . \rangle$ and $\|x\|_2 = \sqrt{\langle x, x \rangle}$.

Fonction coût	Contraintes	Domaine X	Terminologie
Linéaire $f(x) = c^\top x, c \in \mathbb{R}^n$	linéaires	Polytope / Polyèdre	Programmation linéaire (P.L.)
Linéaire	linéaires	$X \subset \mathbb{Z}$	P.L. en nombres entiers
Quadratique $f(x) = c^\top x + x^\top Q x$ $c \in \mathbb{R}^n,\ Q \in \mathbb{S}_n$	linéaires	Polytope / Polyèdre	Programmation quadratique
Convexe	convexes	convexe	Programmation convexe
qq	qq	qq	Programmation non linéaire (P.N.L.)

Figure 7: Classification of optimization problems (courtesy of [Ron17]).

Important remarks

- Continuous and combinatorial optimization followed distinct trajectories.
- Interface between them became active due to complex problems & massive data.
- A combinatorial problem can be converted into a continuous one.
- A constrained problem can be converted into an unconstrained one.

PROGRAMMATION LINÉAIRE		
		Méthode du simplexe Algorithme des points intérieurs
PROGRAMMATION NON LINÉAIRE		
SANS CONTRAINTE	AVEC DÉRIVÉES	Méthodes de type gradient Méthodes de Newton et quasi-Newton Gauss-Newton, Levenberg-Marquardt (pbs de moindres carrés)
	SANS DÉRIVÉES	(DFO, NEWUOA, MADS, NOMADS,...) Méthodes heuristiques : Nelder-Mead, surfaces de réponses (réseaux de neurones, krigeage) Méthodes stochastiques : méthodes à 2 phases algos génétiques, algos évolutionnaires, recuit simulé, recherche tabou
	NON-DIFF.	Méthodes de sous-gradient Méthodes de faisceaux Méthodes d'échantillonnage de gradient Algorithmes proximaux
	$f(x) = g(x) + h(x)$ avec g convexe diff. h convexe non diff.	Algorithmes de splitting : descente de gradient proximale (+variante accélérée) Douglas Rachford, ADMM
AVEC CONTRAINTES	AVEC DÉRIVÉES	Gradient projeté, algorithme de Uzawa Méthode SQP (Newton) Points intérieurs Méthodes de pénalisation, Lagrangien augmenté

Figure 8: Classification of optimization algorithms (courtesy of [Ron17]).

Local minimum

$x^* \in X$ is a local minimum of a function E over X if there exists an $\epsilon > 0$ s.t.

$$E(x^*) \leq E(x), \quad \forall x \in X \text{ with } \|x - x^*\| < \epsilon.$$

Global minimum

$x^* \in X$ is a global minimum of a function E over X if

$$E(x^*) \leq E(x), \quad \forall x \in X.$$

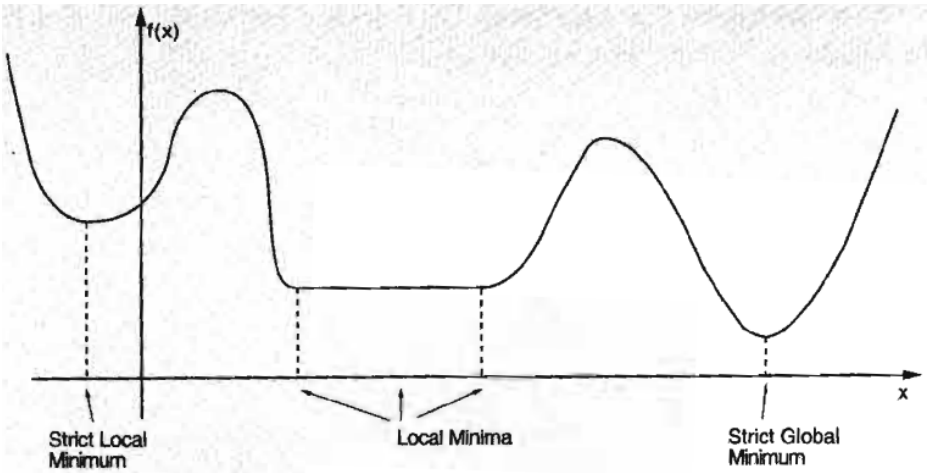


Figure 9: Local and global minima in one dimension (courtesy of [Ber99]).

Remarks

- Any global minimum is a local minimum too.
- Maximizing the function E is equivalent to minimize $-E$.
- Local and global maxima can be defined in a similar way.

Gradient

Let $X = \mathbb{R}^n$. The gradient of a differentiable function E at $x \in X$ is

$$\nabla E(x) = \left(\frac{\partial E}{\partial x_i}(x) \right)_{1 \leq i \leq n} \in \mathbb{R}^n.$$

Hessian matrix

Let $X = \mathbb{R}^n$. The Hessian matrix of a twice differentiable function E at $x \in X$ is

$$H_E(x) = \nabla^2 E(x) = \left(\frac{\partial^2 E}{\partial x_i \partial x_j}(x) \right)_{1 \leq i,j \leq n} \in \mathbb{R}^{n \times n}.$$

Reminders

- E is said to be differentiable over X if $\nabla E(x)$ exists, $\forall x \in X$.
- E is said to be continuously differentiable if $\nabla E(x)$ is a continuous function of x.
- E function E is said to be twice differentiable over X if $H_E(x)$ exists, $\forall x \in X$.
- E said to be twice continuously differentiable if $H_E(x)$ is a continuous function of x.

Remarks

- H_E is a symmetric matrix (differentiation order does not matter).
- Gradient is less costly to store than Hessian matrix.

Definiteness of a symmetric matrix $A \in \mathbb{R}^{n \times n}$

Let us denote by $(\lambda_i)_{i=1}^n$ the eigenvalues of A. A is said to be

- **Positive definite (PD)** iff $(\lambda_i)_{i=1}^n$ are positive.
- **Positive semi-definite (PSD)** iff $(\lambda_i)_{i=1}^n$ are non-negative.
- **Negative definite (ND)** iff $(\lambda_i)_{i=1}^n$ are negative.
- **Negative semi-definite (NSD)** iff $(\lambda_i)_{i=1}^n$ are non-positive.
- **Indefinite** iff $(\lambda_i)_{i=1}^n$ are both positive and negative.

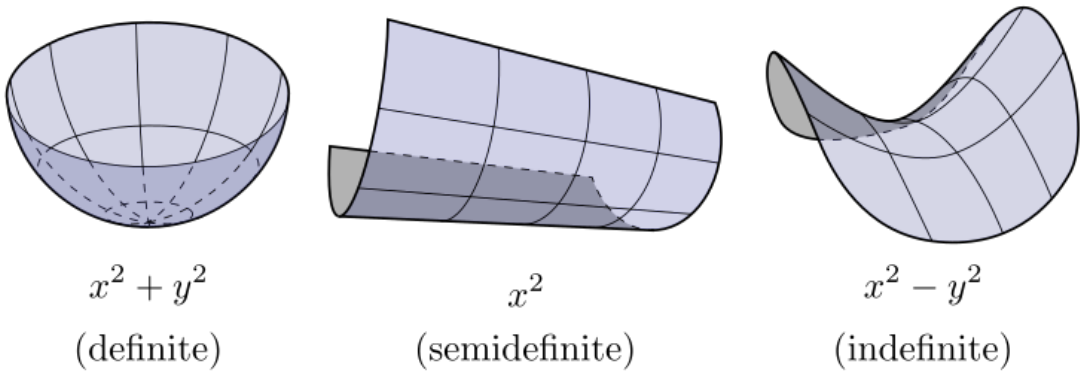


Figure 10: Definiteness of H_E for distinct E.

Convex set

A set $A \subset X$ is said to be convex if

$$\forall x, y \in X, \quad \forall \lambda \in [0,1] \quad \Rightarrow \quad (\lambda x + (1-\lambda)y) \in X.$$

Convex function

A function E is said to be convex over X if X is convex and

$$\forall x, y \in X, \quad \forall \lambda \in [0,1], \quad E((1-\lambda)x + \lambda y) \leq (1-\lambda)E(x) + \lambda E(y).$$

Strictly convex function

A function E is said to be strictly convex over X if X is convex and

$$\forall x, y \in X, \quad x \neq y, \quad \forall \lambda \in [0,1], \quad E((1-\lambda)x + \lambda y) < (1-\lambda)E(x) + \lambda E(y).$$

Concave function

A function E is said to be concave over X if $-E$ is convex.

Strictly concave function

A function E is said to be strictly concave over X if $-E$ is strictly convex.

Important remarks

- Any strictly convex function is convex too.
- A linear combination of convex functions is convex.
- Affine transformations preserve convexity.
- The sum of a strictly convex function and a convex one is strictly convex.
- Pointwise maximum of convex functions is convex.
- L^p norm are convex functions (except L^0 that is neither convex or a norm).
- Indicator function of a convex set is convex.

Examples

- Empty set $\emptyset$ and X are both convex sets.
- Unit ball $\{x \in X \mid \|x\|_2 \leq 1\}$ is a convex set.
- Any polyhedron $\{x \in X \mid Ax \leq b\}$ is a convex set.
- Linear functions $x \mapsto Ax + b$ are both convex and concave.

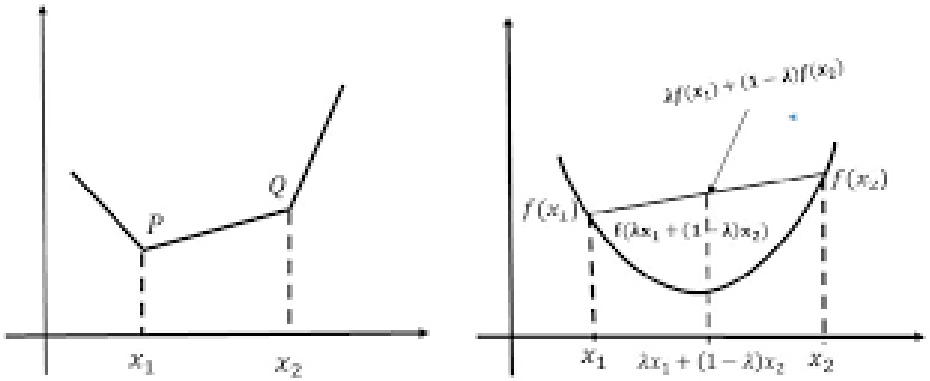


Figure 11: A convex function (left) and a strictly convex one (right). Courtesy of [Raj18].

Condition number

The condition number of $E(x) = Ax - b$ measures how much the output value change for a small change in the input. When A is a real symmetric matrix, it is defined by

$$\kappa(A) = \frac{|\lambda_{max}|}{|\lambda_{min}|} \in [1, +\infty[.$$

where λ_{min} and λ_{max} denote resp. minimum and maximum eigenvalues of A.

- $\kappa(A) \simeq 1$: well-conditioned operator A.
- $\kappa(A) \gg 1$: ill-conditioned operator A.

Remark

Ill-conditioned problems are generally harder to solve.

Example

Let us consider $E_\alpha(x) = \frac{1}{2}\|D_\alpha x\|_2^2$ with $\alpha \geq 1$ and

$$D_\alpha = \begin{pmatrix} 1 & 0 \\ 0 & \alpha \end{pmatrix}.$$

Since $H_E(x) = D_\alpha$ for any $x \in X$, we have $\kappa(H_E) = \alpha \in [1, +\infty[$.

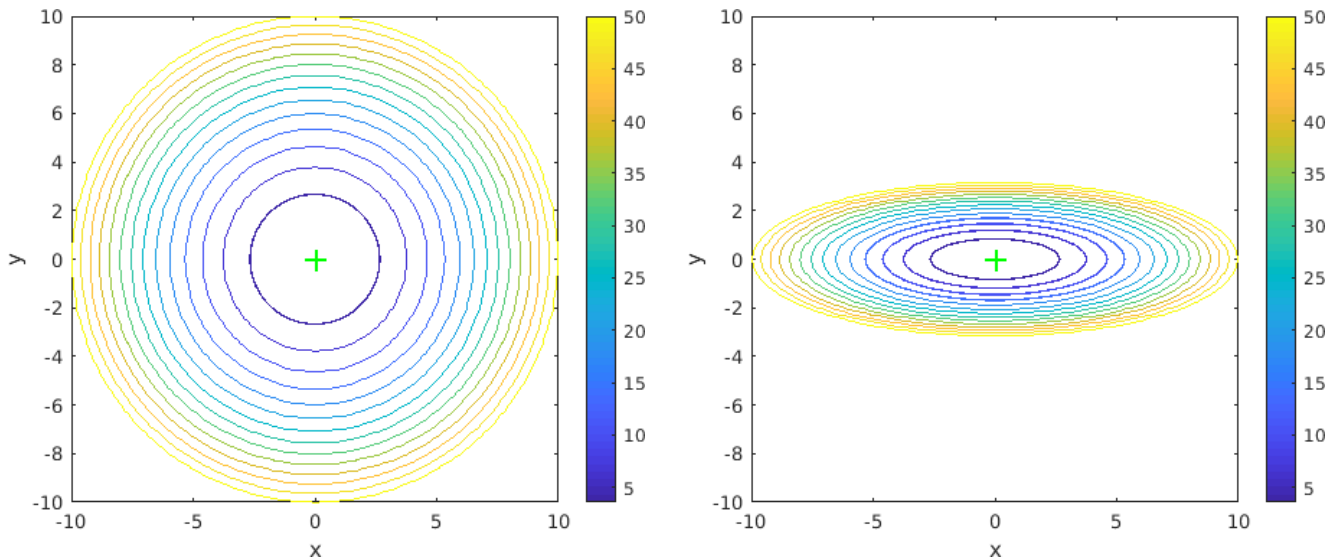


Figure 12: From left to right: A well-conditioned problem ($\alpha = 1$) and an ill-conditioned one ($\alpha = 10$). The green cross denotes the location of the unique global minimizer of E_α.

Existence of minimizer (theorems)

- If E is continuous and $X \subset \mathbb{R}^n$ is compact [a], (1) admits at least one minimizer.
- If E is continuous, coercive and X is closed, (1) admits at least one minimizer.

[a] A set is compact if it is both closed and bounded.

Unicity of minimizer (theorems)

- If X is convex and E is convex, (1) admits at least one minimizer.
- If X is convex and E is strictly convex, (1) admits at most one minimizer.

Remark

Above theorems do not characterize analytically solutions of E!

Necessary conditions of local optimality

- **First order:** If E is cont. diff. over X and $x^* \in X$ is a local min., then $\nabla E(x^*) = 0$.
- **Second order:** If E is twice cont. diff. over X and $x^* \in X$ is a local minimum (resp. maximum): $H_E(x^*)$ is PSD (resp. NSD).

Sufficient conditions of local optimality

Let us assume that E is twice continuously differentiable over X and let $x^* \in X$.

- If $\nabla E(x^*) = 0$ and $H_E(x^*)$ is PD, then x^* is a local minimum.
- If $\nabla E(x^*) = 0$ and $H_E(x^*)$ is ND, then x^* is a local maximum.

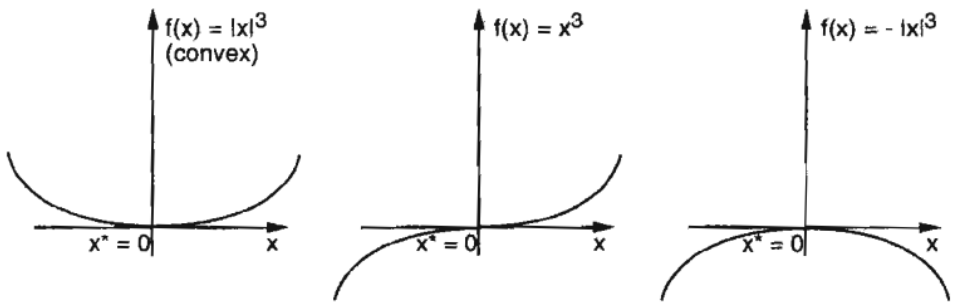


Figure 13: Illustration of necessary and sufficient optimality conditions (courtesy of [Ber99]).

Stationary/critical point

Let us assume that E is cont. diff. at $x \in X$. x is said to be a stationary/critical point if

$$\nabla E(x) = 0.$$

Saddle point

Let us assume that E is twice continuously differentiable at $x \in X$. x is said to be a saddle point if it is a stationary point and $H_E(x)$ is indefinite.

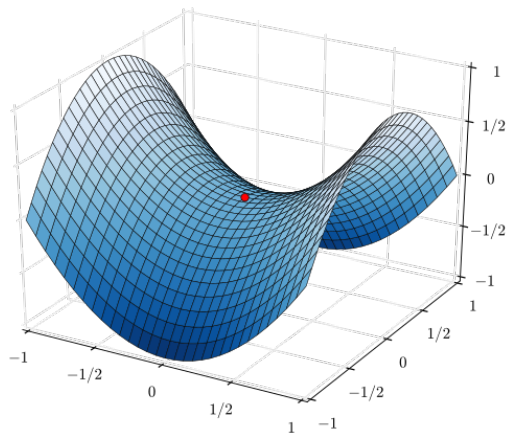


Figure 14: An example of a saddle point (red).

Naive minimization procedure

1. Find stationary points $x \in X$ (i.e. points s.t. $\nabla E(x) = 0$).
2. Filter out points $x \in X$ s.t. $H_E(x)$ is not PSD.
3. Local minima are points $x \in X$ s.t. $H_E(x)$ is PD.

Remarks

- Such an approach is generally impractical on a modern computer.
- Better idea: Iteratively minimize E from a given starting point.

Directional derivative

Let E cont. diff. over X and let $x \in X$. For any $d \in X$ s.t. $d \neq \vec{0}$, $\langle \nabla E(x), d \rangle$ is the slope of E at x in the direction d. The sign of $\langle \nabla E(x), d \rangle$ inform us about the local structure of E:

- If $\langle \nabla E(x), d \rangle > 0$, E increases in the direction d.
- If $\langle \nabla E(x), d \rangle < 0$, E decreases in the direction d (descent direction).
- If $\langle \nabla E(x), d \rangle = 0$, we cannot conclude.

If d is a descent direction, there exists $\epsilon > 0$ s.t. $E(x + td) < E(x)$, $\forall t \in]0, \epsilon]$.

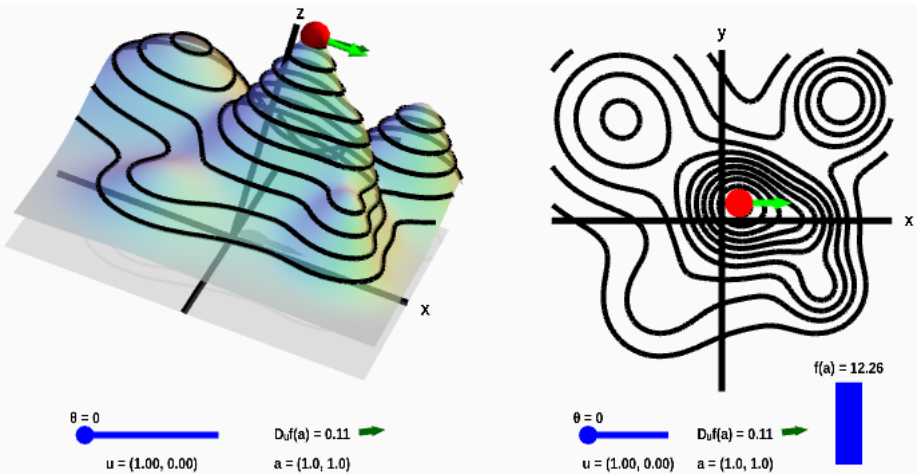


Figure 15: Let's play with directional derivatives! (courtesy of `https://bit.ly/2y24UqE`).

Remarks

- The red point a is the location where the directional derivative is evaluated.
- The directional derivative is computed in the direction of the vector u (light green).
- The dark green vector v emanates from a in the direction u tangent to f.
- The value of the directional derivative is the slope of v ($D_u f(a)$).

Questions

- If u point in a direction parallel to a level curve, what happens to $D_u f(a)$?
- For any u, what happens to $D_u f(a)$ when it point to the opposite direction?
- What happens to the gradient at the tops of the “mountains”?

Algorithm 1: Unconstrained general descent algorithm (e.g. $X = \mathbb{R}^n$)

INPUTS: A (twice) continuously differentiable E, an initial guess $x_0 \in X$
OUTPUTS: An approximate minimizer of E

$k \leftarrow 0$
do
    Compute a descent direction d_k s.t. $\langle \nabla E(x_k), d_k \rangle < 0$
    Select a step size $t_k \in \mathbb{R}_{>0}$ in the direction d_k s.t. $E(x_k + t_k d_k) < E(x_k)$
    Update: $x_{k+1} \leftarrow x_k + t_k d_k$, $k \leftarrow k + 1$
until convergence
return x_k

Requirements of the algorithm

- **Initialization:** how to choose the initial guess x_0?
- **Descent direction:** how to choose the descent direction d_k?
- **Step size rule:** how to choose the step size t_k?
- **Convergence:** what stopping criterion to be used?

Initial guess x_0

- x_0 impacts the convergence speed (less iterations is better).
- x_0 does not always impact the convergence (e.g. when E is convex).
- No general purpose solution (depend on the targeted application).

Stopping criteria

Given some desired accuracy $\epsilon > 0$, multiple criteria can be combined:

- **Stagnation of solution:** $\|x_{k+1} - x_k\|_2 < \epsilon \max(1, \|x_k\|_2)$.
- **Stagnation of E:** $|E(x_{k+1}) - E(x_k)| < \epsilon \max(1, |E(x_k)|)$.
- **Number of iterations N:** bounded from above by N.

$\rightarrow$ A high level of accuracy is generally not required in vision robotics.

Descent direction d_k

- **Gradient direction (first order):** we set $d_k = -\nabla E(x_k)$.
  - Since $\langle \nabla E(x_k), -\nabla E(x_k) \rangle = -\|\nabla E(x_k)\|^2 < 0$, d_k is a descent direction.
  - $d_k = -\nabla E(x_k)/\|\nabla E(x_k)\|$ is the direction of most rapid decrease.
- **Newton's direction (second order):** we set $d_k = -H_E(x_k)^{-1}\nabla E(x_k)$.
  - $\langle \nabla E(x_k), -H_E(x_k)^{-1}\nabla E(x_k) \rangle < 0$ holds if $H_E(x_k)^{-1}$ is PD. Since $H_E(x_k)$ is PD, $H_E(x_k)^{-1}$ is PD too. Hence, d_k is a descent direction.

What is the direction of steepest ascent?

The unit direction of most rapid decrease is the minimizer of

$$\min_{d_k \in X} \langle \nabla E(x_k), d_k \rangle, \quad \text{s.t.} \quad \|d_k\|_2 = 1.$$

Since $\langle \nabla E(x_k), d_k \rangle = \|d_k\|_2 \|\nabla E(x_k)\|_2 \cos(\theta) = \|\nabla E(x_k)\|_2 \cos(\theta)$, one ends up with

$$d_k = -\frac{\nabla E(x_k)}{\|\nabla E(x_k)\|_2}.$$

Normalizing gradient descent

Since $t_k \propto \nabla E(x_k)$, gradient descent algorithm suffers of poor convergence near stationary points. By normalizing out the magnitude of the gradient, one ends up with

$$\begin{aligned} \|x_{k+1} - x_k\|_2 &= \left\| \left(x_k - t_k \frac{\nabla E(x_k)}{\|\nabla E(x_k)\|_2} \right) - x_k \right\|_2 \\ &= \left\| -t_k \frac{\nabla E(x_k)}{\|\nabla E(x_k)\|_2} \right\|_2 \\ &= t_k. \end{aligned}$$

- A small constant ϵ (e.g. 10^{-7}) must be added to avoid division by zero.
- Trades shorter initial steps for longer ones around stationary points.

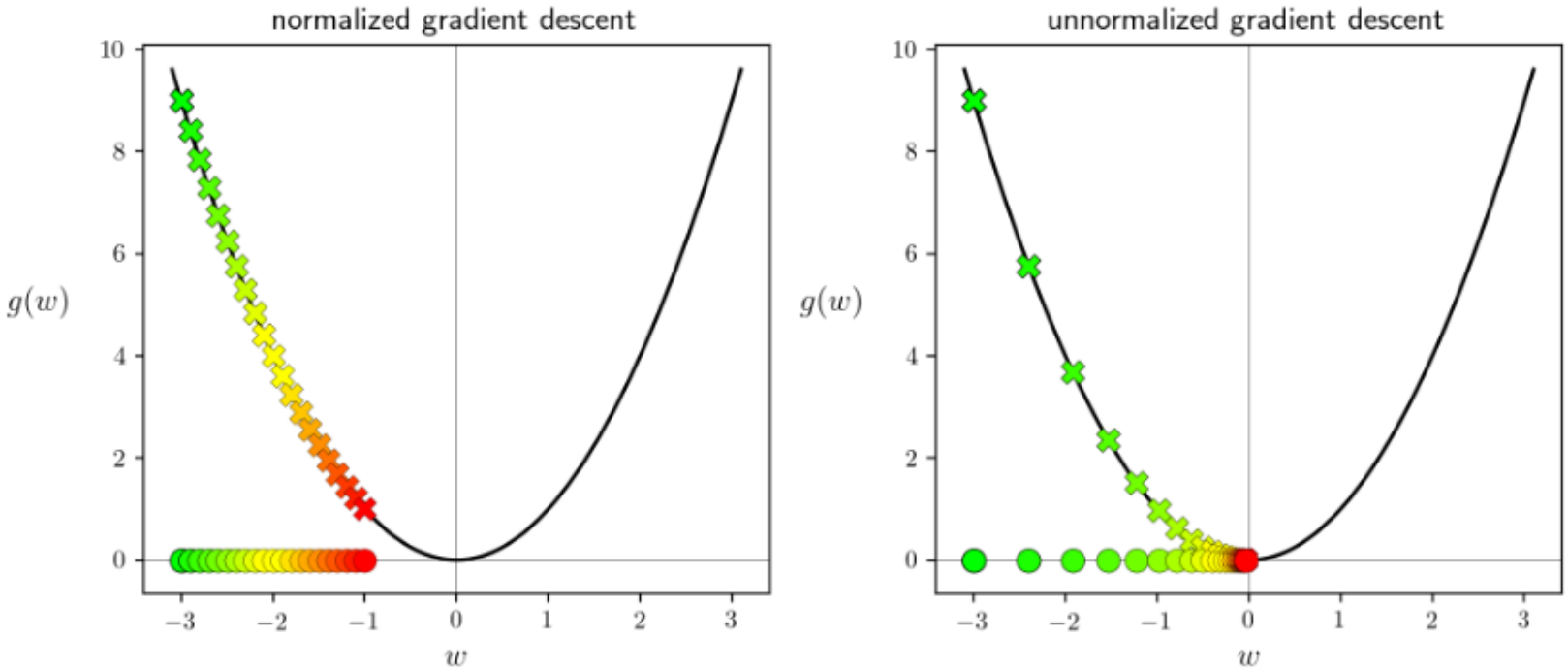


Figure 16: Comparison between normalized and unnormalized gradient descent for $g(w) = w^2$ with $w_0 = -3$ and step size $t_k = 0.1$, for any $k \in \mathbb{N}$ (courtesy of `https://tinyurl.com/z5m4sko8`).

Step size t_k

- For any $k \in \mathbb{N}$, we want to find a global minimizer of the univariate function

$$t_k \mapsto E(x_k + t_k d_k).$$

- Good news: Resolution of a much easier optimization problem.
- Bad news: Finding such a minimizer is generally expensive.
- $\rightarrow$ Tradeoff between substantial reduction of E and cost for computing t_k.

Constant step size rule

- For any $k \in \mathbb{N}$, we set $t_k = \bar{t} \in \mathbb{R}_{\geq 0}$.
- Straightforward to implement since no knowledge about E is required.
- Can cause divergence ($\bar{t}$ is too large) or slow convergence ($\bar{t}$ is too small).

Binary search rule

For any $k \in \mathbb{N}$, let us define $h(t) = E(x_k + td_k)$ and $h'(t) = \langle \nabla E(x_k + td_k), d_k \rangle$.

1. Let $i \leftarrow 0$, $t_l \leftarrow 0$ and $t_u \leftarrow t_{max}$.
2. Repeat until $i \geq N$
   1. Let $t_m \leftarrow \frac{t_l + t_u}{2}$.
   2. If $h'(t_m) < 0$, let $t_l \leftarrow t_m$ and $i \leftarrow i + 1$.
   3. If $h'(t_m) > 0$, let $t_u \leftarrow t_m$ and $i \leftarrow i + 1$.
   4. If $h'(t_m) = 0$, stop.

Remarks

- Not difficult to implement as soon as we are able to compute $h'(t)$.
- Bisection rule is accurate but may be expensive in practice.
- At each iteration $i \in \mathbb{N}$, the length of the interval $[t_l, t_u]$ is $\left(\frac{1}{2}\right)^k t_{max}$.
- To achieve some tolerance ϵ, the number of iterations is bounded by $\left\lceil \log_2 \left(\frac{t_{max}}{\epsilon} \right) \right\rceil$.

Armijo-Goldstein step size rule

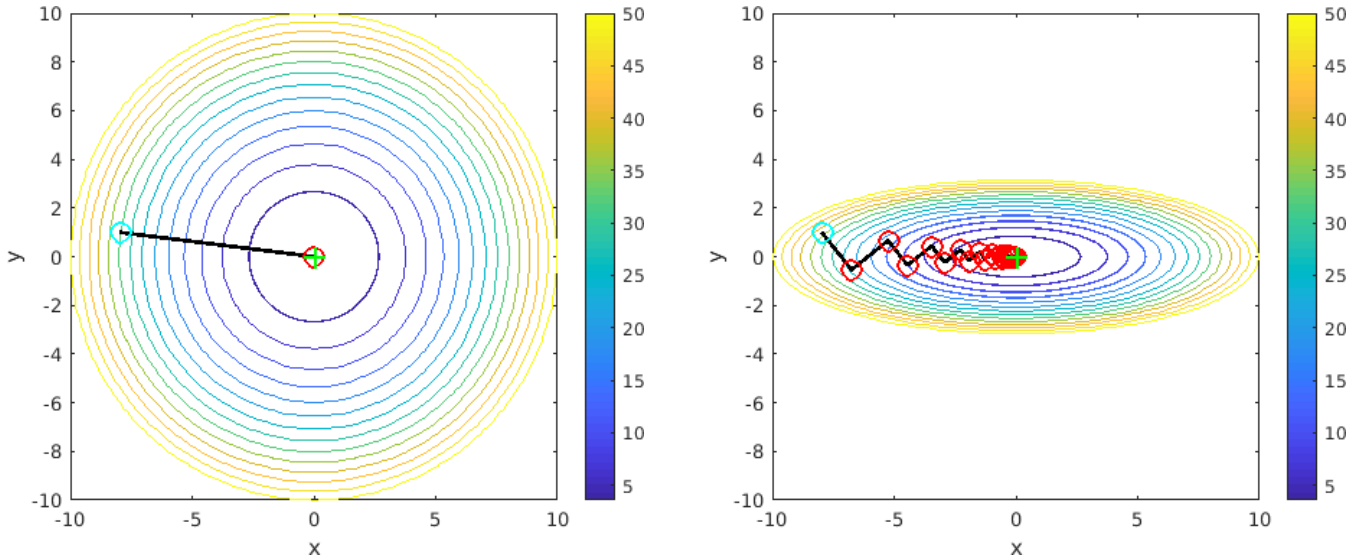
- Accept a suboptimal step size t_k while ensuring a sufficient decrease of E.
- For any $k \in \mathbb{N}$, t_i is selected as the largest step size satisfying

$$\underbrace{E(x_k) - E(x_k + t_i d_k)}_{Actual\ reduction} \geq \sigma t_i \times \underbrace{\langle \nabla E(x_k), d_k \rangle}_{Predicted\ reduction}, \quad \text{where,}$$

- $t_i = \alpha^i t_{max}$ are subsequent step sizes.
- $t_{max} \in \mathbb{R}_{\geq 0}$ denotes the maximum step size.
- $\alpha \in]0, 1[$ and $\sigma \in]0, 1[$ are free parameters.

Remarks

- For any $i \in \mathbb{N}$, a new evaluation of E is required.
- Straightforward to implement as soon as we are able to compute $E(x_k + t_k d_k)$.



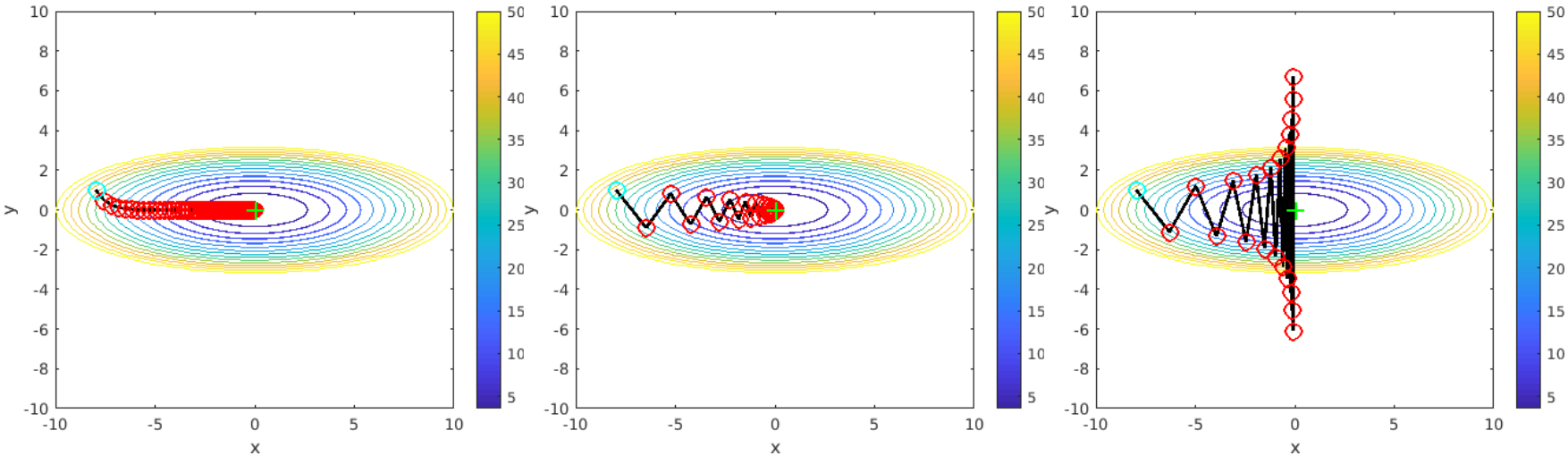
$\alpha = 1$ $\alpha = 10$

Figure 17: Gradient descent over E_α using $\epsilon = 10^{-4}$ and $x_0 = (-8, 1)^T$. Optimal step size is selected at each iteration.

Remark (to do in class)

Let $X_k = (x_k, y_k) \in \mathbb{R}^2$. At each iteration $k \in \mathbb{N}$, the optimal step size is given by

$$t_k^* = \underset{t \in \mathbb{R}_{\geq 0}}{\operatorname{argmin}} E_\alpha(X_k + t d_k) = \begin{pmatrix} \frac{x_k^2 + \alpha^2 y_k^2}{x_k^2 + \alpha^4 y_k^2} \end{pmatrix}.$$



$\overline{t} = 0.05$ (101 iterations) $\overline{t} = 0.19$ (44 iterations) $\overline{t} = 0.21$ (diverge)

Figure 18: Gradient descent over $E_{\alpha=10}$ using $\epsilon = 10^{-4}$, $x_0 = (-8, 1)^T$ and constant step size.

Convergence with gradient descent

- Under particular assumptions, it converges linearly when close enough to x^*.
- Better convergence rate can be achieved (Nesterov or accelerated gradient).

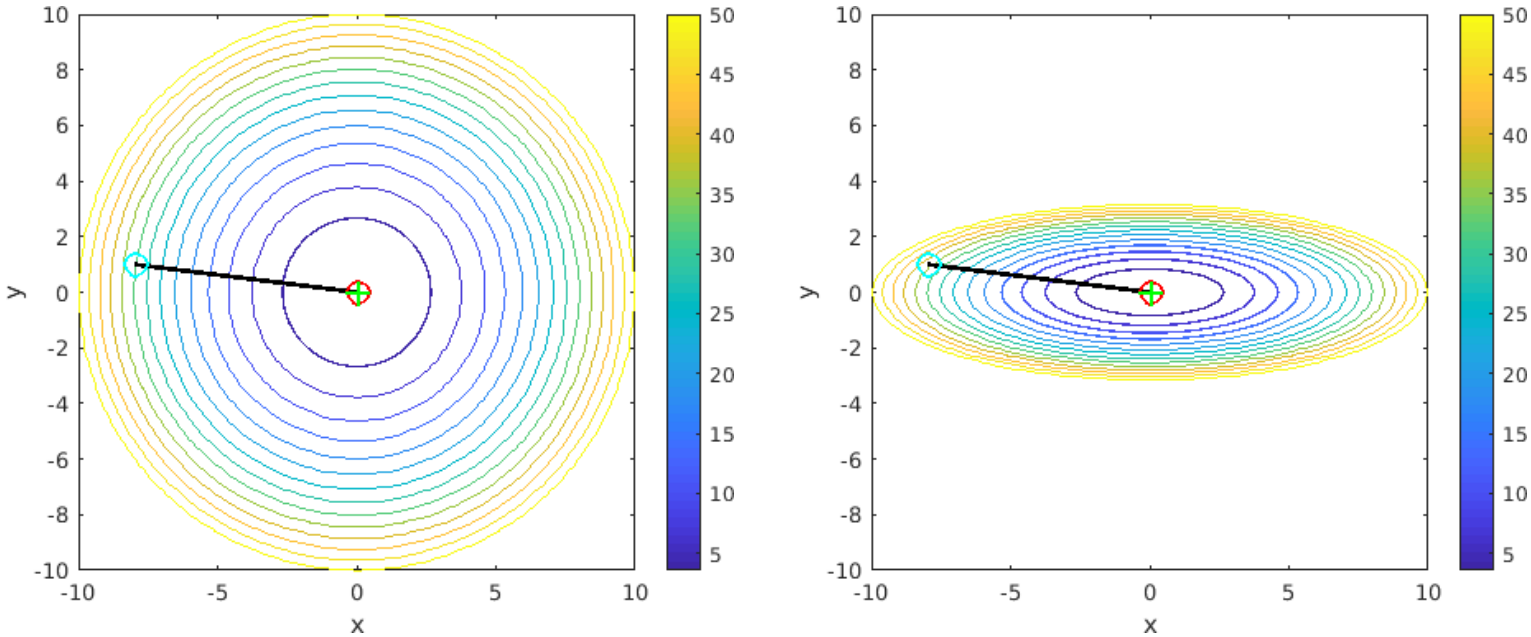
Gradient and level sets (proof in class)

Let us assume that E is cont. diff. over X. For any $c \in \mathbb{R}$, let us consider the parametric curve $x(t)$ of the c-level line. For any t, $\nabla E(x)$ is orthogonal to $x'(t)$: $\langle \nabla E(x), x'(t) \rangle = 0$.

Zig-zag behavior with gradient descent (proof in class)

Let $(x_k)_{k \in \mathbb{N}}$ be the sequence generated by the steepest gradient algorithm with exact line search for minimizing a cont. diff. E. For any $k \in \mathbb{N}$, we have

$$\langle (x_{k+2} - x_{k+1}), (x_{k+1} - x_k) \rangle = 0.$$



$\alpha = 1$ $\alpha = 10$

Figure 19: Newton's descent over E_α using unit step size, $\epsilon = 10^{-4}$ and $x_0 = (-8, 1)^T$.

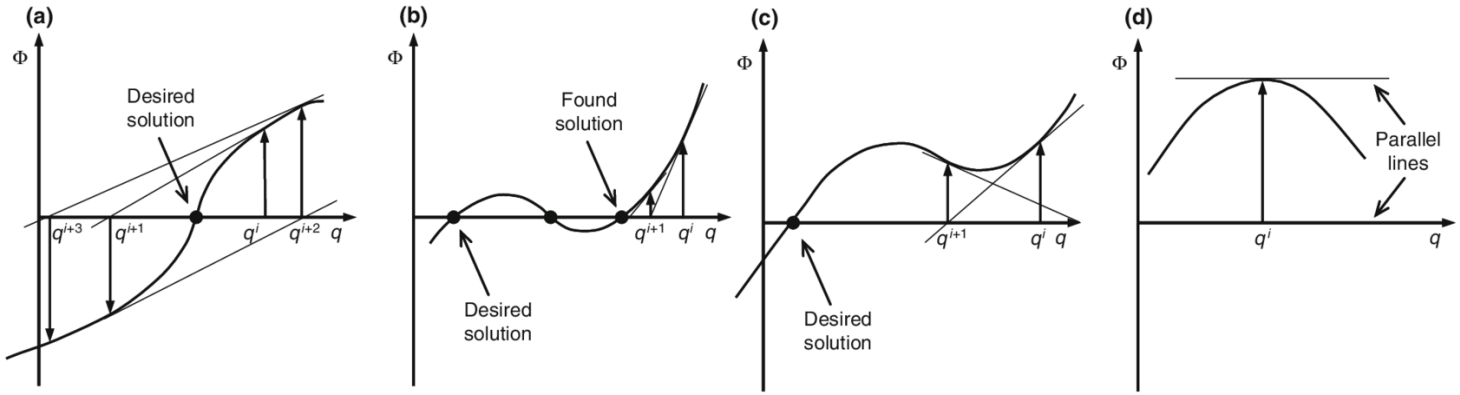
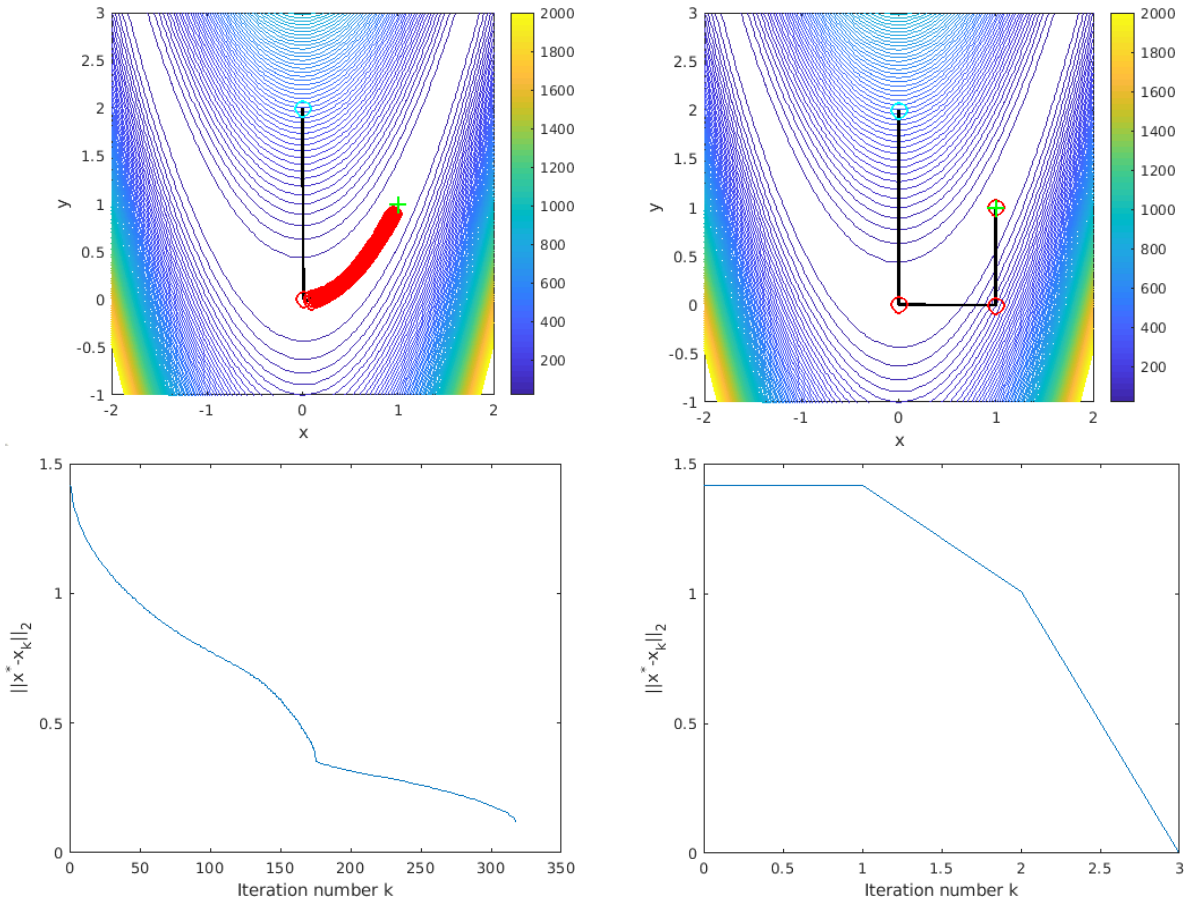


Figure 20: Typical failure cases of Newton's method.

Remarks about Newton's method

- Fits a paraboloid to E and finds minimum of it at each iteration.
- Descent method with implicit unit step size.
- Under particular assumptions, it converges quadratically.
- Computing $H_E(x)$ can be costly / $H_E(x)$ can be unavailable $\rightarrow$ BFGS.
- Even worse: the method can diverge in some situations.



Gradient descent | Newton's descent

Figure 21: Level lines (top row) and distance to the global minimizer $x^* = (1,1)^T$ (bottom row) for the Rosenbrock function $(x,y) \mapsto (1-x)^2 + 100(y-x^2)^2$ using $x_0 = (0,2)^T$ and $\epsilon = 10^{-4}$. Golden search and parabolic interpolation are used for selecting step sizes in gradient descent.

What about constrained optimization problems?

- Projected gradient descent is a way to solve constrained problems.
- Idea: If a point $x_k - t_k \nabla E(x_k)$ leaves X, we project it back onto X:

$$x_{k+1} = P_X(x_k - t_k \nabla E(x_k)),$$

where X is assumed to be a non-empty closed convex set, P_X is the proj. operator

$$P_X(x_0) = \operatorname*{argmin}_{x \in X} \frac{1}{2} \|x - x_0\|_2^2.$$

- Resulting problem is convex but projection can be delicate.

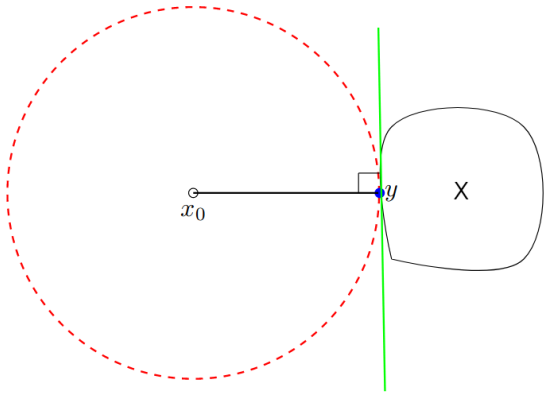


Figure 22: Projection of a point x_0 onto the (convex) constraint set X.

What about non-differentiable E?

- Subgradient methods are iterative schemes for solving convex min. problems.
- These methods are convergent when applied even to a non-differentiable E.
- Originally designed in the 60s and 70s.
- Extension of the notion of gradient to non-differentiable E.

Subgradient

Let $E : X \to \mathbb{R}$ and $x \in X$. The subgradient of E at x is any $g \in X$ s.t.

$$E(y) \geq E(x) + g^T(y - x), \quad \forall y \in X.$$

$\to$ Subgradient provides a linear global underestimator of E at x.

Subdifferential

- Let $E : X \to \mathbb{R}$ and $x \in X$.
- The set of subgradients of E at x is the subdifferential of E at x, denoted by $\partial E(x)$.
- Characterize minimizers of convex E.
- Any point $x^* \in X$ is a global minimizer of E iff $0 \in \partial E(x^*)$.

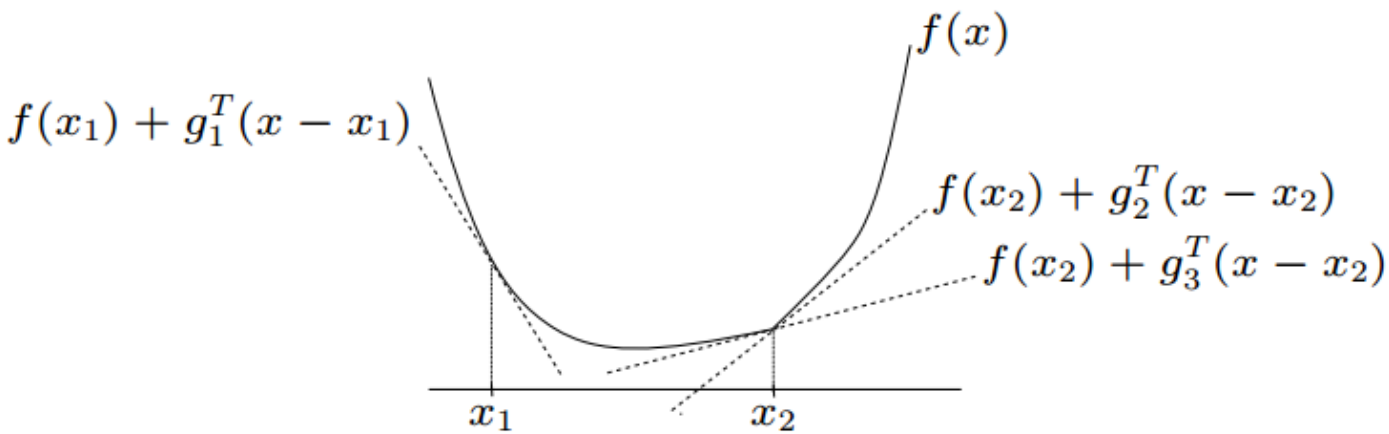


Figure 23: Illustration of subgradients of a non-differentiable convex E.

Algorithm 2: Classical subgradient algorithm (e.g. $X = \mathbb{R}^n$)

INPUTS: A convex $E : \mathbb{R}^n \to \mathbb{R}$, an initial guess $x_0 \in X$
OUTPUTS: An approximate minimizer of E

$k \leftarrow 0$, $E_{best} = E(x_0)$, k_{best}
do
    Compute subgradient $g_k \in \partial E(x_k)$
    Select a step size $t_k \geq 0$ in the direction $-g_k$ s.t. $E(x_k - t_k g_k) < E(x_k)$.
    Update: $x_{k+1} \leftarrow x_k - t_k \frac{g_k}{\|g_k\|_2}$, $k \leftarrow k + 1$
    if $E_{best} < E(x_{k+1})$ **then**
        $E_{best} \leftarrow E(x_{k+1})$
        $k_{best} \leftarrow k + 1$
    end-if
until convergence
return $x_{k_{best}}$

Requirements of the algorithm

- **Initialization:** how to choose the initial guess x_0?
- **Descent direction:** how to choose the descent direction g_k?
- **Step size rule:** how to choose the step size t_k?
- **Convergence:** what stopping criterion to be used?

Initial guess x_0

Same answers as for standard descent algorithm.

Descent direction $-g_k$

- Subgradient of E at any $x_k \in X$.
- CAUTION: It may happen that $-g_k$ is not a descent direction for E at x_k...

Step size rules

- Constant step size $t_k = \bar{t}$.
- Non-summable diminishing step sizes (convergence to x^* is ensured):

$$\lim_{k \to +\infty} t_k = 0, \quad \sum_{k=1}^{+\infty} t_k = +\infty.$$

Stopping criterion

Number of maximum iterations: $k < N$.

Example 1: Fermat-Weber problem (1/3)

For positive integers $d > 0$ and $n > 0$, a set of points $(y_i)_{i=1}^n \in \mathbb{R}^{n \times d}$ and a set of positive weights $(w_i)_{i=1}^n \in \mathbb{R}^n_{>0}$, we want to solve

$$x^* \in \operatorname*{argmin}_{x \in \mathbb{R}^d} E(x) = \sum_{i=1}^n w_i \|x - y_i\|_2. \tag{2}$$

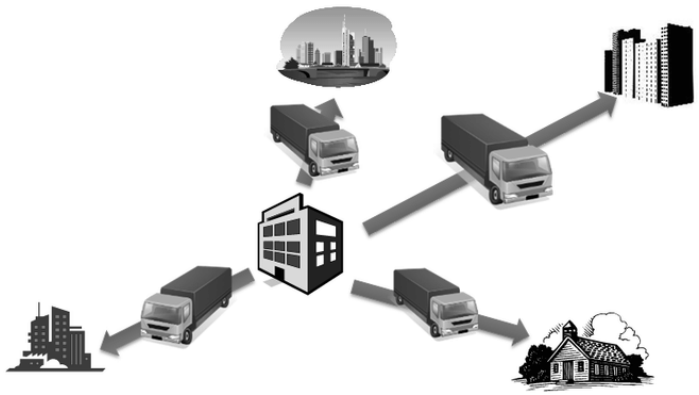


Figure 24: Fermat-Weber problem viewed as a facility location problem.

Example 1: Fermat-Weber problem (2/3)

- Problem proposed by Fermat in the 17th century for $n = 3$, $d = 2$ and unit weights.
- Geometrical solution from Torricelli for $n = 3$, $d = 2$ and unit weights.
- Algorithm proposed by Kuhn and Kuenne in 1962 for any n, d and w [KK62].

Example 1: Fermat-Weber problem (3/3)

If the points $(y_i)_{i=1}^n$ are not colinear, (2) is strictly convex and has a unique solution. The minimizer x^* of (2) can be obtained by gradient descent with $x_0 \neq (y_i)_{i=1}^n$ [a] and

$$x_{k+1} = x_k - t_k \nabla E(x_k),$$

where

$$\nabla E(x_k) = \sum_{i=1}^n w_i \left(\frac{x_k - y_i}{\|x_k - y_i\|_2} \right), \quad \text{and} \quad t_k = \frac{1}{\sum_{i=1}^n \frac{w_i}{\|x_k - y_i\|_2}}.$$

$\rightarrow$ Particular case where step size t_k is given in closed-form.

[a] This choice ensures that E is differentiable at x_0.

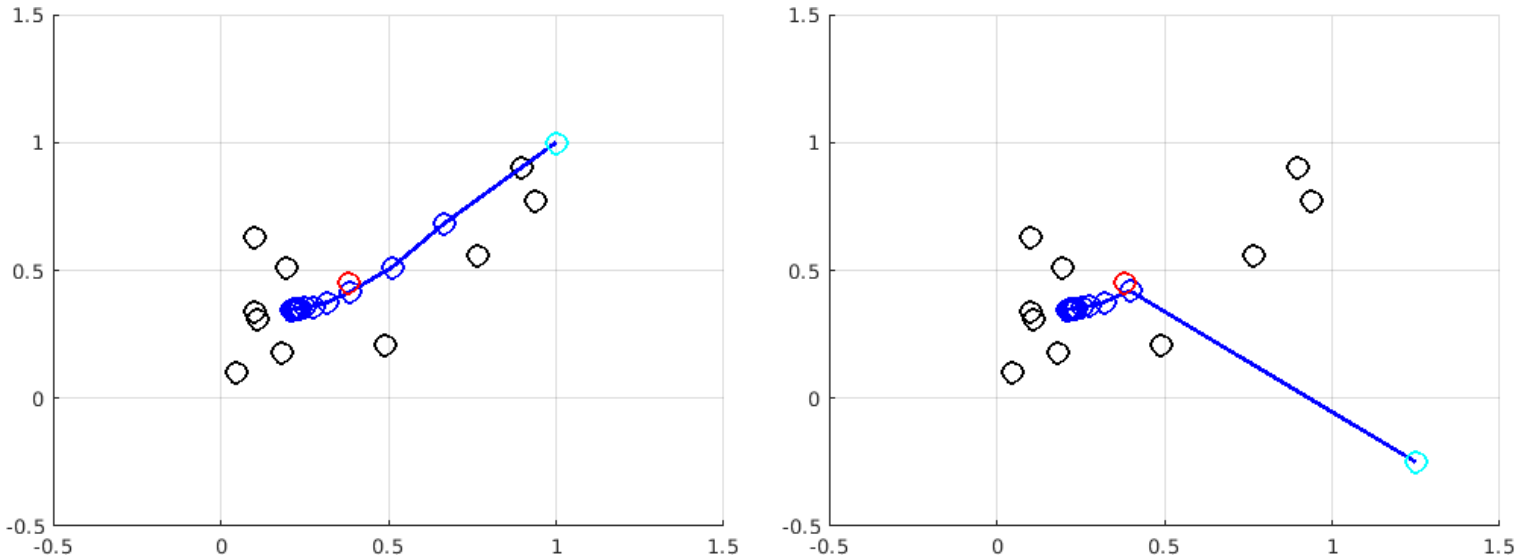


Figure 25: Iterates of the Kuhn/Kuenne algorithm (blue) from starting point $x_0 = (1, 1)^T$ (left) and $x_0 = (1.25, -0.25)^T$ (right) using uniformly dist. weights and $\epsilon = 10^{-5}$. Center of mass is in red.

Remarks about geometric median

- More robust to outliers than center of mass (breakdown point=0.5).
- Invariant to affine Euclidean transformations (translation, rotation, scaling).
- Multiple extensions (L^p norms, multiple locations $(x_i)_{i=1}^m$, negative weights, etc.).

Example 2: square root of a number $a \in \mathbb{R}_{\geq 0}$

Finding the square root of a is equivalent to finding the root of

$$f(x) = (x^2 - a) \in \mathbb{R}.$$

Using Newton's method, one simply ends up with

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)} = x_k - \left(\frac{x_k^2 - a}{2x_k} \right) = \frac{1}{2} \left(x_k + \frac{a}{x_k} \right).$$

k	x_k	$f(x_k)$
0	10	-512
1	35.6	655.36
2	26.395505617978	84.722716828708
3	24.790635492455	2.575608119769
4	24.738688294075	0.002698511403
5	24.738633753767	0.000000003019

Table 1: Example of sequence returned by Newton's method with initial guess $x_0 = 10$. Please note how the number of true digits (underlined) in x_k doubles at each iteration.

Example 3: smallest enclosing circle

- For positive integers $d > 0$ and $n > 0$, let us consider a set $\{y_i\}_{i=1}^n \in (\mathbb{R}^d)^n$. Finding the smallest enclosing circle (x^*, r^*) amounts to solve

$$\begin{cases} x^* & = \min_x \max_{i=1}^n \|x - y_i\|_2, \\ r^* & = \max_{i=1}^n \|x^* - y_i\|_2. \end{cases}$$

- Problem: L^∞ is non-differentiable $\Rightarrow$ subgradient algorithm.

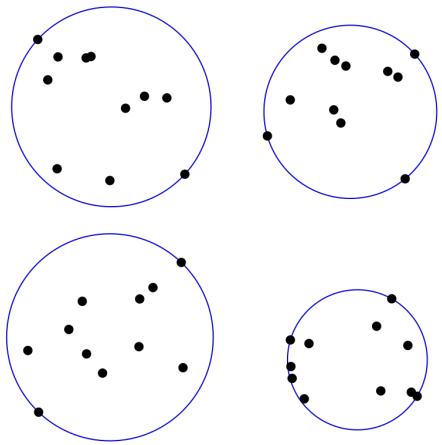


Figure 26: Examples of smallest enclosing circles.

Pros

- Nonlinear E can be minimized.
- Existence of efficient minimization algorithms.
- Absence of metrication artifacts.
- Dependency on the initial guess.
- Highly parallelizable.

Cons

- Necessity of a stopping criterion.
- Derivatives need to be calculated (and eventually modified).

Optimization algorithms for computer vision

The following algorithms were compared on typical vision tasks in [SZS+08]:

- Iterated Conditional Modes (ICM).
- Belief Propagation (BP).
- Graph Cuts (GC).
- Tree Reweighted Message Passing (TRWS), etc.

$\rightarrow$ Graph cuts remain very competitive w.r.t other algorithms.

Notations about graphs

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E}, c)$ be a capacitated and directed graph (without self-loops) where

- $\mathcal{V} = \mathcal{P} \cup \{s, t\}$ is the set of nodes with $\mathcal{P} \subset \mathbb{Z}^d$ $(d > 0)$.
- $\mathcal{E} = (\mathcal{N} \cup \mathcal{E}_t) \subset (\mathcal{V} \times \mathcal{V})$ is the set of edges with

$$\left\{ \begin{array}{lcl} \mathcal{N} & \subset & (\mathcal{P} \times \mathcal{P}) \quad \text{(depend on the choice of the neighborhood)} \\ \mathcal{E}_t & = & (\{s\} \times \mathcal{P}) \cup (\mathcal{P} \times \{t\}) \end{array} \right. ,$$

- $c : (\mathcal{V} \times \mathcal{V}) \to \mathbb{R}^+$ are edge capacities.

Assumptions (without loss of generality)

- For any edge $(p, q) \in \mathcal{E}$, $c(p, q) > 0$.
- For any edge $(p, q) \notin \mathcal{E}$, $c(p, q) = 0$.
- For any node $p \in \mathcal{P}$, $c(s, p) > 0 \Rightarrow c(p, t) = 0$.

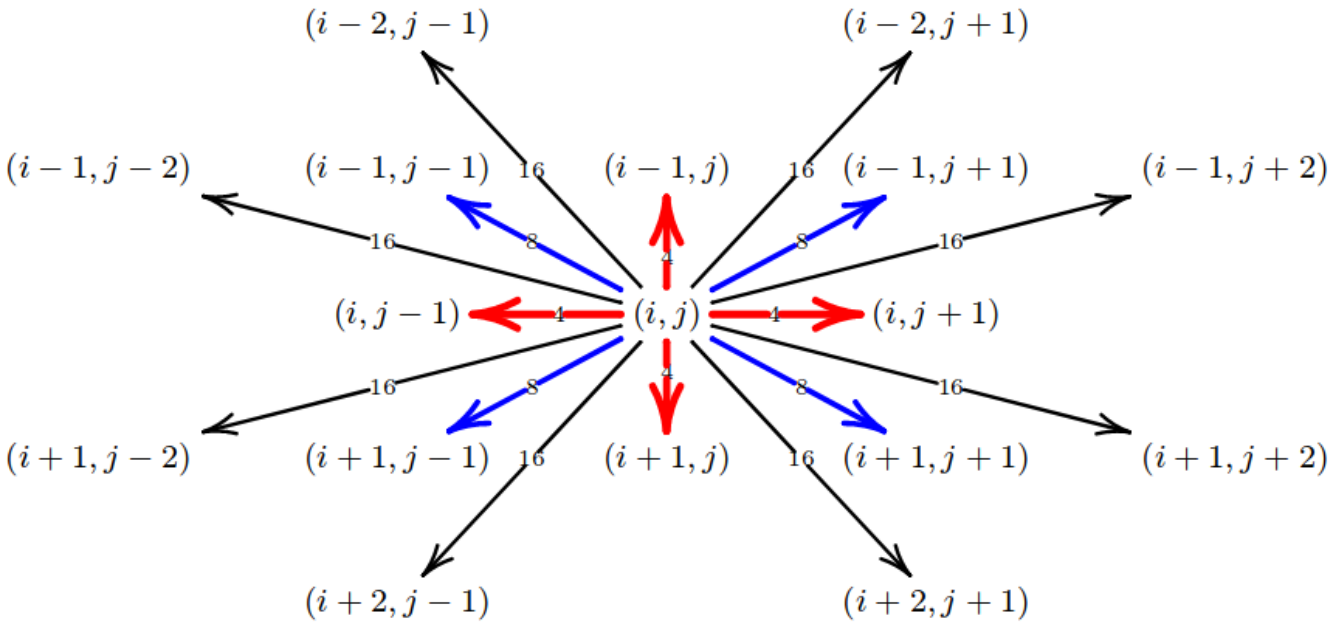


Figure 27: Typical neighborhoods involving 4, 8 and 16 nodes.

s-t cut

An s-t cut $\mathcal{C} = (\mathcal{S}, \mathcal{T})$ is a partition of $\mathcal{V}$ ($\mathcal{V} = \mathcal{S} \cup \mathcal{T}$ and $\mathcal{S} \cap \mathcal{T} = \emptyset$) s.t. $s \in \mathcal{S}$, $t \in \mathcal{T}$.

s-t cut capacity

The capacity (or weight) of an s-t cut $\mathcal{C}$ in a graph $\mathcal{G}$ is defined by

$$|\mathcal{C}|_{\mathcal{G}} = \sum_{(p,q) \in (\mathcal{S} \times \mathcal{T})} c(p,q) = \sum_{(p,q) \in \mathcal{C}} c(p,q).$$

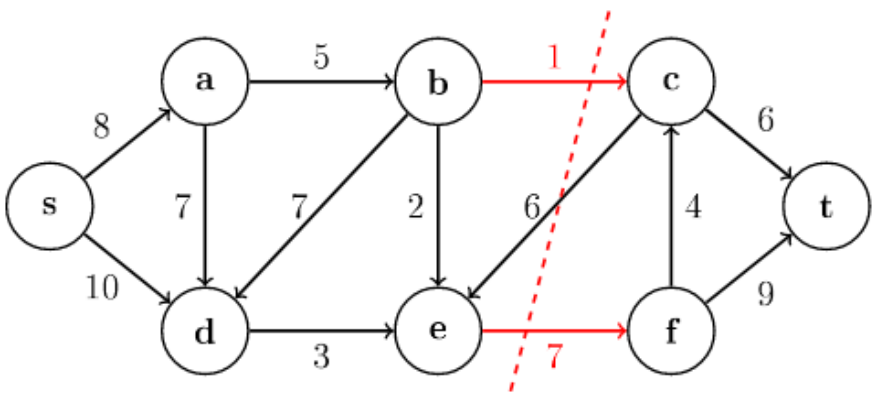


Figure 28: An s-t cut (red dashed line) whose value is 8. Reverse edges are not taken into account.

Minimum s-t cut

A minimum s-t cut $\mathcal{C}^*$ of a graph $\mathcal{G}$ is an s-t cut of minimum capacity among the set of possible s-t cuts (whose cardinality is bounded by $2^{\sharp\mathcal{P}}$).

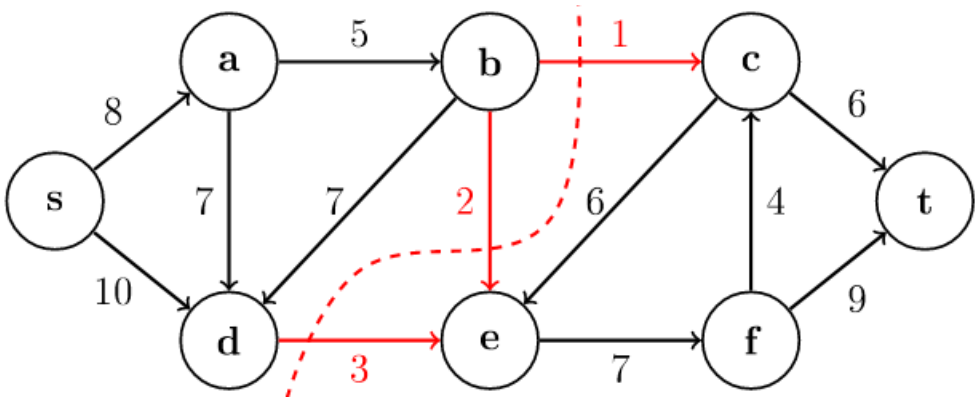


Figure 29: Minimum s-t cut $\mathcal{C}^*$ of the graph $\mathcal{G}$ in Fig. 28.

Valid flow

A flow $f : (\mathcal{V} \times \mathcal{V}) \to \mathbb{R}^+$ in a graph $\mathcal{G}$ is said to be valid if

- $0 \leq f(p,q) \leq c(p,q), \quad \forall (p,q) \in (\mathcal{V} \times \mathcal{V})$ (flow capacity).
- $\sum_{(p,q)\in\mathcal{E}} f(p,q) = \sum_{(q,p)\in\mathcal{E}} f(q,p), \quad \forall p \in \mathcal{P}$ (flow conservation).

Let us remark it is guaranteed that $f(p,q) = 0$, for any edge $(p,q) \notin \mathcal{E}$.

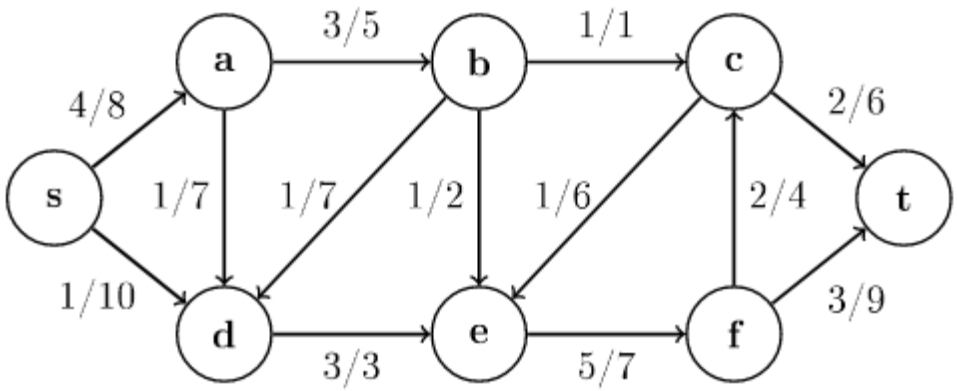


Figure 30: A valid flow in the graph $\mathcal{G}$ in Fig. 28.

Flow value

The value of a flow f in a graph $\mathcal{G}$ is given by $\sum_{(s,p)\in\mathcal{E}} f(s,p)$.

Maximum flow problem

A maximum flow f^* in a graph $\mathcal{G}$ is a solution to the linear program

$$\left\{\begin{array}{ll} \max_{f\in\mathbb{R}^{\mathcal{E}}} & \sum_{(s,p)\in\mathcal{E}} f(s,p), \\ \forall (p,q)\in\mathcal{E} & 0\leq f(p,q)\leq c(p,q), \\ \forall p\in\mathcal{P} & \sum_{(p,q)\in\mathcal{E}} f(p,q) = \sum_{(q,p)\in\mathcal{E}} f(q,p). \end{array}\right. .$$

Theorem [DF56, ECS56]

The max-flow of a graph $\mathcal{G}$ is equal to the smallest capacity of any s-t cut. We therefore have $|f^*| = |\mathcal{C}^*|$ and $\mathcal{C}^*$ can be deduced from f^*.

Remarks

- All edges in $\mathcal{C}^*$ are saturated but there can exist saturated edges not in $\mathcal{C}^*$.
- The min s-t cut can be seen as the bottleneck in a network rooting water.
- Since the min s-t cut $\mathcal{C}^*$ can be deduced from the max-flow f^*, algorithms computing max-flows are also able to compute min s-t cuts.

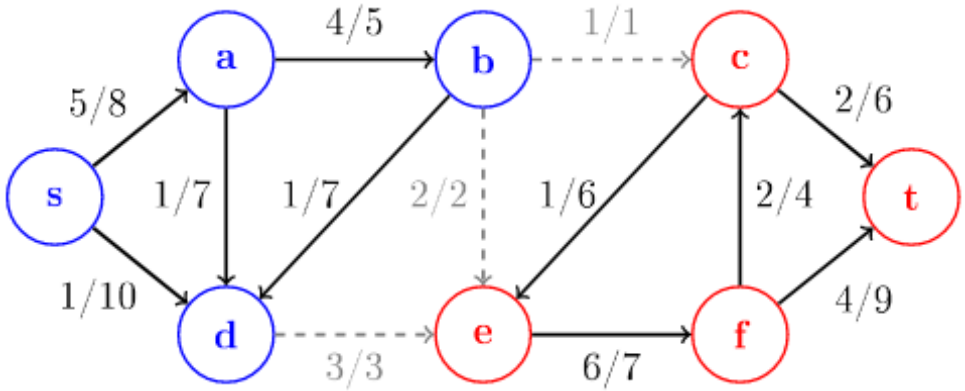


Figure 31: Max-flow / min s-t cut of the graph $\mathcal{G}$ in Fig. 28.

Families of max-flow / min s-t cut algorithms

- **Augmenting paths:** Iteratively pushes flow along augmenting paths in residual graphs (flow conservation hold).
- **Preflow-push:** Iteratively pushes flow excesses towards nodes near t (flow conservation is semi-relaxed).
- **Pseudoflow:** Seeks a partition of deficit/excesses nodes (flow conservation is fully relaxed).
- **Probabilistic:** Iteratively perform contraction of nodes.

Remarks

- Max-flow / min s-t cut algorithms empirically perform linearly with $\sharp\mathcal{P}$ [BK04].
- Unlike preflow/pseudoflow algorithms, augmenting path-based ones are global.

Residual graph

Let f be a valid flow in a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, c)$. We define the edge capacities of the graph $\mathcal{G}_f = (\mathcal{V}, \mathcal{E}_f, c_f)$ associated to the flow f in $\mathcal{G}$ for any $(p,q) \in \mathcal{E}$ by

$$\begin{cases} c_f(p,q) = c(p,q) - f(p,q) & \text{if } f(p,q) < c(p,q) \\ c_f(q,p) = f(p,q) & \text{if } f(p,q) > 0 \end{cases}.$$

Augmenting path

An augmenting path is a path $(u_1, \ldots, u_k)$ in the residual graph $\mathcal{G}_f$ where $u_1 = s$, $u_k = t$ and $c_f(u_i, u_{i+1}) > 0$, $\forall i \in \{1, \ldots, k-1\}$.

Theorem

A valid flow f in $\mathcal{G}$ has a max-flow value $|f^*|$ iff no more augmenting path exist in $\mathcal{G}_f$.

Lemma

Let $\mathcal{P}' = \{p \in \mathcal{P} \mid \exists \text{ an augmenting path } (s, \ldots, p) \text{ in } \mathcal{G}_{f^*}\}$ be the set of nodes that can be reached from s in $\mathcal{G}_{f^*}$. Then, the min s-t cut is $\mathcal{C}^* = (\mathcal{P}' \cup \{s\}, \mathcal{V} \setminus (\mathcal{P}' \cup \{s\}))$.

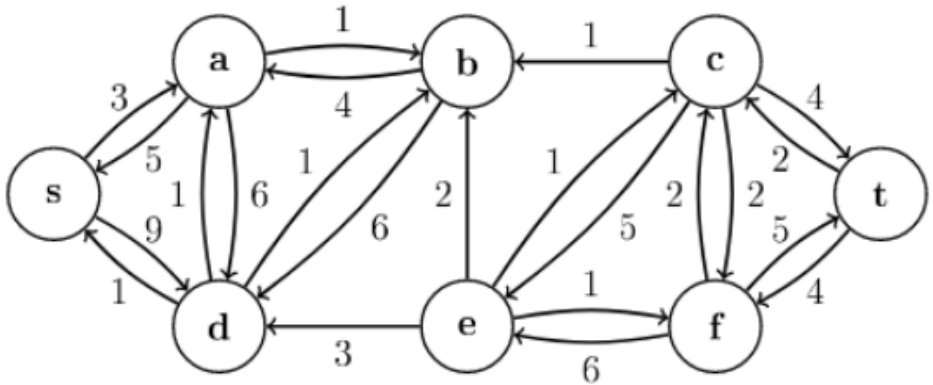


Figure 32: Residual graph $\mathcal{G}_{f*}$ of Fig. 31. The set of nodes reachable from s is $\{s, a, b, d, e\}$.

Algorithm 3: Ford Fulkerson (FF) algorithm

INPUTS: A graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, c)$
OUTPUTS: Residual graph $\mathcal{G}_f = (\mathcal{V}, \mathcal{E}_f, c_f)$

$\mathcal{G}_f \leftarrow \mathcal{G}$
while $\exists$ an augmenting path $(u_1, \ldots, u_k)$ in $\mathcal{G}_f$ **do**
    % Compute amount of flow to push along augmenting path
    $\Delta f \leftarrow \min \{c_f(u_i, u_{i+1}) \mid \forall i \in \{1, \ldots, k-1\}\}$
    % Update residual capacities
    $c_f(u_i, u_{i+1}) \leftarrow c_f(u_i, u_{i+1}) - \Delta f, \quad \forall i \in \{1, \ldots, k-1\}$
    $c_f(u_{i+1}, u_i) \leftarrow c_f(u_{i+1}, u_i) + \Delta f, \quad \forall i \in \{1, \ldots, k-1\}$
end-while
return $\mathcal{G}_f$

Remarks

- Let $\mathcal{S}$ be the set of nodes that can be reached from s in $\mathcal{G}_f$ at the end of the Ford Fulkerson (FF) algorithm. Then, the minimum s-t cut is given by $\mathcal{C}^* = (\mathcal{S}, \mathcal{V} \setminus \mathcal{S})$.
- FF algorithm do not specify how to select augmenting paths.
- Worst-case complexity of FF algorithm is pseudo-polynomial: $O(\sharp\mathcal{E}|f^*|)$.

Selection of augmenting paths

- **Shortest pipes:** [Din70] focuses on shortest paths only using BFS and blocking flows to explore nodes in $O(\sharp\mathcal{V}^2\sharp\mathcal{E})$; [BK04] is growing two non-overlapping trees from s and t in pseudo-polynomial complexity $O(\sharp\mathcal{V}^2\sharp\mathcal{E}^2|f^*|)$.
- **Fattest pipes:** [EK72] focuses on largest capacity paths only using BFS to explore nodes in $O(\sharp\mathcal{V}\sharp\mathcal{E}^2)$.
- **Fattest/shortest pipes:** focus on shortest paths with larger capacity first [JB07].

Minimum cost-flow problem

- This problem aims at finding the cheapest possible way of sending flow in a graph.
- Application: finding the best delivery route from a factory to a warehouse where the road network has some capacity and cost associated.
- If all costs vanish, this reduces to the standard max-flox min s-t cut problem.

Multi-source multi-sink max-flow problem

- This problem aims at finding a max-flow in a graph $\mathcal{G}$ with a set of sources $\{s_1, \ldots, s_n\}$ and a set of sinks $\{t_1, \ldots, t_m\}$.
- By connecting a "super source" and a "super sink" with their respective sets using infinite edge capacities, this reduces to the standard max-flox min s-t cut problem.

One-to-one correspondence

For any s-t cut $\mathcal{C} = (\mathcal{S}, \mathcal{T})$ in a graph $\mathcal{G}$, we define for any node $p \in \mathcal{P}$

$$u_p^{\mathcal{C}} = \begin{cases} 0 & \text{if} \quad p \in \mathcal{S} \\ 1 & \text{if} \quad p \in \mathcal{T} \end{cases} .$$

$\rightarrow$ One-to-one correspondence between s-t cuts $\mathcal{C}$ and elements of $\{0,1\}^{\mathcal{P}}$.

Idea

- Set edge capacities in a graph $\mathcal{G}$ s.t. $|\mathcal{C}| = E(u^{\mathcal{C}}) + K$ (K independent of u).
- If $\mathcal{C}^*$ is a min s-t cut in $\mathcal{G}$ then $u^{\mathcal{C}^*}$ is a minimizer of E.
- Hence, $u^{\mathcal{C}^*}$ can be (efficiently) minimized using a max-flow min s-t cut algorithm.

Questions

- What energies can be minimized using graph cuts and how?
- How to represent such energies with graphs?

Class of functions $\mathcal{F}^k$

Given $k \in \mathbb{N}^*$ and binary variables $(x_i)_{i=1}^n \in \{0,1\}^n$, the class $\mathcal{F}^k$ are $E : \{0,1\}^n \to \mathbb{R}$ that write as a sum of terms of up to k variables at a time.

When $k = 2$, E belonging to $\mathcal{F}^2$ write

$$E(x_1, \ldots, x_n) = \sum_{i=1}^{n} E_p(u_p) + \sum_{i \leq i,j \leq n} E_{p,q}(u_p, u_q).$$

Graph-representability (definition)

E is said to be graph-representable if there exists a graph $\mathcal{G}$ s.t. for any s-t cut $\mathcal{C}$, $|\mathcal{C}| = E(u^{\mathcal{C}}) + K$ (K is a constant independent of u).

$\to$ E is said to be exactly represented by $\mathcal{G}$ if K is null.

Graph-representability (theorem [KZ04])

Let E be a function of class $\mathcal{F}^2$. Then, E is graph-representable iff for any $(p,q) \in \mathcal{N}$

$$E_{p,q}(0,0) + E_{p,q}(1,1) \leq E_{p,q}(1,0) + E_{p,q}(0,1). \tag{3}$$

- A function E satisfying (3) is said to be regular or submodular.
- Constructive proof is given for sufficient condition (using additivity theorem).
- Necessary condition is proven in a more general setting (regular functions).

Remarks

- Has been extended to E of class $\mathcal{F}^3$ [KZ04] and beyond [FD05].
- [KZ04] explains how to represent submodular E (representation is not unique).

Graph-representability of terms E_p

- Any term E_p is graph-representable,
- Subtracting $\min\{E_p(0), E_p(1)\}$ to edge capacities leads to less edges.

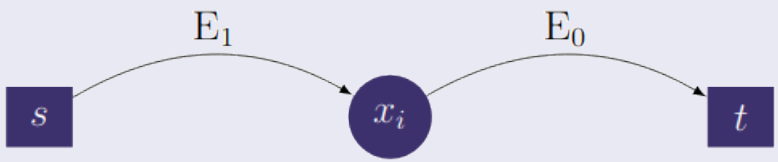


Figure 33: Graph construction of terms E_p.

Graph-representability of terms $E_{p,q}$

Any term $E_{p,q}$ is graph-representable if it is submodular. $E_{p,q}$ can be decomposed as follows (the decomposition is not unique)

$$E_{p,q}(u_p,u_q) = \begin{cases} E_{01} & \text{if} \quad u_p = 0 \\ E_{11}, & \text{if} \quad u_p = 1 \end{cases} + \begin{cases} E_{00} - E_{01} & \text{if} \quad u_q = 0, \\ 0 & \text{if} \quad u_q = 1 \end{cases} + \begin{cases} E_{01} + E_{10} - E_{00} - E_{11} & \text{if } u_p = 0, u_q = 1, \\ 0 & \text{otherwise} \end{cases} - E_{10}. \tag{4}$$

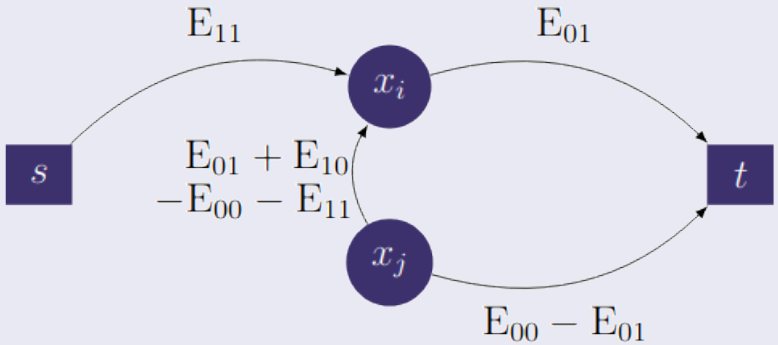


Figure 34: Graph construction of terms $E_{p,q}$.

Problem

- Is it possible to construct a graph so that the cost of a cut is arbitrarily close to the Euclidean metric of the corresponding contours?
- How the topology of the graph impacts the approximation of the contours length?
- Surprisingly, this problem was still open until a few years ago.

Consequences

- If so, graph cuts could be used to find globally minimum contours/surfaces.
- Metrication artifacts could be reduced in existing graph cuts-based methods.

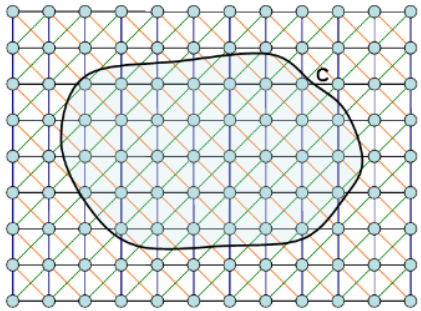


Figure 35: A curve C and a set of intersecting lines.

Answered by [BK03] (continuous case)

Let $L(\rho, \theta)$ be a straight line in $\mathbb{R}^2$ in polar coordinates. Assuming that graph topology is fixed, the Cauchy-Crofton formula establishes a link between the Euclidean length of a curve $\mathcal{C}$ and a finite set of intersecting lines:

$$|\mathcal{C}|_{\mathcal{E}} = \frac{1}{2} \int_0^{\pi} \int_{-\infty}^{+\infty} N(\rho, \theta) d\rho d\theta,$$

where $N(\rho, \theta)$ denotes the number of intersections of $L(\rho, \theta)$ with $\mathcal{C}$.

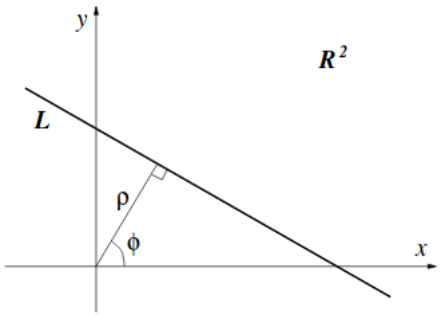


Figure 36: A straight line in the plane $\mathbb{R}^2$ in polar coordinates.

Answered by [BK03] (discrete case)

Let us assume that $\mathcal{G}$ is a δ-regular grid with neighborhood systems described by a set of vectors $(e_k)_{k=1}^n$ enumerated in increasing order of their angular orientation θ_k. Any e_k is assumed to be of the shortest length and generates a family of edge-lines.

By appropriately sampling $[0, \pi] \times \mathbb{R}$, one ends up with

$$|\mathcal{C}|_{\mathcal{E}} \simeq \sum_{k=1}^{n} n_c(k) \underbrace{\left(\frac{\delta^2 \Delta\theta_k}{2|e_k|} \right)}_{w_k} \simeq |\mathcal{C}|_{\mathcal{G}}, \quad \text{where}$$

- $n_c(k)$ is the number of intersections of $\mathcal{C}$ with the kth family of lines.
- w_k is the edge weight of the kth family of lines.

Theorem ([BK03])

If C is a continuously differentiable regular curve in $\mathbb{R}^2$ intersecting each straight line a finite number of times then

$$|\mathcal{C}|_{\mathcal{G}} \to |\mathcal{C}|_{\mathcal{E}},$$

as δ, $\sup_k |\Delta\theta_k|$ and $\sup_k |e_k|$ get to zero.

Remark

The results of [BK03] also hold for any Riemannian metric and for any dimension.

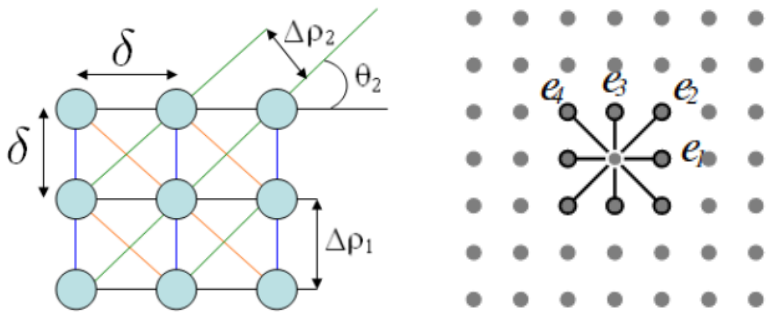


Figure 37: (a) A regular graph $\mathcal{G}$ and (b) its neighborhood system.

Example

Let us consider a graph $\mathcal{G}$ as a 1-regular grid with neighborhood systems defined by

$$\{(1,0),(1,1),(0,1),(-1,-1)\}.$$

The edge weights of the graph $\mathcal{G}$ are then given by

$$\begin{cases} w_1 = w_3 = \frac{1\cdot\pi/4}{2\cdot 1} = \frac{\pi}{4} \\ w_2 = w_4 = \frac{1\cdot\pi/4}{2\cdot\sqrt{2}} = \frac{\pi\sqrt{2}}{16} \end{cases}.$$

Remark

- The memory consumption for storing graphs can be prohibitive.
- The needed amount of memory is proportional to $\sharp\mathcal{P}$ and $\sharp\mathcal{E}$.
- Totally intractable when $\sharp\mathcal{P}$ or $\sharp\mathcal{E}$ are large.

	Connectivity 0	Connectivity 1
2D	6138	4912
3D	308	209
4D	70	42

Table 2: Maximum size of a square image for which the graph fits in 2GB of RAM using [BK04].

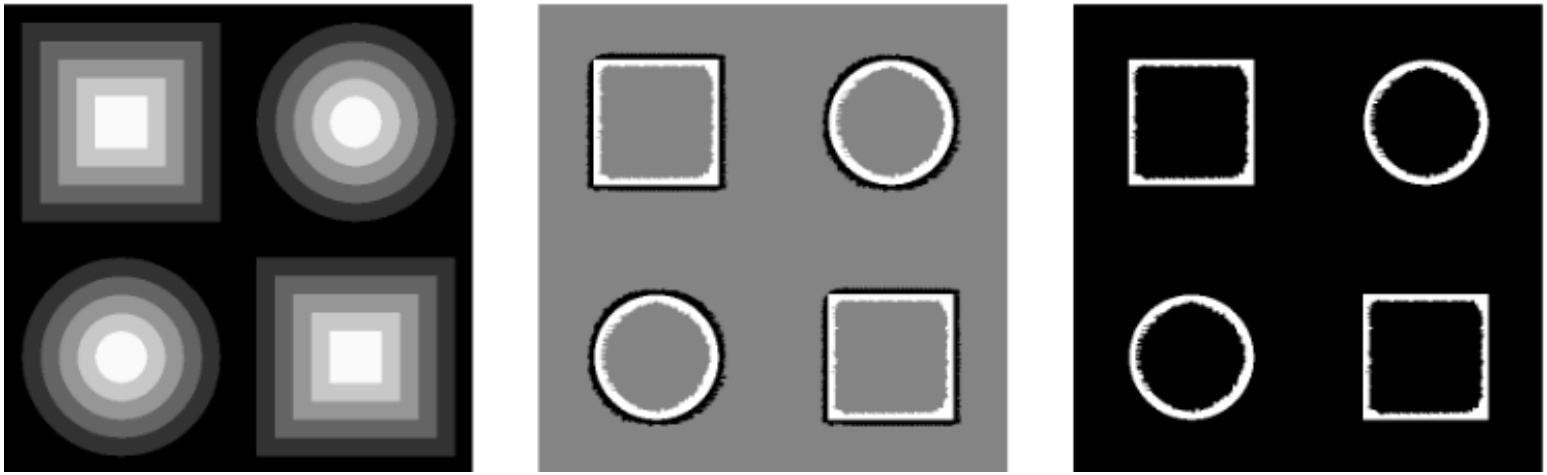


Figure 38: Flow passing through t-links (middle) and n-links (right) for segmenting an image.

Remarks

- Only a few nodes participate to the max-flow computation.
- Thus, one would like to minimize $\sharp\mathcal{V}'/\sharp\mathcal{V}$ s.t. $|f'^*| \simeq |f^*|$ [LML10].

Priors vs minimization

- **Convex prior [RC98, Ish03]:**
  - Layered graph construction (number of layers=$\sharp\mathcal{L}$).
  - Global minimization.
  - Prohibitive memory usage.
- **TV prior [Cha05, Dar05, CD09, LMHT12, RLLHM18]:**
  - Decomposition into nested level-sets,.
  - Global minimization.
  - Reasonable memory usage.
- **Robust prior [BVZ01]:**
  - NP-hard, many local minima.
  - Approximated minimization (within a known factor from the global minima).
  - Fast and memory efficient iterative minimization procedure.

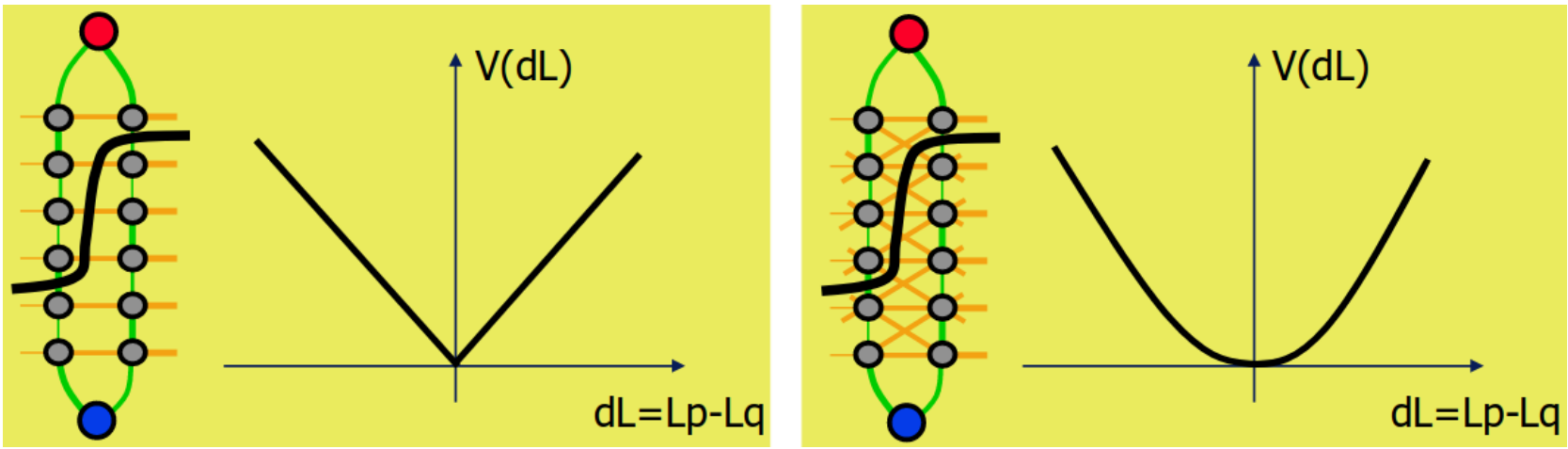


Figure 39: Layered graph construction for linear (left) and convex (right) pairwise terms [Ish03].

Principle of α-expansion algorithm [BVZ01]

- Iterative algorithm minimizing a sequence of submodular E' over $\mathcal{L}$.
- Any E' only involves binary variables and is solved with max-flow/min-cut.
- For any $\alpha \in \mathcal{L}$, u_p is either unchanged or assigned to α.
- E is guaranteed to decrease at each iteration.

Principle of α-β swap algorithm [BVZ01]

- Iterative algorithm minimizing a sequence of submodular E' over $(\mathcal{L} \times \mathcal{L})$.
- Any E' only involves binary variables and is solved with max-flow/min-cut.
- For any $(\alpha, \beta) \in \mathcal{L}^2$, u_p is either unchanged or assigned to α or β.
- E is guaranteed to decrease at each iteration.

Energetic formulation of α-expansion

Let us define $v \in \{0,1\}^{\mathcal{P}}$ as follows:

$$u_p^v = \begin{cases} \alpha & \text{if} \quad v_p = 0 \\ u_p & \text{if} \quad v_p = 1 \end{cases}, \quad \forall p \in \mathcal{P}.$$

For any $v \in \{0,1\}^{\mathcal{P}}$, we also define

$$E(u^v) = E'(v) = \beta \sum_{p \in \mathcal{P}} E'_p(v_p) + \sum_{(p,q) \in \mathcal{N}} E'_{p,q}(v_p, v_q), \qquad \text{where,}$$

$$E'_p(0) = E_p(u_p), \qquad E'_p(1) = E_p(\alpha), \qquad \text{and}$$

$$\begin{pmatrix} E'_{p,q}(0,0) & E'_{p,q}(0,1) \\ E'_{p,q}(1,0) & E'_{p,q}(1,1) \end{pmatrix} = \begin{pmatrix} E_{p,q}(u_p, u_q) & E_{p,q}(u_p, \alpha) \\ E_{p,q}(\alpha, u_q) & E_{p,q}(\alpha, \alpha) \end{pmatrix}.$$

Algorithm 4: α-expansion algorithm

INPUTS: Elements permitting the computation of E
OUTPUTS: An approximate minimizer of E

Initialize $u = (u_p)_{p \in \mathcal{P}}$
do
  $u' \leftarrow u$
  for $\alpha \in \mathcal{L}$ **do**
    $v \in \operatorname{argmin}_{v \in \{0,1\}^{\mathcal{P}}} E'(v)$
    $\forall p \in \mathcal{P}$, if $v_p = 1$ then $u_p \leftarrow \alpha$
  end-for
until $E(u') \leq E(u)$

Applicability of α-expansion

It can only be employed if, for any $u_p, u_q, \alpha \in \mathcal{L}$, we have

$$E_{p,q}(u_p, u_q) + E_{p,q}(\alpha, \alpha) \leq E_{p,q}(u_p, \alpha) + E_{p,q}(\alpha, u_q).$$

If $E_{p,q}(\alpha, \alpha) = 0$ (which is mostly the case), a triangular inequality appears.

Energetic formulation of α-β swap

Let us define $\mathcal{P}_{\alpha,\beta} = \{p \in \mathcal{P} \mid u_p = \alpha \text{ or } u_p = \beta\}$ and $v \in \{0,1\}^{\mathcal{P}}$ as follows

$$u_p^v = \begin{cases} \alpha & \text{if} \quad v_p = 0 \\ \beta & \text{if} \quad v_p = 1 \end{cases}, \quad \forall p \in \mathcal{P}_{\alpha,\beta}.$$

For any $v \in \{0,1\}^{\mathcal{P}_{\alpha,\beta}}$, we also define

$$E(u^v) = E'(v) = \beta \sum_{p \in \mathcal{P}_{\alpha,\beta}} E'_p(v_p) + \sum_{(p,q) \in (\mathcal{N} \cap \mathcal{P}^2_{\alpha,\beta})} E'_{p,q}(v_p, v_q), \qquad \text{where,}$$

$$E'_p(0) = E_p(\alpha), \qquad E'_p(1) = E_p(\beta), \qquad \text{and}$$

$$\begin{pmatrix} E'_{p,q}(0,0) & E'_{p,q}(0,1) \\ E'_{p,q}(1,0) & E'_{p,q}(1,1) \end{pmatrix} = \begin{pmatrix} E_{p,q}(\alpha,\alpha) & E_{p,q}(\alpha,\beta) \\ E_{p,q}(\beta,\alpha) & E_{p,q}(\beta,\beta) \end{pmatrix}.$$

Algorithm 5: α-β swap algorithm

INPUTS: Elements permitting the computation of E
OUTPUTS: An approximate minimizer of E

Initialize $u = (u_p)_{p \in \mathcal{P}}$
do
  $u \leftarrow u'$
  for $(\alpha, \beta) \in \mathcal{L}^2$ **do**
    $v \in \operatorname{argmin}_{v \in \{0,1\}^{\mathcal{P}_{\alpha,\beta}}} E'(v)$
    $\forall p \in \mathcal{P}$, if $v_p = 0$ then $u_p \leftarrow \alpha$, otherwise $u_p \leftarrow \beta$
  end-for
until $E(u') \leq E(u)$

Applicability of α-β swap

It can only be employed if, for any $(\alpha, \beta) \in \mathcal{L}^2$, we have

$$E_{p,q}(\alpha, \alpha) + E_{p,q}(\beta, \beta) \leq E_{p,q}(\alpha, \beta) + E_{p,q}(\beta, \alpha).$$

Remark: weaker condition of applicability than for α-expansion.

Remarks

- u can be initialized in different manners:
  - Winner-take-all (without regards to pairwise terms).
  - Random (e.g. uniformly chosen in $\mathcal{L}$).
  - Constant label (same label α for all pixels).
- Labels can be scanned in several ways:
  - Forward or backward.
  - Randomly.
- α-expansion generally converges in only a few iterations on typical vision tasks (but theoretical convergence speed is unknown).
- When the terms $(E_{p,q})_{(p,q)\in\mathcal{N}}$ satisfy the axioms of a metric on label set $\mathcal{L}$, the solution is guaranteed to be within a constant factor from the global minimizer.



Figure 40: Example of run of the α-expansion algorithm in stereovision.

Pros about graph cuts

- Mature approach applied to a wide range of problems.
- Existence of fast max-flow / min s-t cuts algorithms.
- Characterization of E that can be exactly minimized.
- Good heuristics in the multi-labels case ($\sharp\mathcal{L} > 2$).
- Derivative-free approach.
- No parameters about differentiability or convergence to tune.

Cons about graph cuts

- Prohibitive memory usage (especially if $\mathcal{V}$ and/or $\mathcal{E}$ are large).
- Presence of metrication artifacts (due to grid discretization).
- Unappropriate for recovering thin structures (shrinking bias).

Outline

1 Motivations

2 Bayes justification

3 Elements of optimization

4 Applications

5 Conclusion

Mathematical modeling

Given $d, d' \in \mathbb{N}_{>0}$, image restoration aims at recovering an approximation image $u^* \in (\mathbb{R}^{d'})^{N'^2}$ from observations $v \in (\mathbb{R}^d)^{N^2}$ and can be modeled as

$$v = Hu + b, \quad \text{where,}$$

- Acquisition device is modeled by the operator $H : (\mathbb{R}^{d'})^{N'^2} \to (\mathbb{R}^d)^{N^2}$.
- Noise is modeled by $b \in (\mathbb{R}^d)^{N^2}$ (e.g. white Gaussian noise).

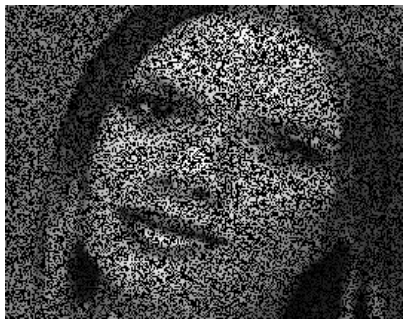
Common choices about H, N and N'

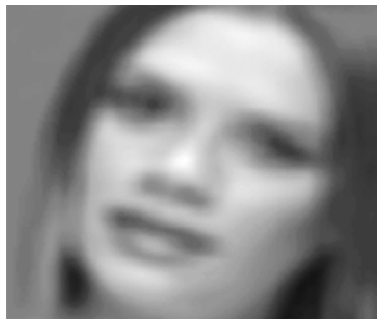
- $H = I$ ($d = d'$, $N = N'$): basic image denoising.
- H is a blurring operator ($d = d'$, $N = N'$): deconvolution.
- H is a "masking" operator ($d = d'$, $N = N'$): inpainting.
- H is a linear operator ($d = 1$, $d' = 3$, $N = N'$): demosaicing.
- H combines blurring/subsampling operators ($d = d'$, $N < N'$): super-resolution.


Original image


Noise


Spatial quantization


Partial image loss


Optical blur


Motion blur

Figure 41: Different types of degradations.

General energetic formulation

Given data $v \in (\mathbb{R}^d)^{N^2}$, operator $H : (\mathbb{R}^{d'})^{N'^2} \to (\mathbb{R}^d)^{N^2}$ as well as parameters $\beta \in \mathbb{R}_{\geq 0}$ and $p \in \{1, 2\}$, u^* is taken as the minimizer of

$$\min_{u \in (\mathbb{R}^{d'})^{N'^2}} E(u) = \|Hu - v\|_p^p + \beta E_s(u). \tag{5}$$

A widespread approach is to use Total Variation (TV) for the regularization term E_s [a]:

$$E_s(u) = TV(u) = \int_{[0,N'^2]} \|\nabla u\|_2 dxdy \simeq \sum_{1 \leq i,j \leq N'} \sqrt{(\partial_x u_{i,j})^2 + (\partial_y u_{i,j})^2},$$

where ∂_x and ∂_y are partial derivative operators.

[a] We assume that u is periodic.

Partial derivative operators ($d' = 1$)

The operators ∂_x and ∂_y are approximated by finite differences and defined by

$$\begin{array}{rcl} \partial_x : \mathbb{R}^2 & \to & \mathbb{R}^2 \\ (u_{i,j})_{1 \leq i,j \leq N} & \mapsto & (u_{i+1,j} - u_{i,j})_{1 \leq i,j \leq N} \end{array},$$

and

$$\begin{array}{rcl} \partial_y : \mathbb{R}^2 & \to & \mathbb{R}^2 \\ (u_{i,j})_{1 \leq i,j \leq N} & \mapsto & (u_{i,j+1} - u_{i,j})_{1 \leq i,j \leq N} \end{array}.$$

Other finite differences schemes can be used as well.

Properties of $\|.\|_p^p$

- $\|.\|_p^p$ is coercive and non-negative; $\|.\|_2^2$ is strictly convex while $\|.\|_1^1$ is not.
- $\|.\|_2^2$ is continuously differentiable everywhere while $\|.\|_1^1$ is not.

Properties of TV

- $TV(u)$ is continuous, non-negative, convex, not coercive [a] and not differentiable [b].
- $TV(u)$ is the sum of perimeters of the level sets of u (coarea formula).
- Some algorithms its anisotropic version: $TV(u) = \sum_{0 \leq i,j \leq N'} \left(|\nabla_x u_{i,j}| + |\nabla_y u_{i,j}| \right)$.

[a] For instance, we have $TV(u + c) = TV(u)$.
[b] In particular, at $\|\nabla u\|_2 = 0$.

Remarks about (5)

- First term of (5) encourages Hu to be close to the observations v.
- $TV(u)$ discourages u from having oscillations but allows it to have discontinuities.
- $p = 2$ deals with Gaussian noise [ROF92]; $p = 1$ deals with Laplace noise [All97].

Continuous minimization of (5) (1/5)

Practically, one instead considers ϵ-regularized versions for $\|.\|_1^1$ and $TV(u)$:

$$TV_\epsilon(u) = \sum_{1 \leq i,j \leq N'} \phi_\epsilon(\|\nabla u_{i,j}\|_2^2),$$

where

$$\phi_\epsilon(x) = \sqrt{x+\epsilon}, \quad \text{and} \quad \phi'(x) = \frac{1}{2\sqrt{(x+\epsilon)}}, \quad \forall x \in \mathbb{R}_{\geq 0}.$$

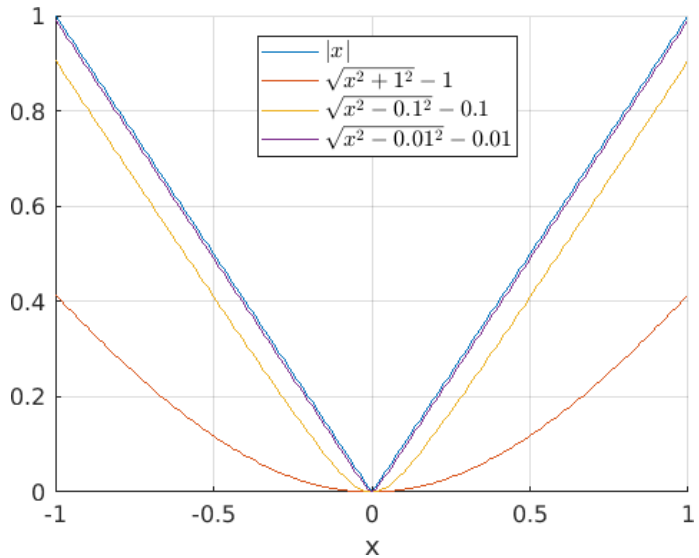


Figure 42: ϵ-regularized versions of $x \mapsto |x|$.

Continuous minimization of (5) (2/5)

To obtain an approximate minimizer of (5), we need to calculate $\nabla E(u)$.

For doing so, we need to define the notion of adjoint operators. Let us consider a linear operator $A : H_1 \to H_2$. The adjoint of A is a linear operator that fulfill

$$\langle Ah_1, h_2 \rangle_{H_2} = \langle h_1, A^* h_2 \rangle_{H_1}, \quad \forall (h_1, h_2) \in (H_1 \times H_2).$$

We say that A is self-adjoint if $A^* = A$, or equivalently if

$$\langle Ah_1, h_2 \rangle_{H_2} = \langle h_1, Ah_2 \rangle_{H_1}, \quad \forall (h_1, h_2) \in (H_1 \times H_2).$$

Exercise

What are the adjoint of partial derivative operators ∂_x and ∂_y?

Continuous minimization of (5) (3/5)

For any $u, u' \in \mathbb{R}^{N^2}$, we have:

$$\begin{aligned}
\|H(u+u') - v\|_2^2 - \|Hu - v\|_2^2 &= \langle H(u+u') - v, H(u+u') - v\rangle - \|Hu - v\|_2^2 \\
&= \langle Hu - v + Hu', Hu - v + Hu'\rangle - \|Hu - v\|_2^2 \\
&= \langle Hu - v, Hu - v\rangle + 2\langle Hu', Hu - v\rangle \\
&+ \langle Hu', Hu'\rangle - \|Hu - v\|_2^2 \\
&= \langle u', 2H^*(Hu - v)\rangle + o(\|u'\|),
\end{aligned}$$

where $H^* : (\mathbb{R}^d)^{N^2} \to (\mathbb{R}^{d'})^{N'^2}$ is the adjoint operator of H.

$\to$ Usually, H^* is obtained by calculating $\langle Hu', u\rangle$.

Continuous minimization of (5) (4/5)

To obtain an approximate minimizer of (5), we need to calculate $\nabla E(u)$.

For any $u, u' \in \mathbb{R}^{d'N^2}$, we have:

$$\begin{aligned} TV_\epsilon(u+u') - TV_\epsilon(u) &= \sum_{1\leq i,j\leq N} \phi_\epsilon(|\nabla(u+u')_{i,j}|^2) - \phi_\epsilon(|\nabla u_{i,j}|^2) \\ &= \sum_{1\leq i,j\leq N} \phi_\epsilon(|\nabla(u)_{i,j}|^2 + 2\langle \nabla u_{i,j}, \nabla u'_{i,j}\rangle + o(|\nabla u'_{i,j}|)) - \phi_\epsilon(|\nabla u_{i,j}|^2) \\ &= \sum_{1\leq i,j\leq N} 2\phi'_\epsilon(|\nabla(u)_{i,j}|^2)\langle \nabla u_{i,j}, \nabla u'_{i,j}\rangle + o(|u'_{i,j}|). \end{aligned}$$

By denoting $X_{i,j} = 2\phi'_\epsilon(|\nabla u_{i,j}|^2)\partial_x u_{i,j}$ and $Y_{i,j} = 2\phi'_\epsilon(|\nabla u_{i,j}|^2)\partial_y u_{i,j}$, one ends up with

$$\begin{aligned} TV_\epsilon(u+u') - TV_\epsilon(u) &= \langle X, \partial_x u'\rangle + \langle Y, \partial_y u'\rangle + o(\|\nabla u'\|_1) \\ &= \langle u', \partial_x^* X + \partial_y^* Y\rangle + o(\|\nabla u'\|_1). \\ &= \left\langle u', \operatorname{div}\left(\frac{\nabla u^n}{\sqrt{\|\nabla u^n\|_2^2 + \epsilon}}\right)\right\rangle + o(\|u'\|_2). \end{aligned}$$

Continuous minimization of (5) (5/5)

Given $\epsilon \simeq 0$, gradient descent can be applied until $\|u^{n+1} - u^n\|_2^2 < T, \forall n \in \mathbb{N}$

$$u^{n+1} = u^n - \Delta t^n \underbrace{\left(\frac{H^*(Hu^n - v)}{\sqrt{(u^n - v)^2 + \epsilon}} - \beta \operatorname{div}\left(\frac{\nabla u^n}{\sqrt{\|\nabla u^n\|_2^2 + \epsilon}} \right) \right)}_{\nabla E(u^n)}, \quad \text{(TV+L1)},$$

$$u^{n+1} = u^n - \Delta t^n \underbrace{\left(H^*(Hu^n - v) - \beta \operatorname{div}\left(\frac{\nabla u^n}{\sqrt{\|\nabla u^n\|_2^2 + \epsilon}} \right) \right)}_{\nabla E(u^n)}, \quad \text{(TV+L2)},$$

where $\Delta t^n \in \mathbb{R}_{\geq 0}$ denotes the step size at iteration n.

Discrete minimization of (5)

- Given a total and finite ordered set of labels $\mathcal{L} = \{0, \dots, L-1\}$ and neighborhood relationships $\mathcal{N}$, $u^* \in (\mathcal{L}^{d'})^{\mathcal{P}}$ is taken as a minimizer of

$$\min_{u \in \mathcal{L}^{d'}{}^{\mathcal{P}}} \|Hu - v\|_p^p + \beta \sum_{i=1}^{d'} \sum_{(p,q) \in \mathcal{N}} w_{p,q} |u_p^i - u_q^i|,$$

  where $u^i \in \mathbb{R}^{N^2}$ denotes the ith channel of u and $w_{p,q} \in \mathbb{R}_{>0}$ are coefficients.
- Whatever p and since TV is convex, α-expansion [BVZ01] or more efficient schemes can be applied [Cha05, Dar05, CD09, LMHT12, RLLHM18].

Exercise

What are the edge capacities of the graph $\mathcal{G}$?

Image denoising

- We remind that corresponds to the case where $N = N'$ and $H = I$.
- For simplicity, we consider $d = d' = 1$ (grayscale images).



Noise-free image u

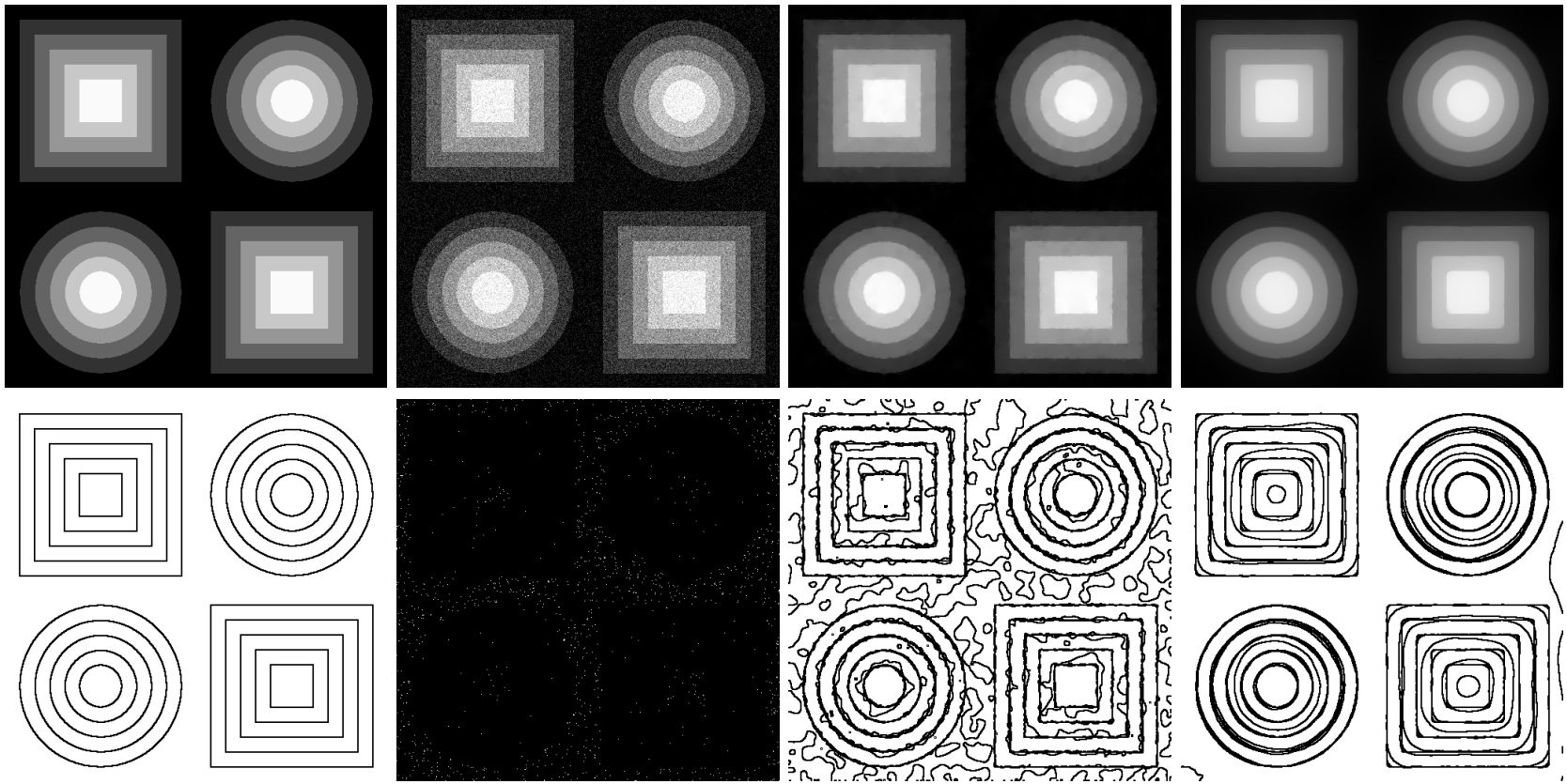


Noisy image v



Approximation u^*

Figure 43: Example of image denoising.



Source image $\beta = 0$ $\beta = 1$ $\beta = 1.4$

Figure 44: Intensity images (top row) and 20 uniformly sampled level lines (bottom row) of TV+L^2 denoising in the continuous domain ($\sigma^2 = 0.01$) using $\epsilon = 10^{-3}$, $T = 10^{-9}$, $\Delta t = 2 \times 10^{-4}$.



Source image $\beta = 0$ $\beta = 0.4$ $\beta = 1.2$

Figure 45: Intensity images (top row) and 20 uniformly sampled level lines (bottom row) of TV+L^2 denoising in the continuous domain ($\sigma^2 = 0.01$) using $\epsilon = 10^{-3}$, $T = 10^{-9}$, $\Delta t = 2 \times 10^{-4}$.

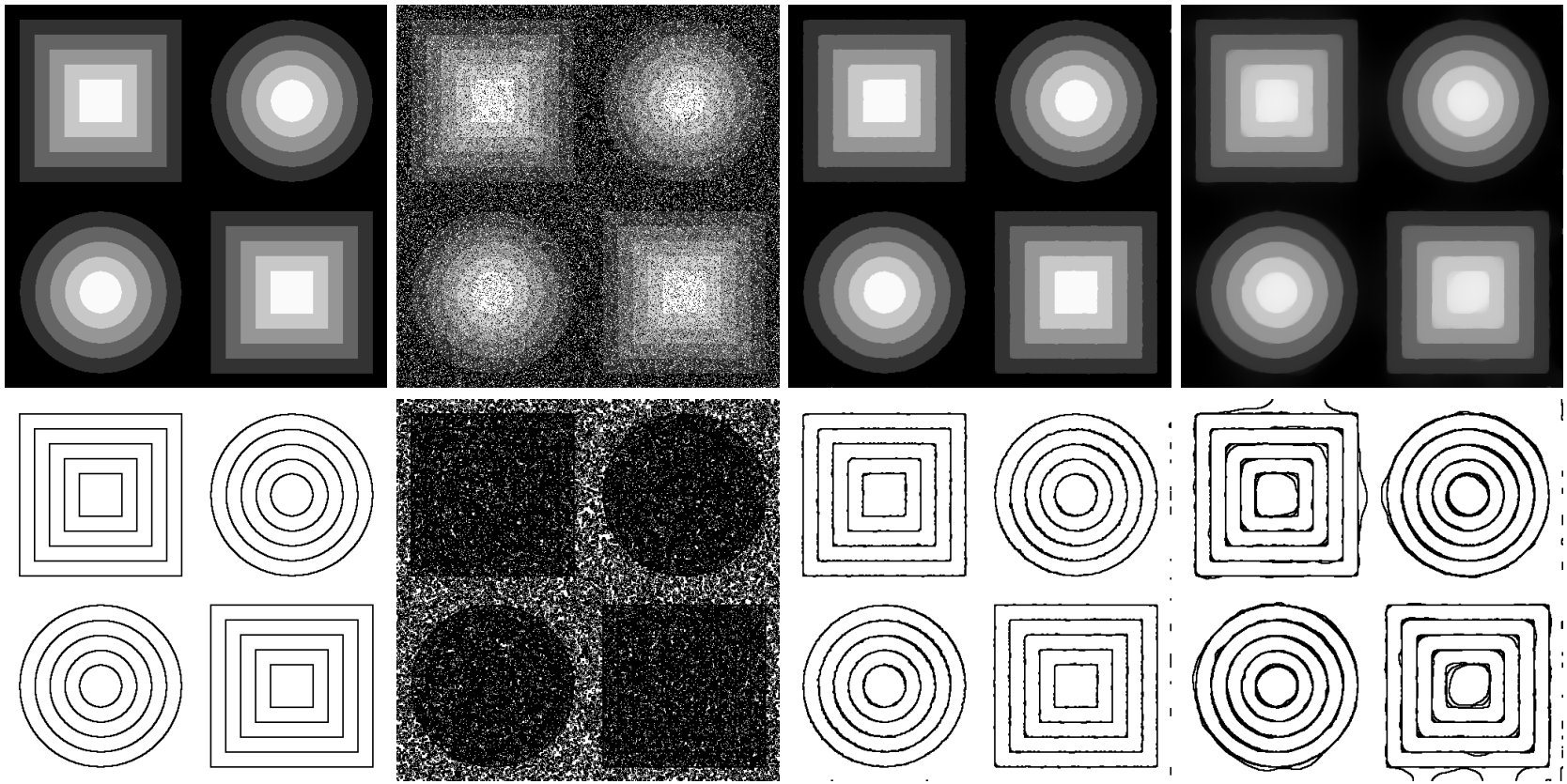
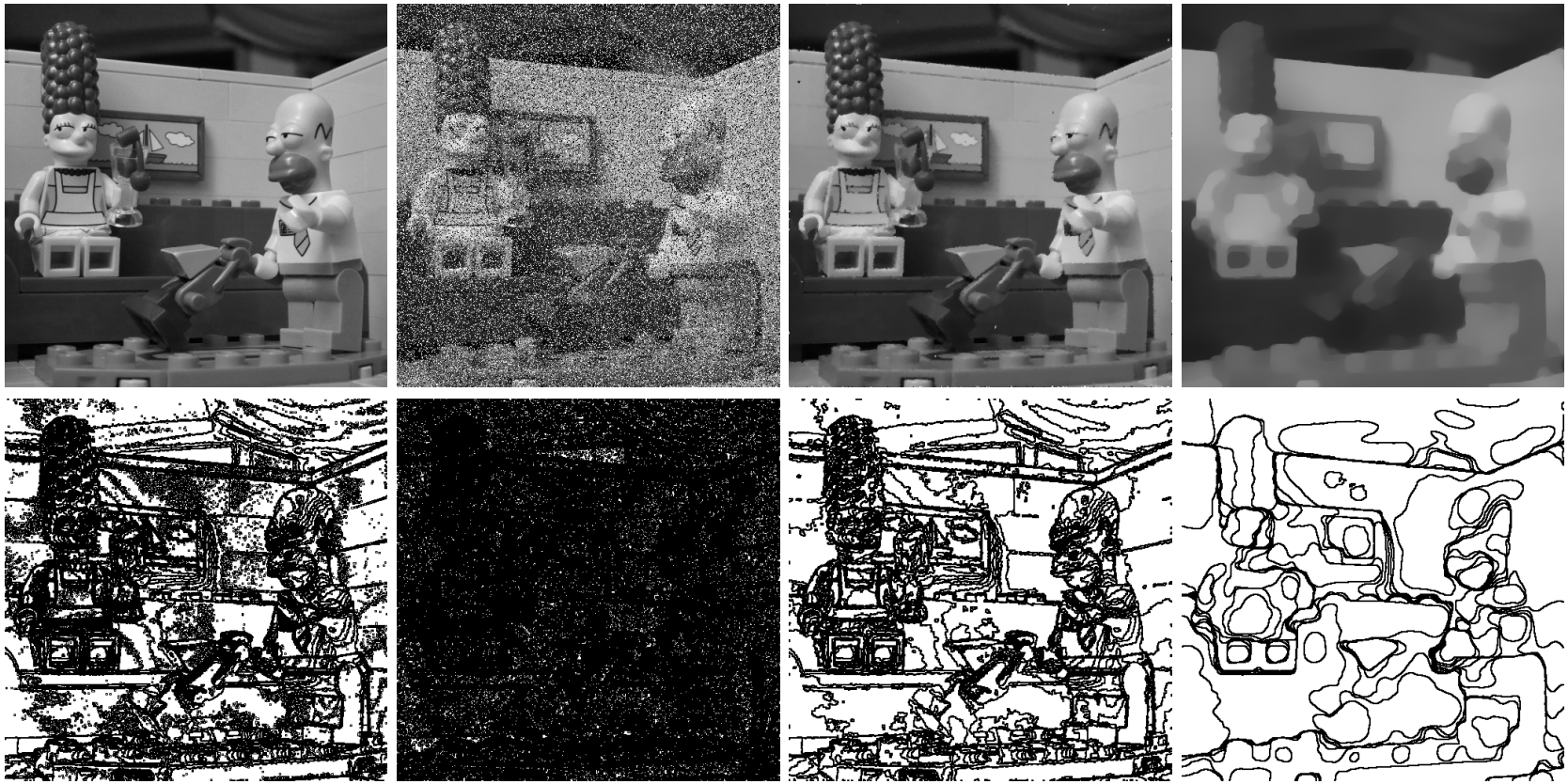
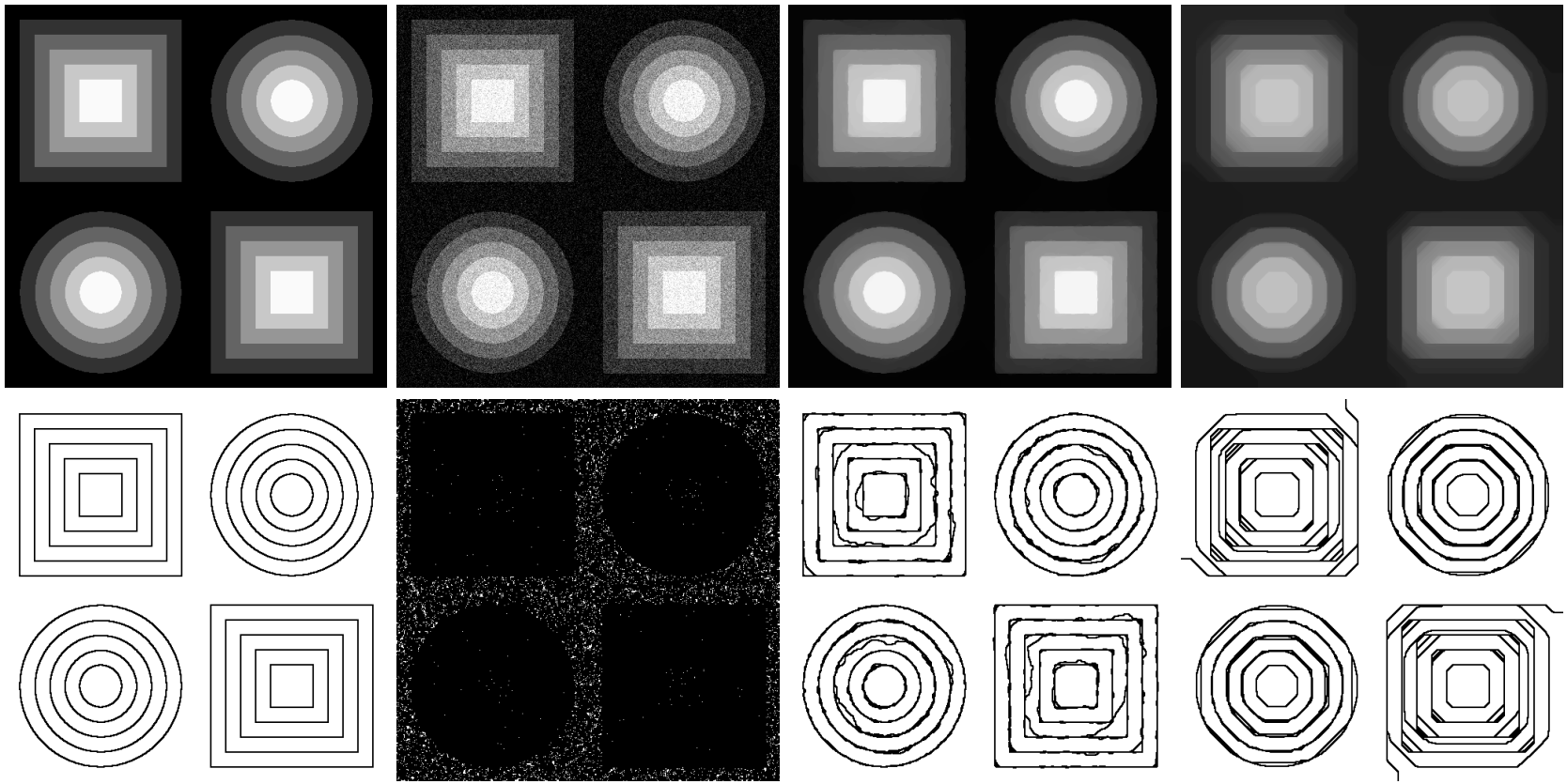


Figure 46: Intensity images (top row) and 20 uniformly sampled level lines (bottom row) of TV+L^1 denoising in the continuous domain ($d = 30\%$) using $\epsilon = 10^{-3}$, $T = 10^{-9}$, $\Delta t = 5 \times 10^{-4}$.



Source image $\beta = 0$ $\beta = 0.8$ $\beta = 6$

Figure 47: Intensity images (top row) and 20 uniformly sampled level lines (bottom row) of TV+L^1 denoising in the continuous domain ($d = 30\%$) using $\epsilon = 10^{-3}$, $T = 10^{-9}$, $\Delta t = 5 \times 10^{-4}$.



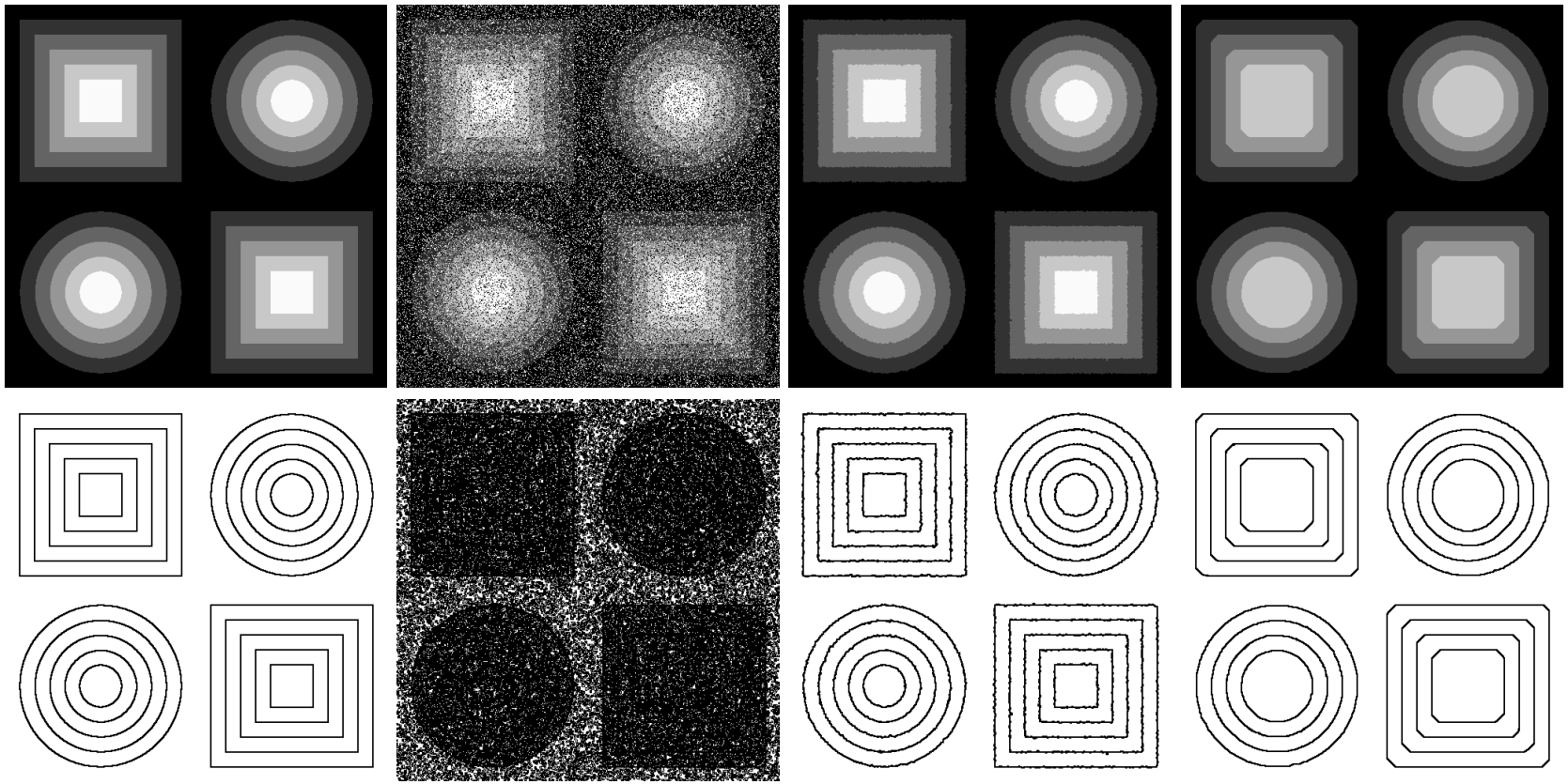
Source image $\beta = 0$ $\beta = 0.001$ $\beta = 0.01$

Figure 48: Intensity images (top row) and 20 uniformly sampled level lines (bottom row) of TV+L^2 denoising in the discrete domain ($\sigma^2 = 0.01$) using connectivity 1 and $L = 256$.



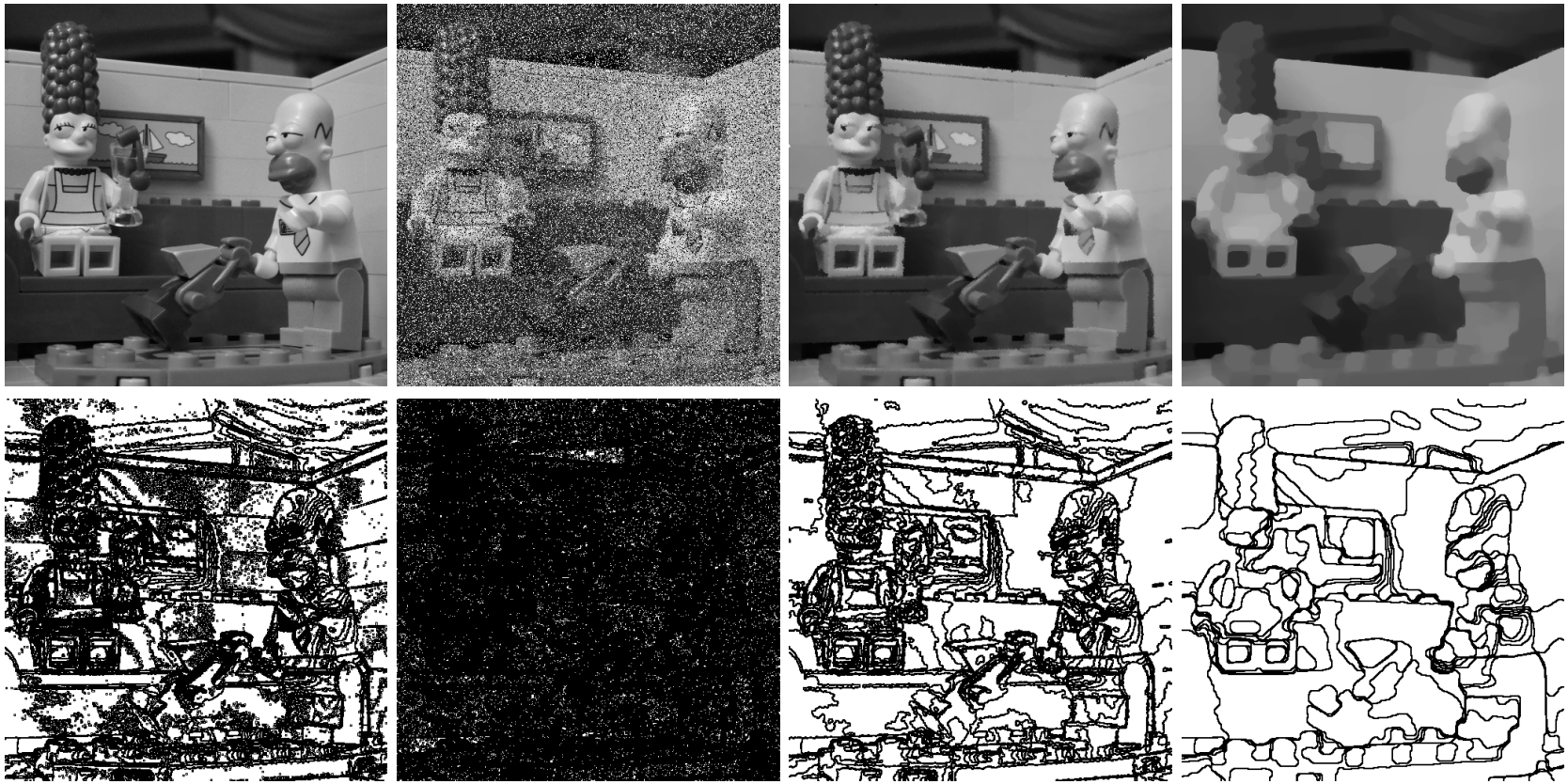
Source image $\beta = 0$ $\beta = 0.00025$ $\beta = 0.002$

Figure 49: Intensity images (top row) and 20 uniformly sampled level lines (bottom row) of TV+L^2 denoising in the discrete domain ($\sigma^2 = 0.01$) using connectivity 1 and $L = 256$.



Source image $\beta = 0$ $\beta = 0.0015$ $\beta = 0.02$

Figure 50: Intensity images (top row) and 20 uniformly sampled level lines (bottom row) of TV+L^1 denoising in the discrete domain ($d = 30\%$) using connectivity 1 and $L = 256$.



Source image $\beta = 0$ $\beta = 0.0015$ $\beta = 0.006$

Figure 51: Intensity images (top row) and 20 uniformly sampled level lines (bottom row) of TV+L^1 denoising in the discrete domain ($d = 30\%$) using connectivity 1 and $L = 256$.

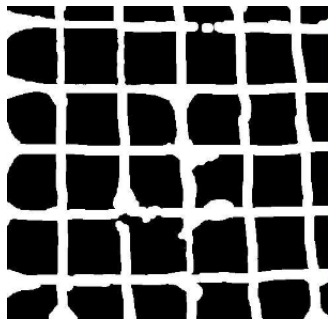
Image inpainting

- This corresponds to the case where $N = N'$ and H is a linear operator.
- $H : \mathcal{L}^{\mathcal{P}} \to \mathcal{L}^{\mathcal{P}}$ is a masking operator such that $v = Hu$ is defined by

$$v_p = \begin{cases} u_p & \text{if} \quad m_p = 0 \\ 0 & \text{if} \quad m_p = 1 \end{cases}, \quad p \in \mathcal{P},$$

  where $m : \mathcal{P} \to \{0, 1\}$ represents a given mask image.
- For simplicity, we consider $d = d' = 1$ (grayscale images).
- Exercise: What is the adjoint operator H^* of H?


Unmasked image u


Mask H


Masked image v


Approximation u^*

Figure 52: Example of image inpainting.

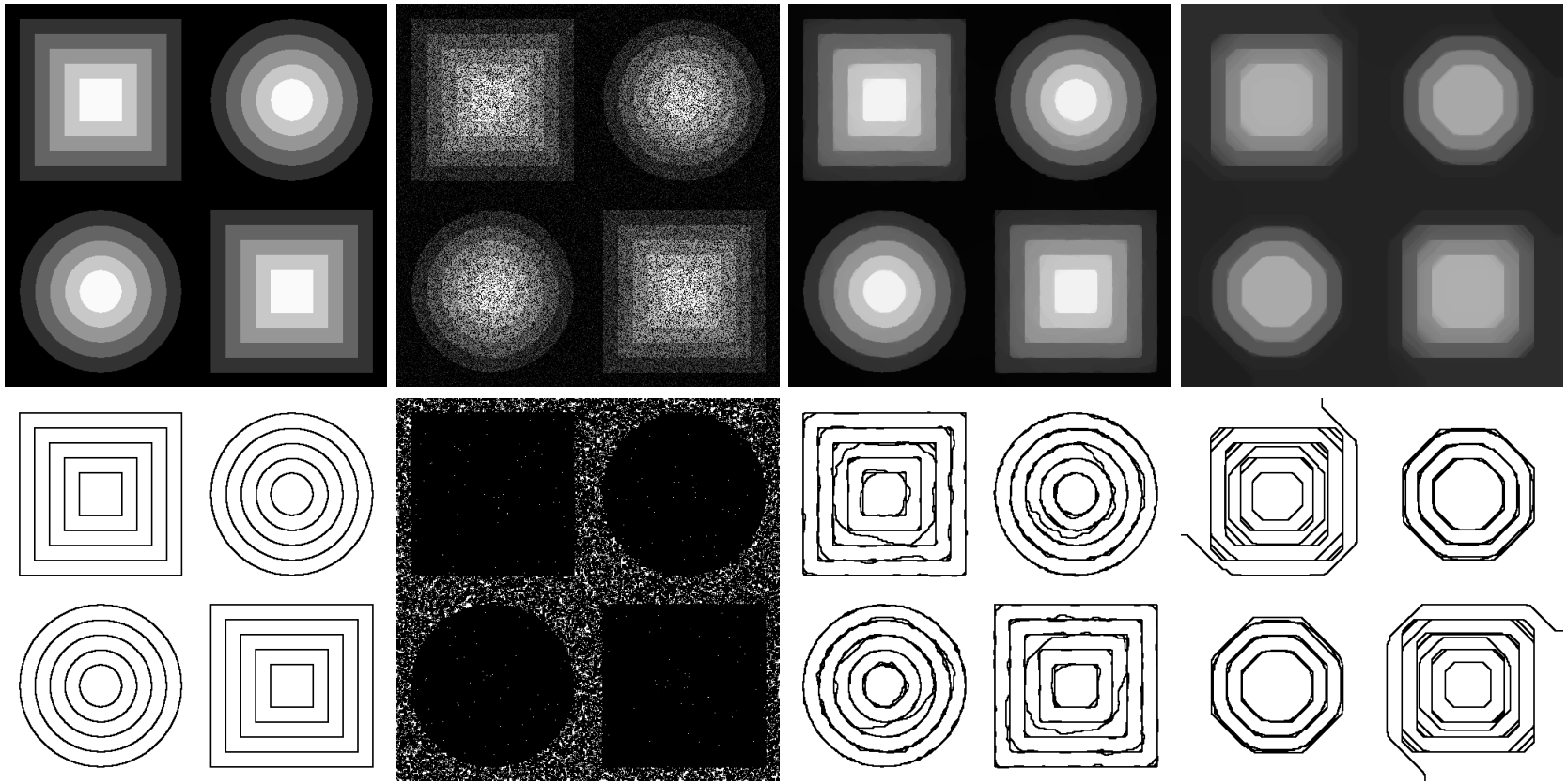
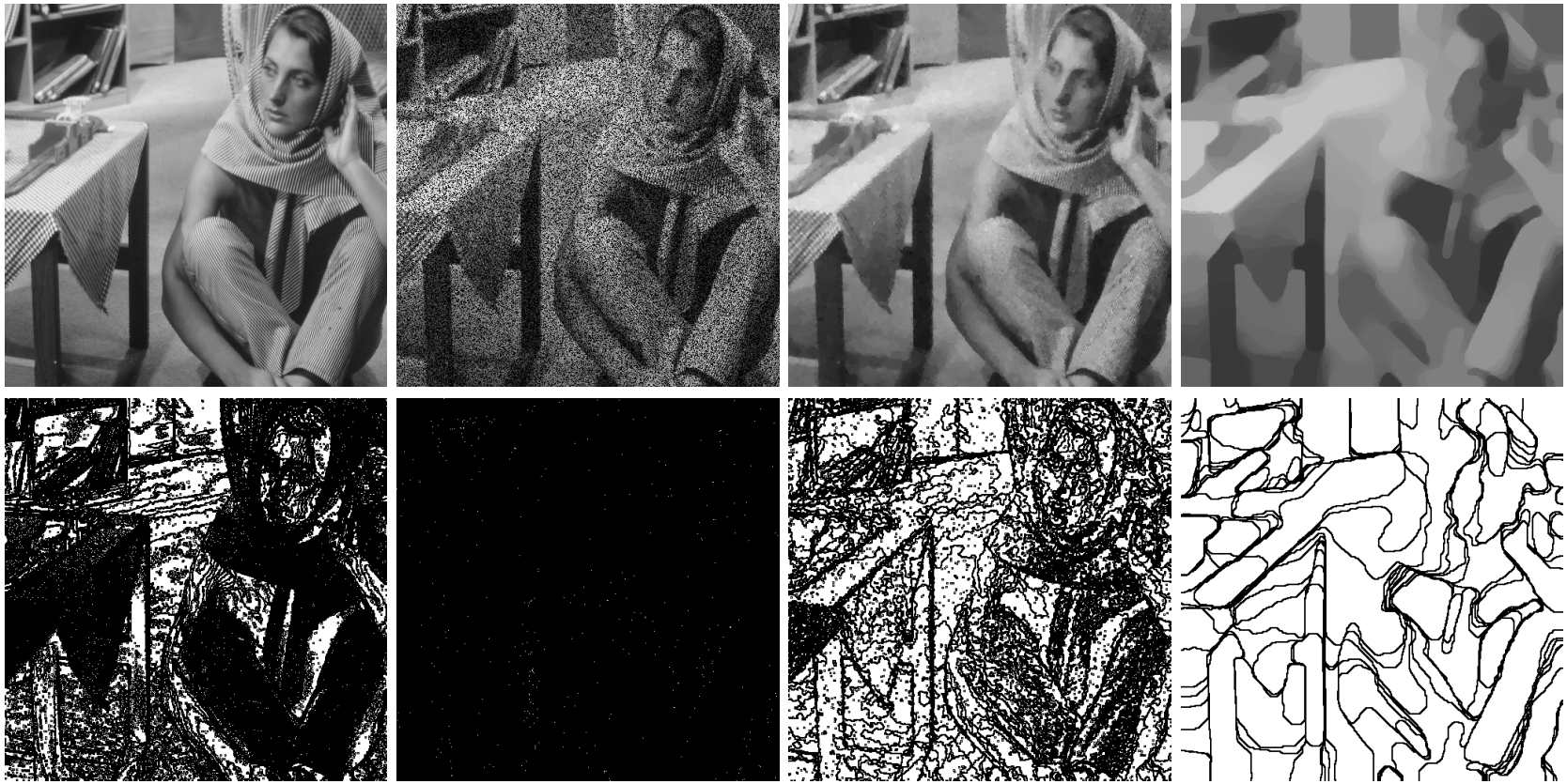
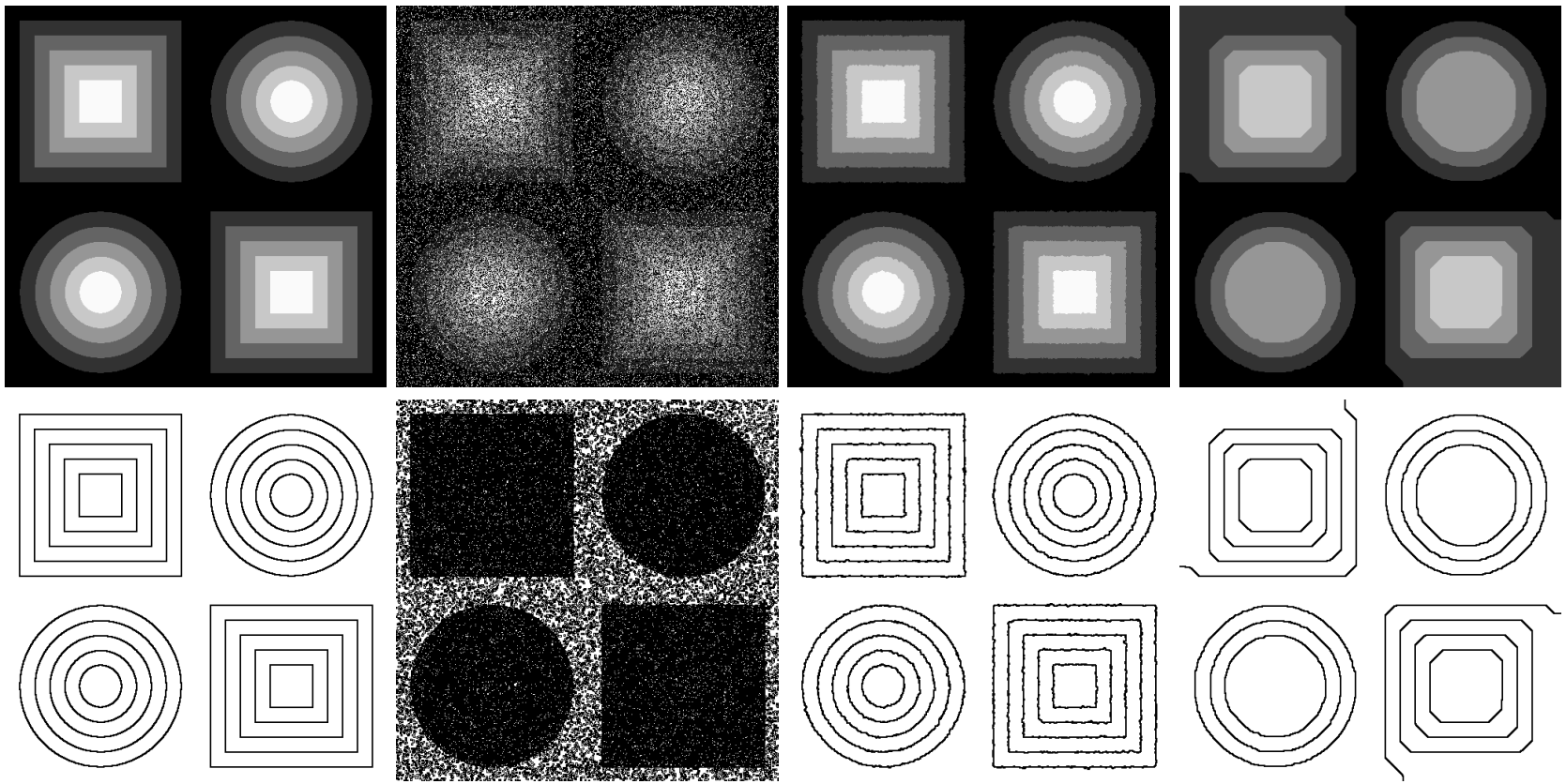


Figure 53: Intensity images (top row) and associated 20 uniformly sampled level lines (bottom row) of TV+L^2 inpainting in the discrete domain ($\sigma^2 = 0.01$, $MP = 30\%$) using connectivity 1 and $L = 256$.



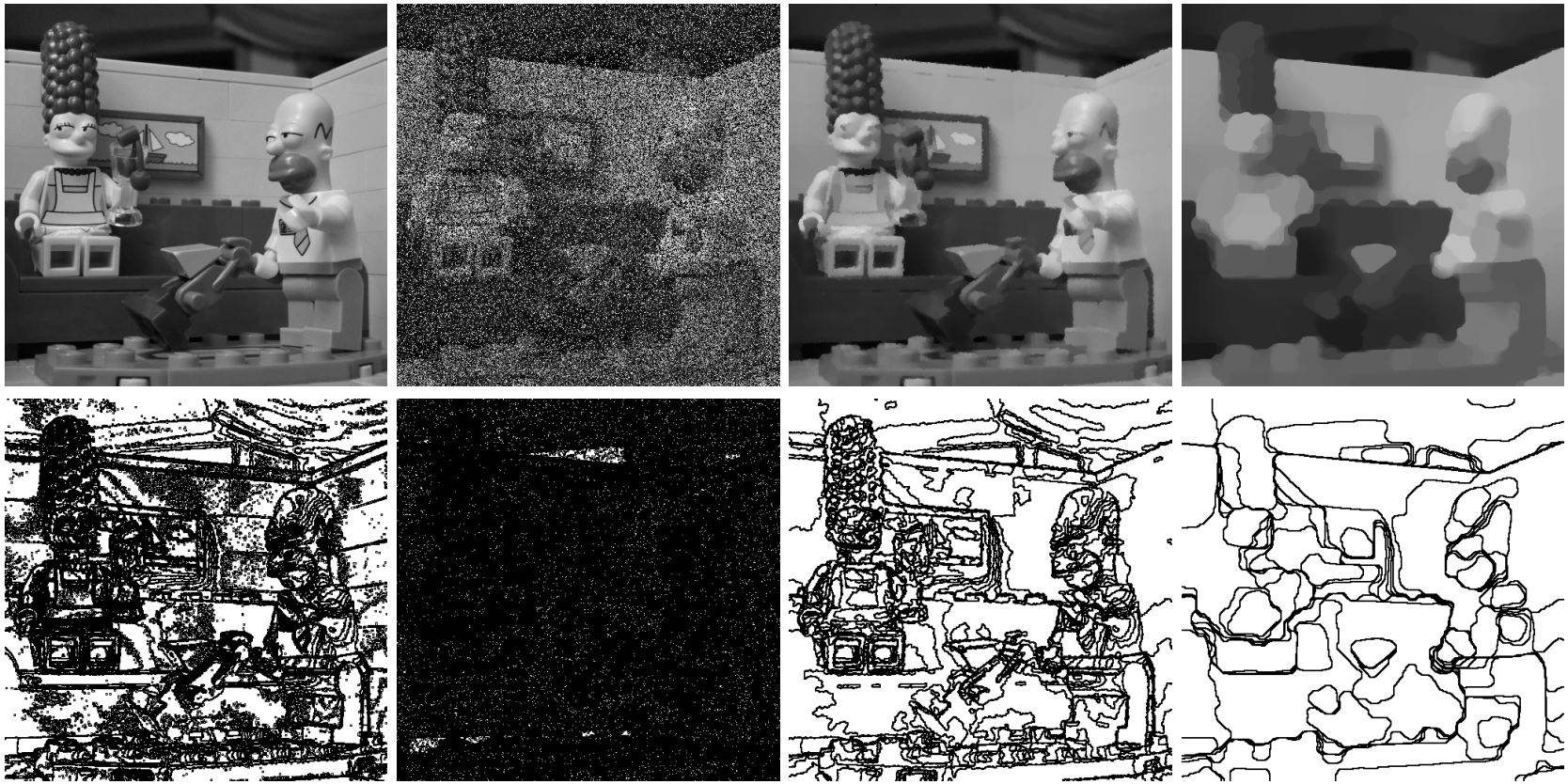
Source image $\beta = 0$ $\beta = 0.00025$ $\beta = 0.002$

Figure 54: Intensity images (top row) and associated 20 uniformly sampled level lines (bottom row) of TV+L^2 inpainting in the discrete domain ($\sigma^2 = 0.01$, $MP = 30\%$) using connectivity 1 and $L = 256$.



Source image $\beta = 0$ $\beta = 0.0015$ $\beta = 0.02$

Figure 55: Intensity images (top row) and associated 20 uniformly sampled level lines (bottom row) of TV+L^1 inpainting in the discrete domain ($d = 30\%$, $MP = 30\%$) using connectivity 1 and $L = 256$.



Source image $\beta = 0$ $\beta = 0.0015$ $\beta = 0.006$

Figure 56: Intensity images (top row) and associated 20 uniformly sampled level lines (bottom row) of TV+L^1 inpainting in the discrete domain ($d = 30\%$, $MP = 30\%$) using connectivity 1 and $L = 256$.

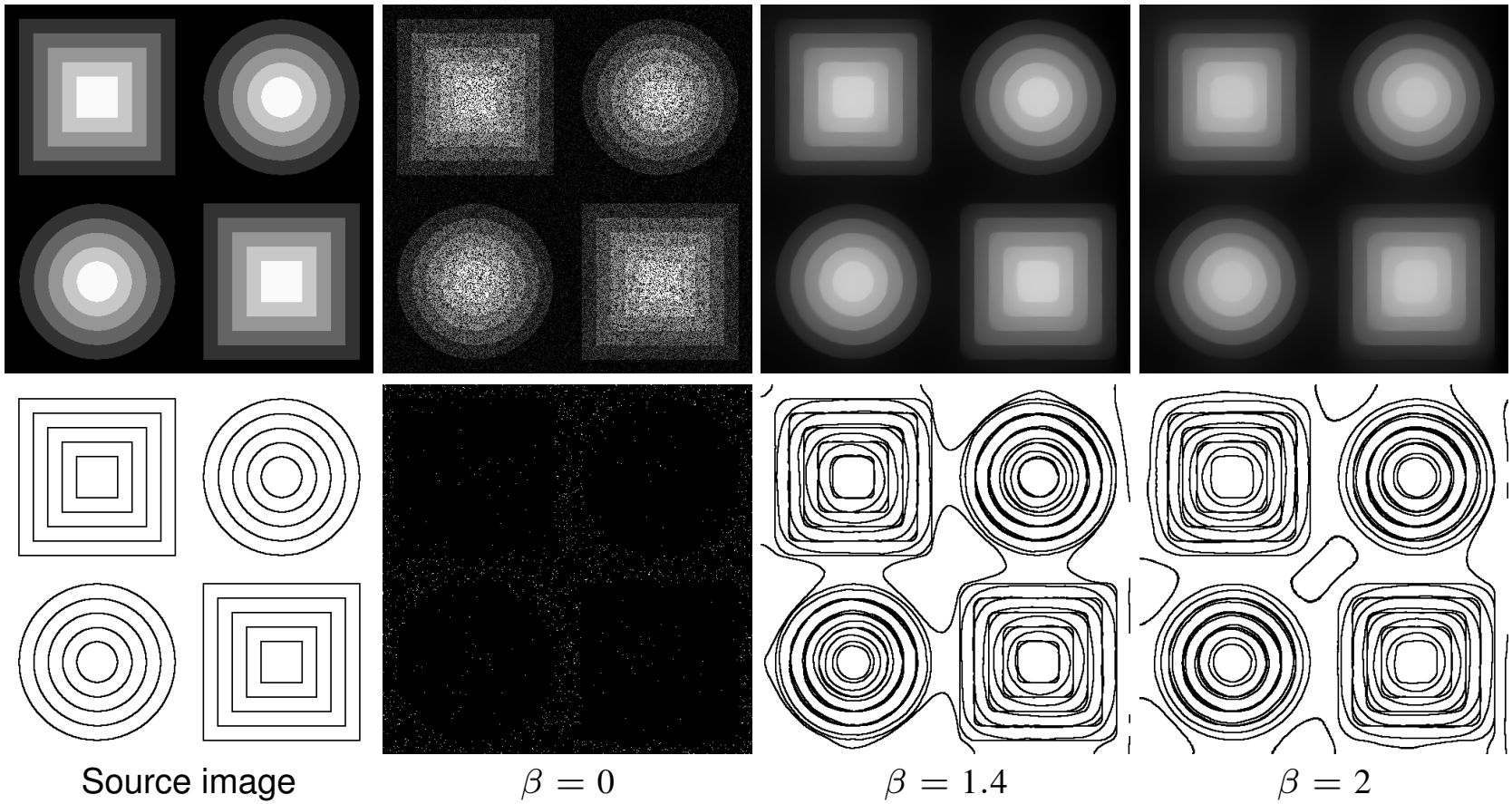
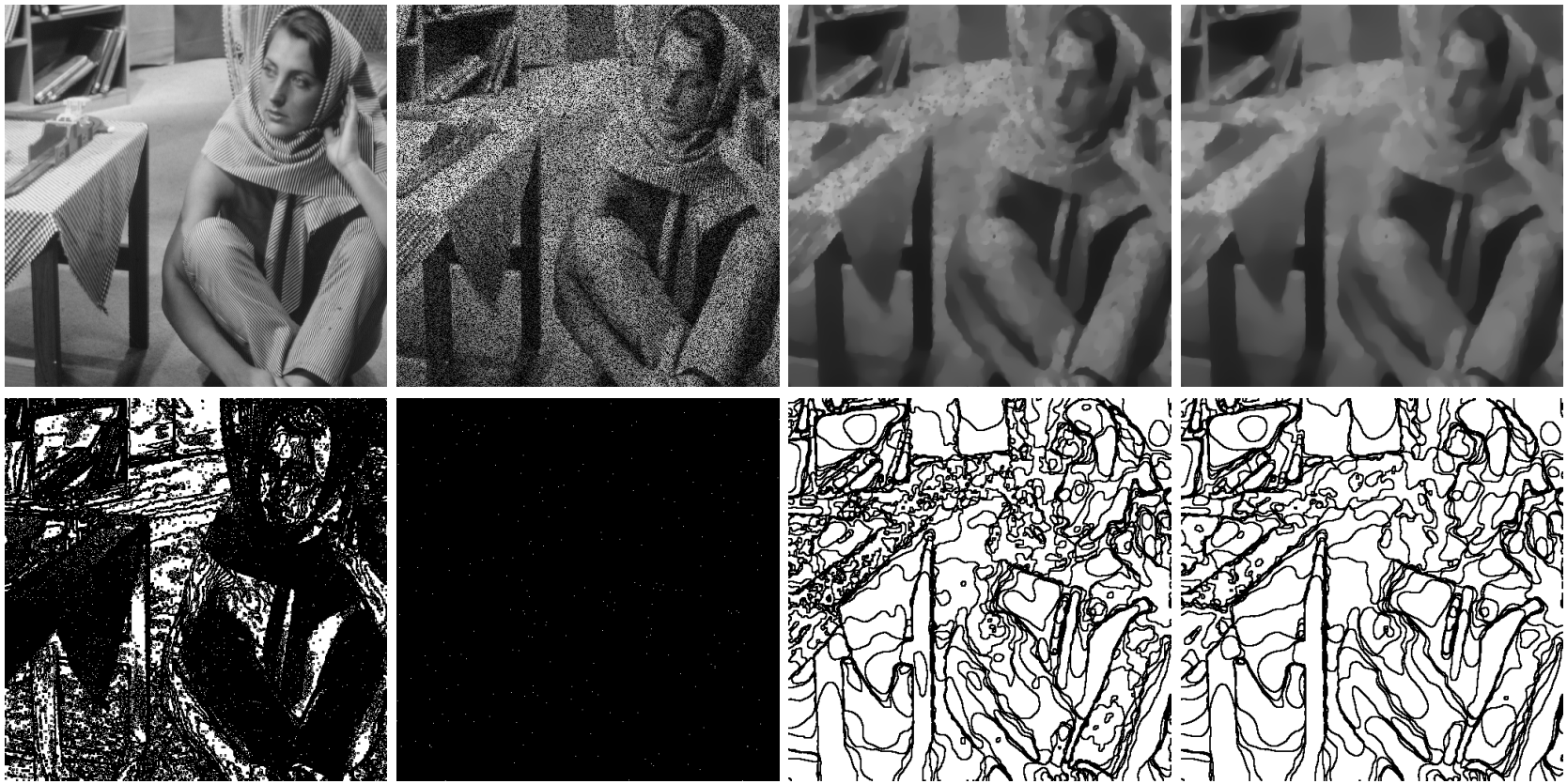


Figure 57: Intensity images (top row) and 20 uniformly sampled level lines (bottom row) of TV+L^2 inpainting in the continuous domain ($\sigma^2 = 0.01$, $MP = 30\%$) using $\epsilon = 10^{-3}$, $T = 10^{-9}$, $\Delta t = 2 \times 10^{-4}$.



Source image $\beta = 0$ $\beta = 0.8$ $\beta = 1.2$

Figure 58: Intensity images (top row) and 20 uniformly sampled level lines (bottom row) of TV+L^2 inpainting in the continuous domain ($\sigma^2 = 0.01$, $MP = 30\%$) using $\epsilon = 10^{-3}$, $T = 10^{-9}$, $\Delta t = 2 \times 10^{-4}$.

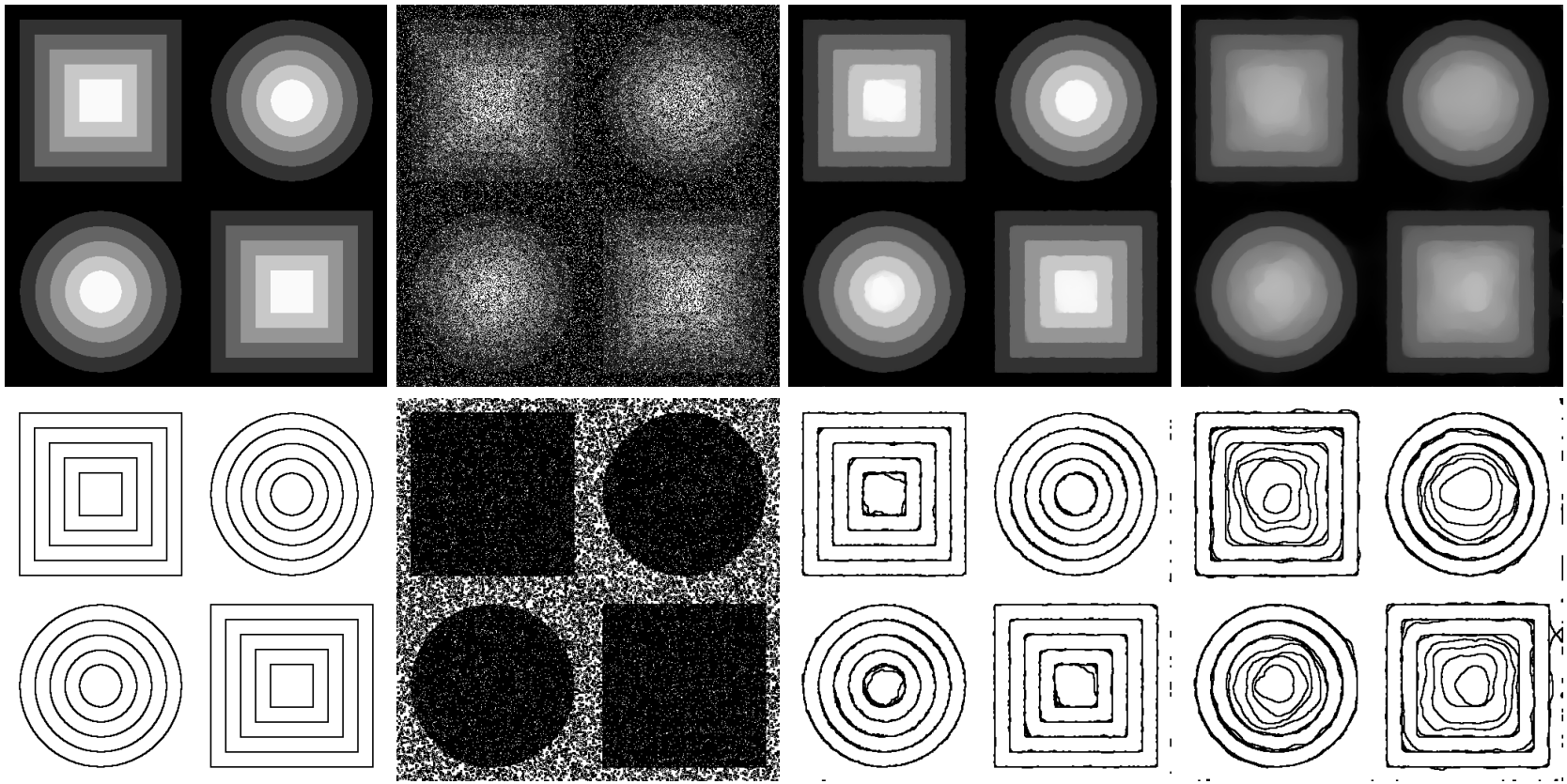
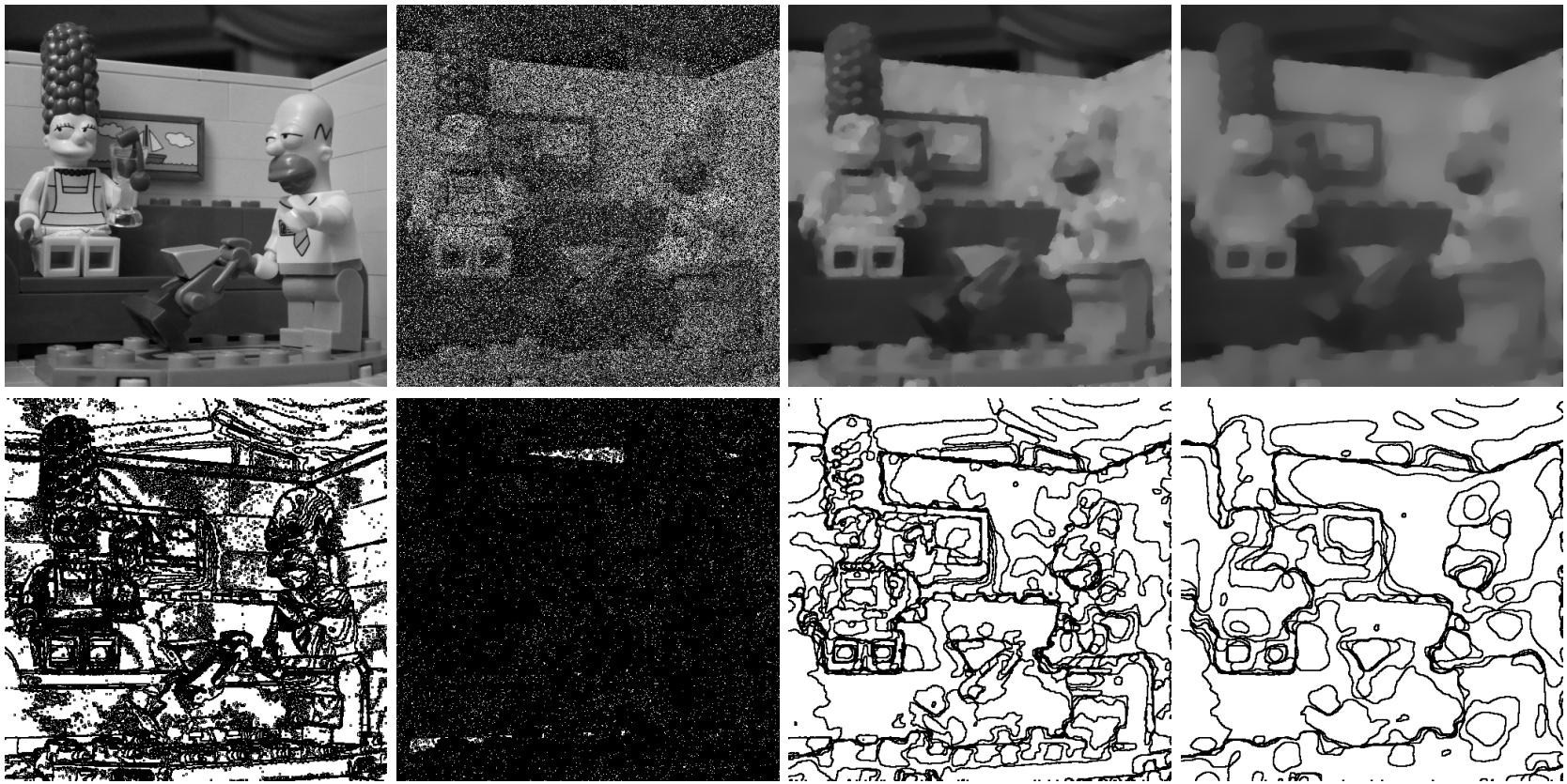


Figure 59: Intensity images (top row) and 20 uniformly sampled level lines (bottom row) of TV+L^1 inpainting in the continuous domain ($d = 30\%$, $MP = 30\%$) using $\epsilon = 10^{-3}$, $T = 10^{-9}$, $\Delta t = 5 \times 10^{-4}$.



Source image $\beta = 0$ $\beta = 1$ $\beta = 4$

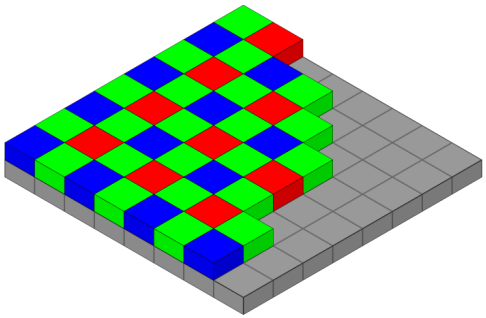
Figure 60: Intensity images (top row) and 20 uniformly sampled level lines (bottom row) of TV+L^1 inpainting in the continuous domain ($d = 30\%$, $MP = 30\%$) using $\epsilon = 10^{-3}$, $T = 10^{-9}$, $\Delta t = 5 \times 10^{-4}$.

Image demosaicing

- This corresponds to the case where $N = N'$ and H is a linear operator.
- $H : (\mathcal{L}^3)^{\mathcal{P}} \to \mathcal{L}^{\mathcal{P}}$ is a linear operator such that $v = Hu$ is defined by

$$v_p = \begin{cases} u_p^b & \text{if } i\%2 = 0 \text{ and } j\%2 = 0, \\ u_p^r & \text{if } i\%2 = 1 \text{ and } j\%2 = 1, \\ u_p^g & \text{otherwise.} \end{cases}, \quad \forall p = (i,j) \in \mathcal{P},$$

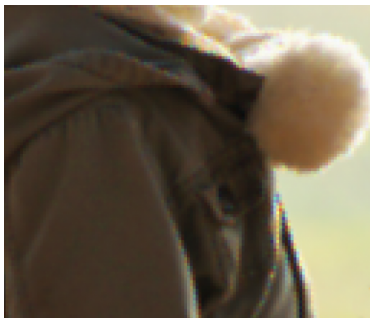
- Additionally, we have $d = 1$ and $d' = 3$.
- Exercise: What is the adjoint operator H^* of H?



Operator H

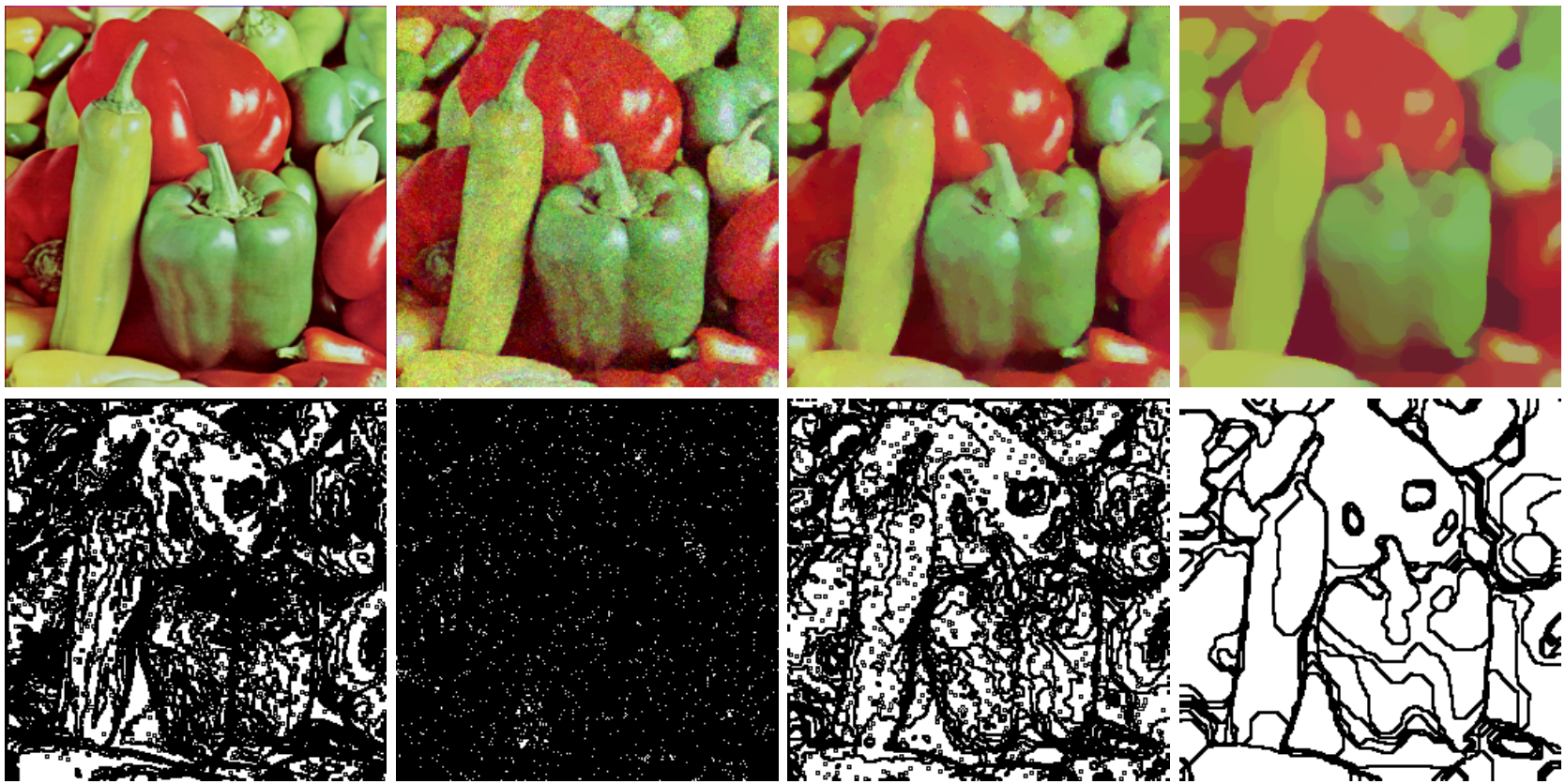


Observation v



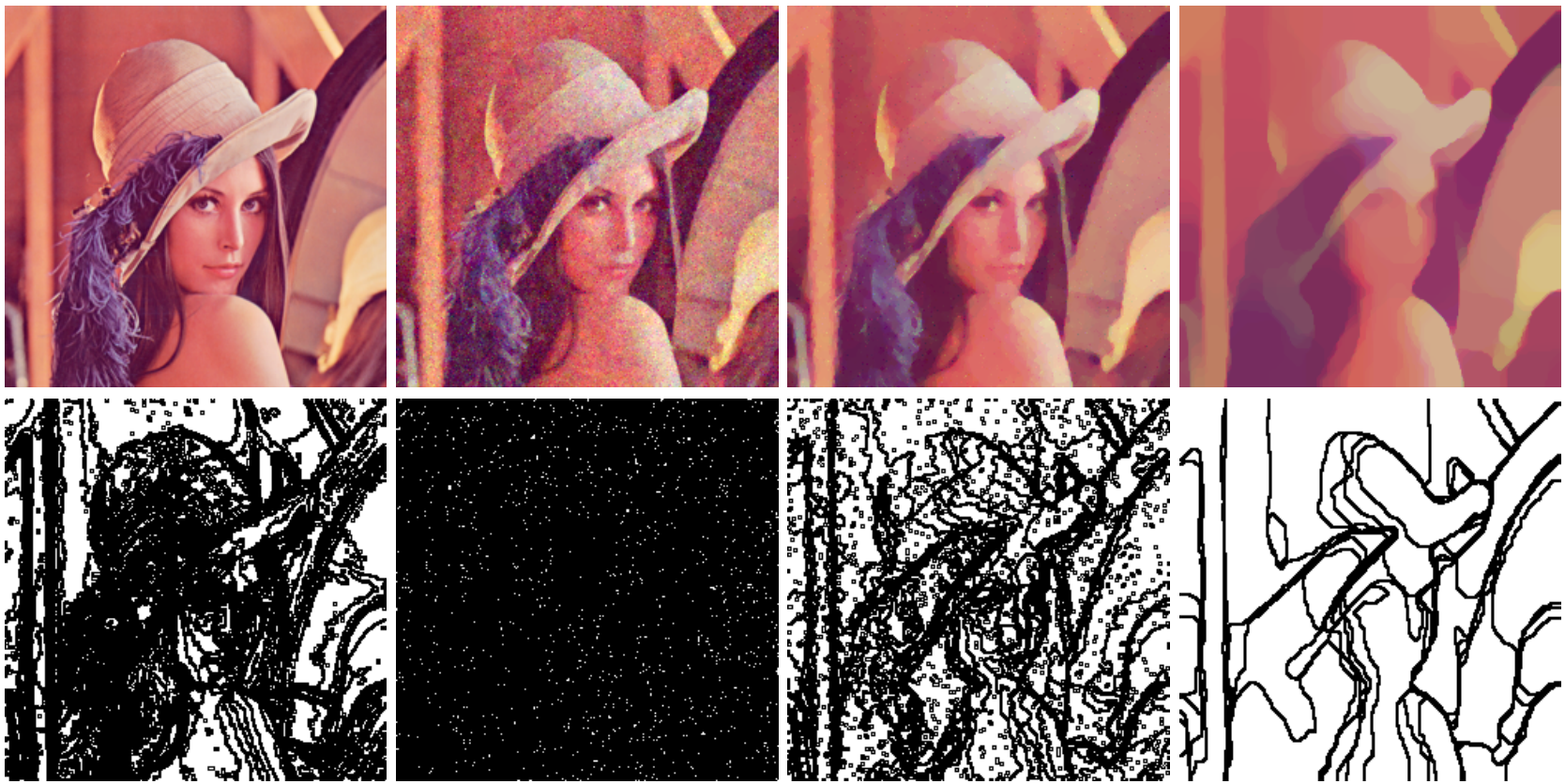
Approximation u^*

Figure 61: Example of image demosaicing.



Source image $\beta = 100000$ $\beta = 5000$ $\beta = 1000$

Figure 62: Intensity images (top row) and associated 20 uniformly sampled level lines (bottom row) of TV+L^2 demosaicing in the discrete domain ($\sigma^2 = 0.01$) using connectivity 1 and $L = 256$.



Source image $\beta = 100000$ $\beta = 5000$ $\beta = 1000$

Figure 63: Intensity images (top row) and associated 20 uniformly sampled level lines (bottom row) of TV+L^2 demosaicing in the discrete domain ($\sigma^2 = 0.01$) using connectivity 1 and $L = 256$.

General remarks

- As $\beta \to \infty$, images become cartoon-like (sharp jumps between flat regions),
- β is a scale parameter allowing us to select the desired level of details,
- TV suffers from "starcasing effect" and geometry loss,
- TV+L2 model suffers from a contrast loss (see [CEPY04] for solutions).

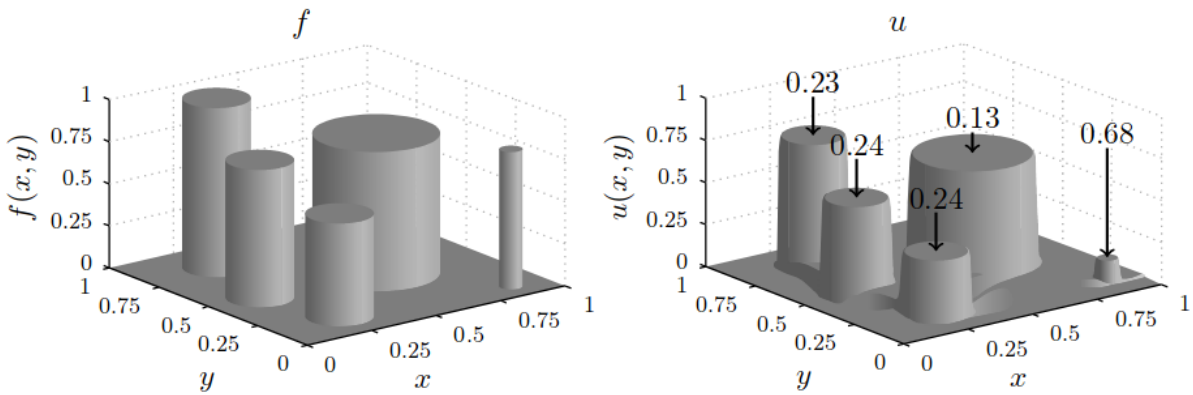


Figure 64: Contrast loss on a toy example with TV+L2 model (courtesy of P. Getreuer). It has been shown that intensity within a disk of radius $r > 0$ decreases by $2/(\beta r)$.

Tuning of parameter β

No closed-form formula to get a of β. Among possible strategies, we can:

- Select β to optimize some criterion of the denoising result (e.g. MSE).
- Select β to match the noise variance σ^2. For doing so, [Cha04] propose an efficient iterative algorithm. One iterates until convergence
  1. Compute u^{n+1} as an approximate minimizer of (5).
  2. $\beta^{n+1} \leftarrow \frac{\beta \|v - u^{n+1}\|}{N^2 \sigma^2}$.

  $\rightarrow$ The sequence $(\beta)_{i \in \mathbb{N}}$ converges to the unique β such that $\sigma(v - u^*) = \sigma^2$.
  $\rightarrow$ Convergence is theoretically guaranteed only when L^2 norm is used in (5).

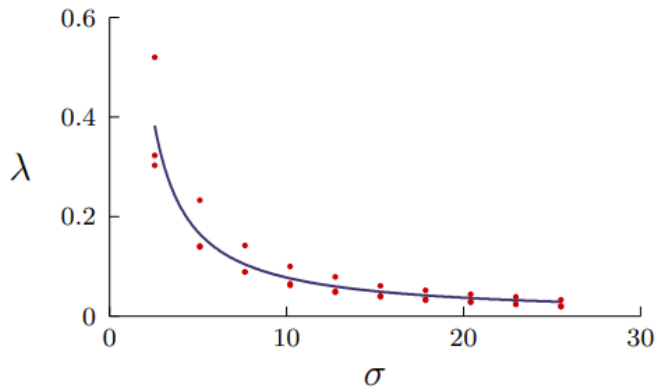


Figure 65: Optimal values of β (dots) and empirical estimate (solid line).

Definition

It is the process of partitioning an image $v : \mathcal{P} \subset \mathbb{R}^2 \to \mathbb{R}^d$ into N (non necessarily connected) regions. The resulting partition is a simplification of v and must verify that

- $\cup_{i=1}^{N} R_i = \mathcal{P}$,
- $\forall (i,j) \in \{1,\dots,N\}^2$ s.t. $i \neq j$, $R_i \cap R_j = \emptyset$.

Equivalently, we want to assign a unique label $l \in \{1,\dots,N\}$ to any pixel $p \in \mathcal{P}$.

Interest?

Ease the analysis of objects in an image, allowing us for instance to

- Count the number of objects of a certain type.
- Measure geometric properties (area, perimeter, holes, etc.) of objects.
- Study properties of an individual object (intensity, texture, etc.).

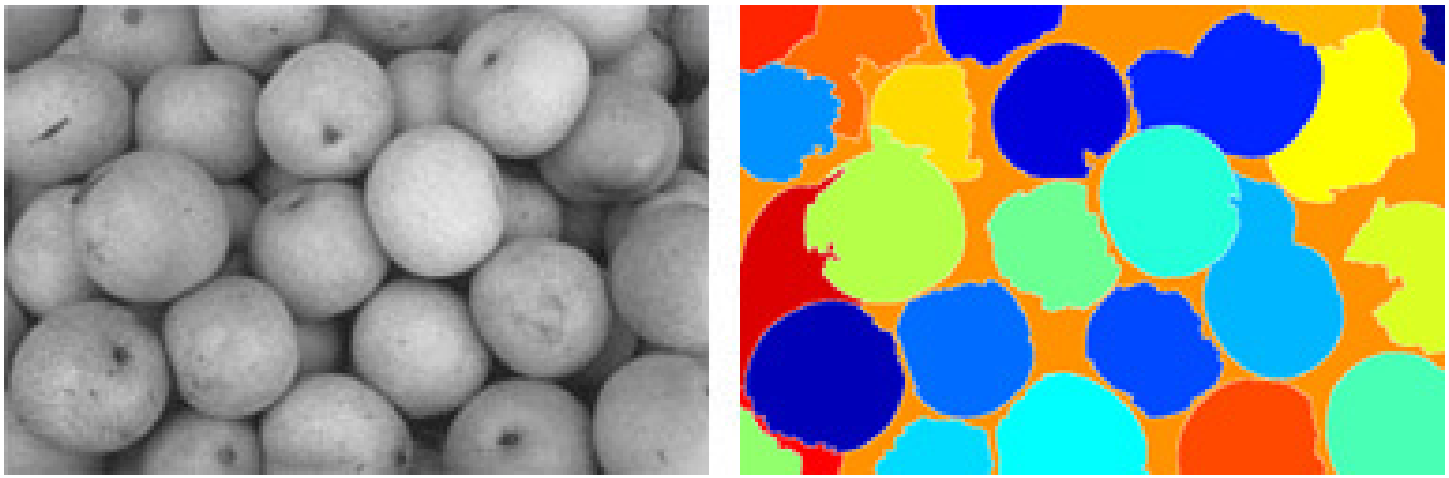


Figure 66: Example of a segmentation (right) of a natural image (left).

Remarks

- Segmentation is one of the hardest problem to solve in image analysis.
- Duality between regions and contours $\Rightarrow$ two approaches:
  - Region-based approaches look for homogeneities.
  - Contour-based approaches look for discontinuities.
- Research topic still of great interest (1.6M of results in Google scholar).
- Segmentation is a natural and powerful process in animals.
- Is it possible to mimic this process with a machine?

Remarks

- Image segmentation is a difficult task to solve for a machine!
- Several segmentations can exist for the same image.
- Requires a true understanding of the objects in a scene.
- Ill-posed problem $\Rightarrow$ necessity to introduce some a priori knowledge.



Figure 67: What is the "good" segmentation of this image?

Chan-Vese model - Formulation

This model aims at minimizing among $u \in \{0,1\}^{\mathcal{P}}$, $c_0 \in \mathbb{R}$ and $c_1 \in \mathbb{R}$ [CV99]

$$E(u, c_0, c_1) = \sum_{p \in \mathcal{P}} E_p(u_p) + \nu \sum_{p \in \mathcal{P}} E'_p(u_p) + \beta \sum_{(p,q) \in \mathcal{N}} E_{p,q}(u_p, u_q), \quad \text{where,}$$

$$E_p(u_p) = \begin{cases} \|v_p - c_0\|_2^2 & \text{if} \quad u_p = 0 \\ \|v_p - c_1\|_2^2 & \text{if} \quad u_p = 1 \end{cases}, \quad E'_p(u_p) = \begin{cases} 1 & \text{if} \quad u_p = 0 \\ 0 & \text{if} \quad u_p = 1 \end{cases},$$

$$E_{p,q}(u_p, u_q) = w_{p,q}|u_p - u_q|, \text{ and where,}$$

- c_0, c_1 denote mean intensity in object (label 0) and background (label) 1 resp.
- $\nu \in \mathbb{R}$, $\beta \in \mathbb{R}^+$ are weighting parameters that need to be tuned.
- $w_{p,q} \in \mathbb{R}_{>0}$ are coeff. approximating the boundary length of $\{p \in \mathcal{P} \mid u_p = 0\}$.

Chan-Vese model – Role of the energy terms

- First term penalize the discrepancy between piecewise constant model u and v.
- Second term penalizes the area of $\{p \in \mathcal{P} \mid u_p = 0\}$.
- Third term penalizes the length of $\{p \in \mathcal{P} \mid u_p = 0\}$.

$\rightarrow$ A minimizer of E can be found by alternating minimization on u, c_0 and c_1.

Chan-Vese model – Remarks

- When u is assumed to be piecewise constant and $\nu = 0 \Rightarrow$ Mumford-Shah model.
- By computing the critical points of $\partial E / \partial c_0$ and $\partial E / \partial c_1$, one ends up with

$$c_0 = \frac{\sum_{p \in \mathcal{P}} v_p}{\sharp\{p \in \mathcal{P} \mid u_p = 0\}}, \quad \text{and} \quad c_1 = \frac{\sum_{p \in \mathcal{P}} v_p}{\sharp\{p \in \mathcal{P} \mid u_p = 1\}},$$

- Extension to color images is straightforward [CSV00].
- Minimization with more than two labels is possible [VC02].
- Variance of Gaussian distributions can be taken into account.
- Parameters can be locally estimated to account for inhomogeneities [WHX10].

Algorithm 6: Binary segmentation using Chan-Vese model

INPUTS: $v \in \mathbb{R}^{\mathcal{P}}$, $\nu \in \mathbb{R}$, $\beta \in \mathbb{R}^+$, $\epsilon \in \mathbb{R}_{>0}$
OUTPUTS: An approximate minimizer of E

Initialize c_0^1, c_1^1 *% (e.g. using [weighted] k-means procedure)*
Let $u_p^0 = +\infty, \forall p \in \mathcal{P}$
while $\|u^n - u^{n-1}\|_2^2 > \epsilon \sharp \mathcal{P}$ **do**
    $u^n \in \operatorname{argmin}_{u \in \{0,1\}^{\mathcal{P}}} E(u, c_0^n, c_1^n)$
    $c_0^{n+1} = \frac{\sum_{p \in \mathcal{P}} v_p}{\sharp\{p \in \mathcal{P} \mid u_p^n = 0\}}$
    $c_1^{n+1} = \frac{\sum_{p \in \mathcal{P}} v_p}{\sharp\{p \in \mathcal{P} \mid u_p^n = 1\}}$
end-while

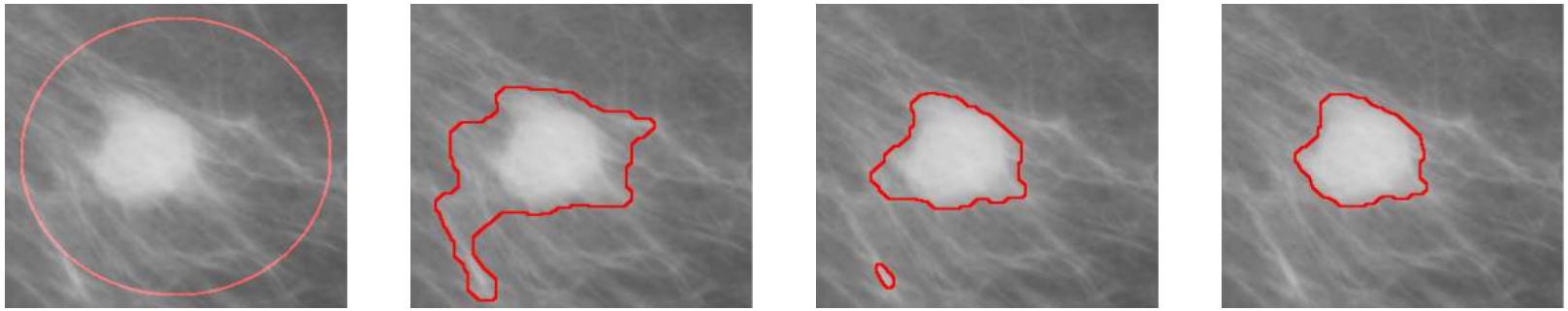


Figure 68: Example 1 - Segmentation of a mammogram (courtesy of [EZXSE07]).

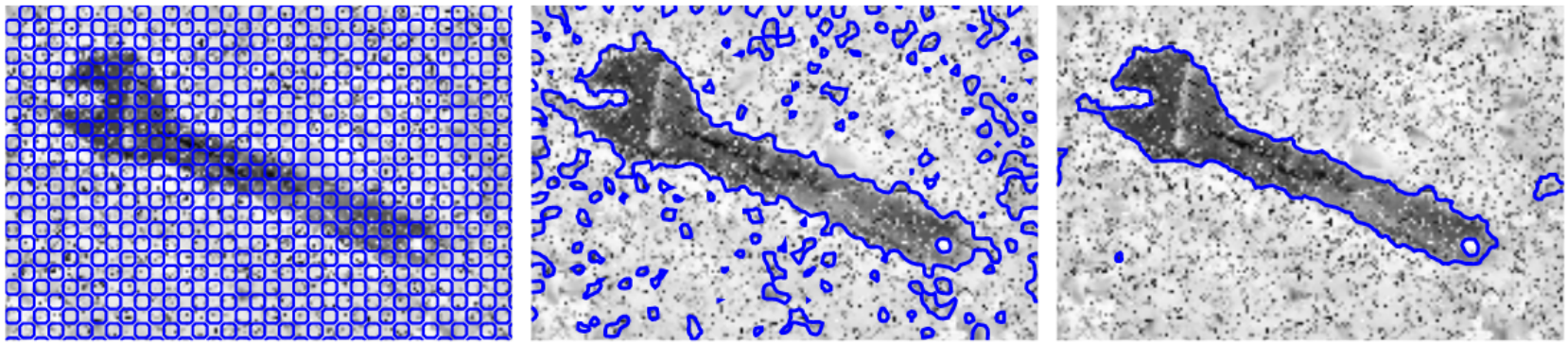


Figure 69: Example 2 - Segmentation of a noisy wrench image (courtesy of [Get12]).

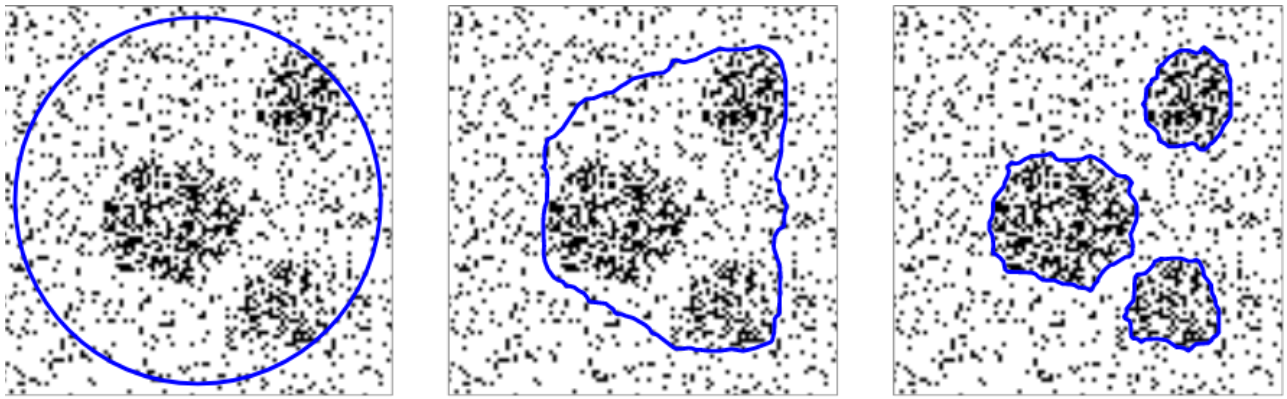


Figure 70: Example 3 - Segmentation of a points cloud (courtesy of [Get12]).

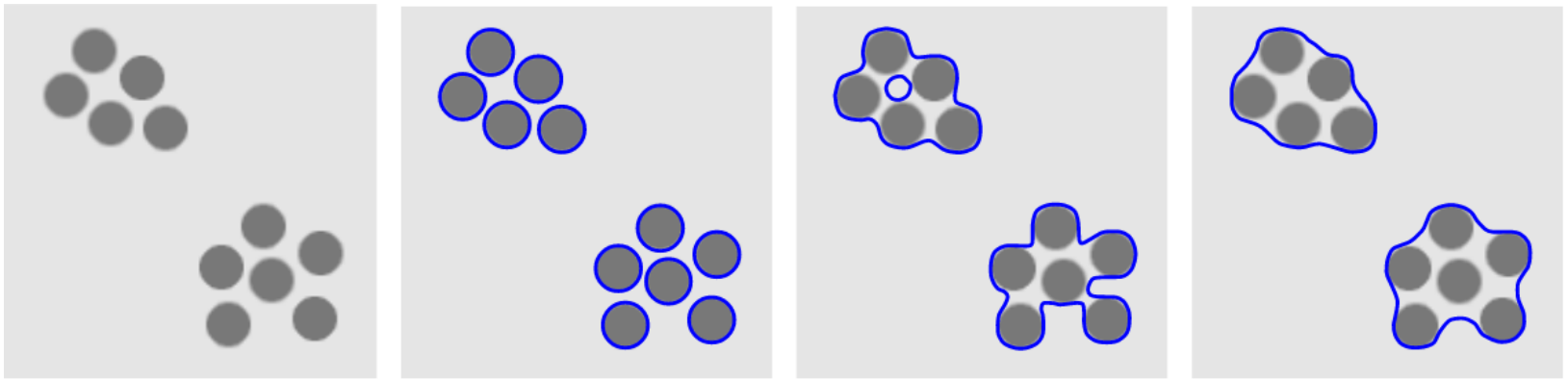


Figure 71: Influence of the parameter β (courtesy of [Get12]).

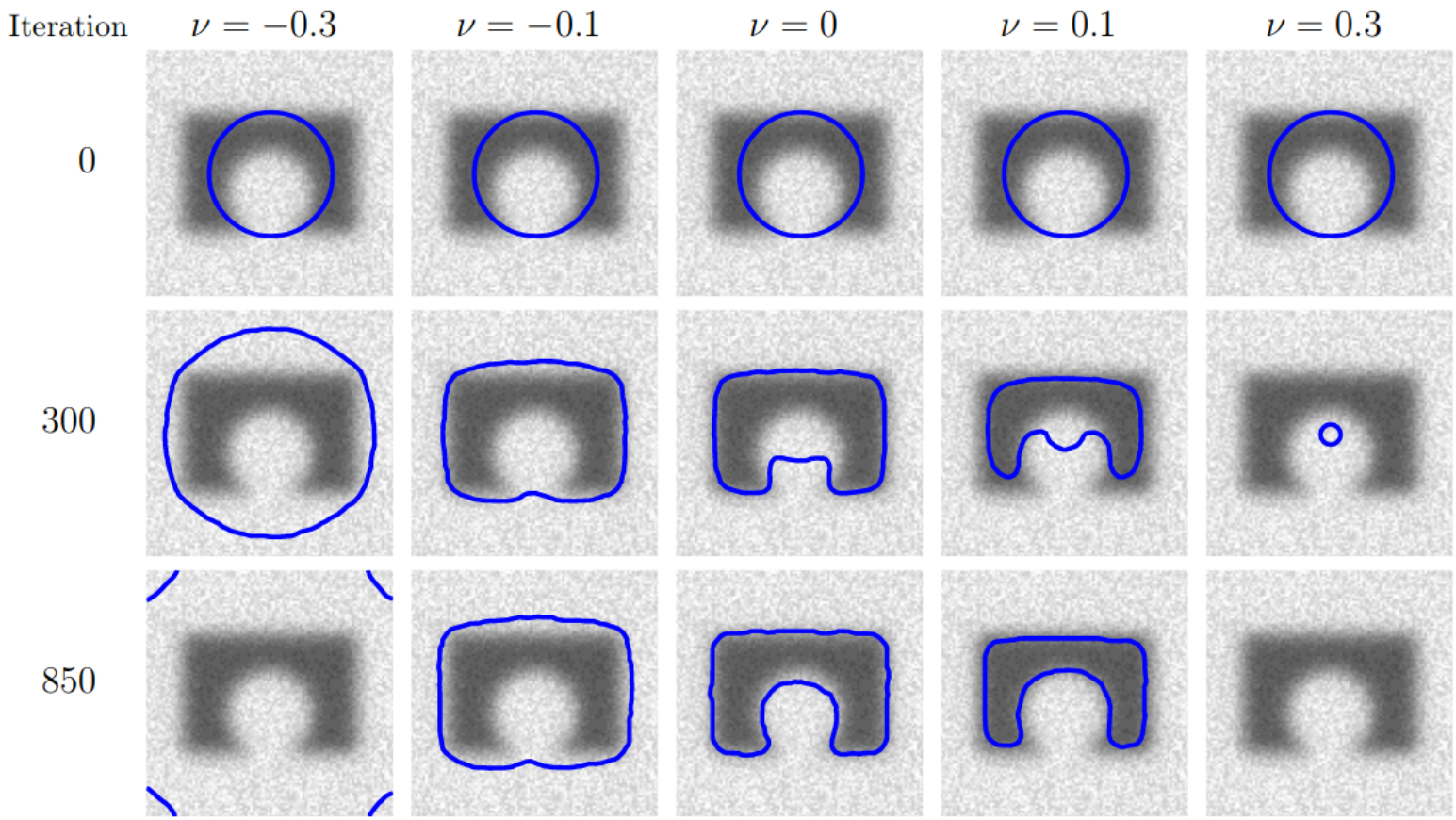


Figure 72: Influence of the parameter ν (courtesy of [Get12]).

TV-based model – Formulation

Let us consider a grayscale image $v : \mathcal{P} \subset \mathbb{Z}^2 \to \{0, \dots, L-1\}$ and the functionals

$$\min_{u \in \{0,\dots,L-1\}^{\mathcal{P}}} \sum_{(p,q)\in\mathcal{N}} w_{p,q}|u_p - u_q| + \frac{1}{2\beta}\sum_{p\in\mathcal{P}}(v_p - u_p)^2, \tag{6}$$

$$\min_{x \in \{0,1\}^{\mathcal{P}}} \sum_{(p,q)\in\mathcal{N}} w_{p,q}|x_p - x_q| + \sum_{p\in\mathcal{P}}(\alpha - v_p)x_p, \tag{7}$$

where

- $\alpha \in \{0, \dots, L-1\}$ is a threshold.
- $\beta \in \mathbb{R}^+$ is a regularization parameter.
- $w_{p,q} \in \mathbb{R}_{>0}$ are weighting coefficients.

TV-based model – Theorem (see [Cha05])

Solving (6) and thresholding u at α is equivalent to solving (7).



Figure 73: Weighted norm of optical flow segmented for $\alpha = 0.5, 0.6, 0.7, 0.8$ (courtesy of [Cha05]).

Remarks

- Solutions for a variety of α's can efficiently computed,
- (6) can be solved with parametric max-flow+divide-and-conquer strategy [CD09].
- Global minimum of the shape optimization problem (unlike classical snakes).
- Extension to vector-valued images is possible but not straighforward [Dar05].
- Any L^p norm can be used for the data term [GY09].

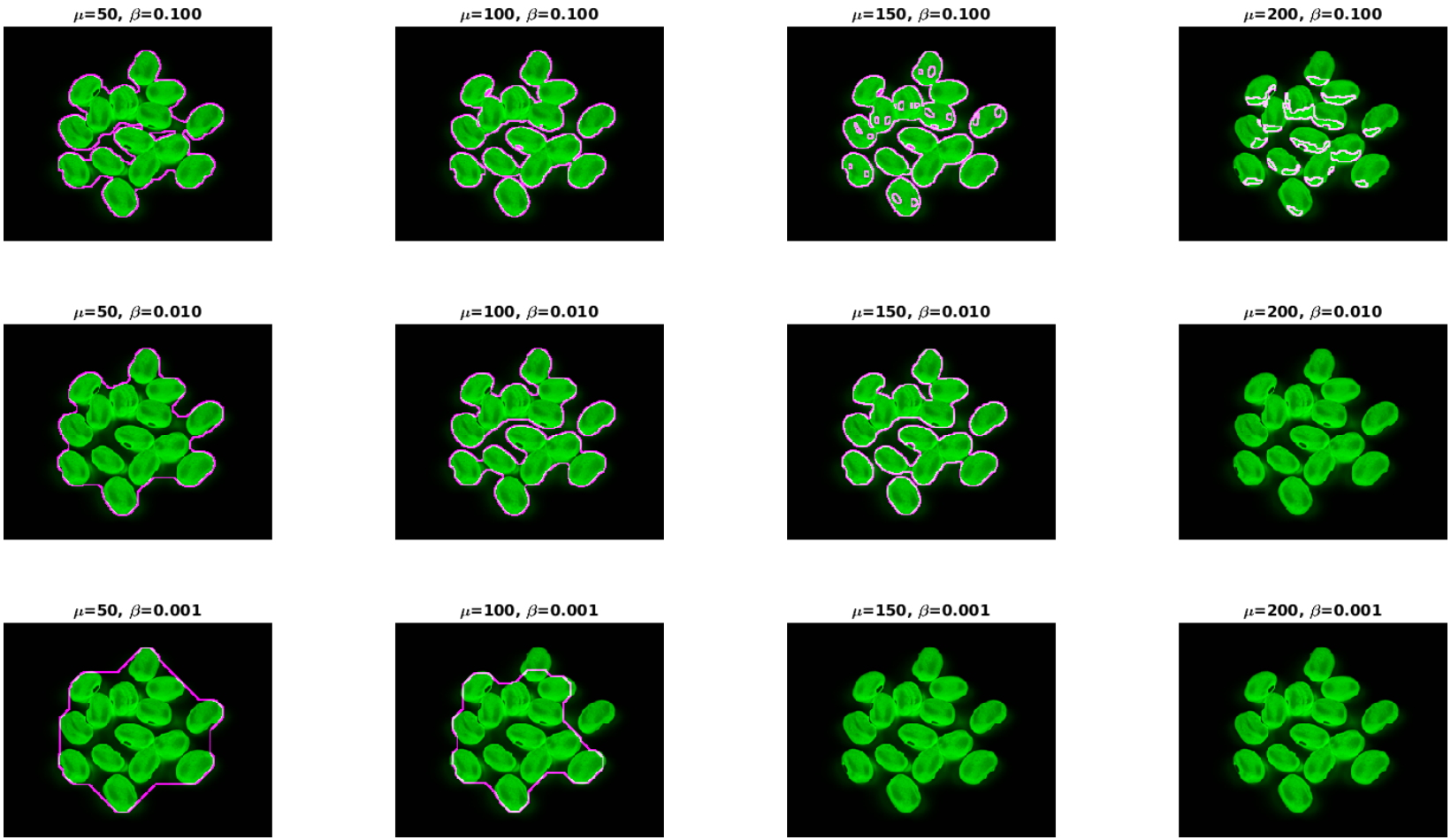


Figure 74: Segmentation of a beans image for distinct values of β and μ with 8-connectivity.

Boykov-Jolly model – Formulation

Given an image $v : \mathcal{P} \subset \mathbb{Z}^2 \to [0,1]^d$, the Boykov-Jolly model writes [BJ01]

$$E(u, p_{\mathcal{O}}, p_{\mathcal{B}}) = \beta \sum_{p \in \mathcal{P}} E_p(u_p) + \sum_{(p,q) \in \mathcal{N}} E_{p,q}(u_p, u_q),$$

among $u \in \{0,1\}^{\mathcal{P}}$, $p_{\mathcal{O}} \in \mathbb{R}^{[0,1]^d}$, $p_{\mathcal{B}} \in \mathbb{R}^{[0,1]^d}$ and $\beta \in \mathbb{R}^+$ with

$$\begin{cases} E_p(1) &= -\log p_{\mathcal{O}}(v_p \mid p \in \mathcal{O}) \\ E_p(0) &= -\log p_{\mathcal{B}}(v_p \mid p \in \mathcal{B}) \end{cases} \quad \text{and} \quad E_{p,q}(u_p, u_q) = B_{p,q} \mathbf{1}_{u_p \neq u_q},$$

where $\mathcal{O}, \mathcal{B} \subset \mathbb{Z}^2$ denotes resp. foreground and background. Choices for $B_{p,q}$ include

$$B_{p,q} = \frac{1}{\|p-q\|_2} \exp\left(-\frac{\|v_p - v_q\|_2^2}{2\sigma^2}\right), \quad \text{with } \sigma > 0, \quad \text{or,}$$

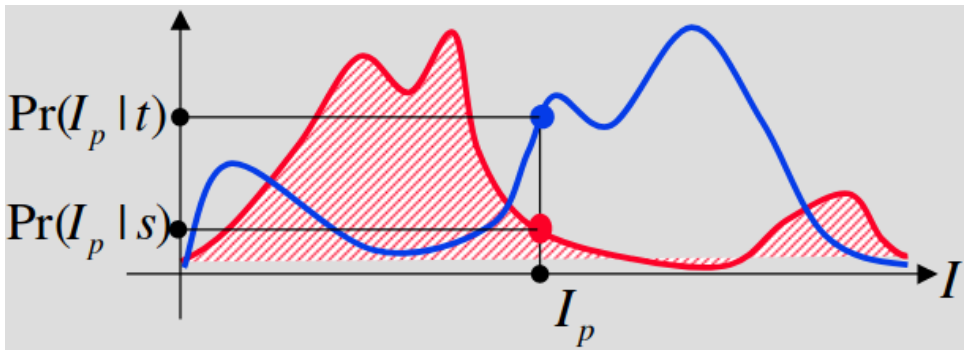
$$B_{p,q} = \frac{1}{\|p-q\|_2} \frac{1}{1 + \|v_p - v_q\|_\alpha^\alpha}, \quad \text{with } \alpha > 1.$$

Boykov-Jolly model – Some remarks about data terms

- $p_{\mathcal{O}}$,$p_{\mathcal{B}}$ can be fixed or iteratively learnt [RKB04] (E is guaranteed to decrease).
- Modeling of the distribution of image intensities is more flexible.
- PDFs must be estimated (histograms, kernel density estimation, EM, etc).
- PDFs can be initially set using seeds from the user.



(a)



(b)

Figure 75: (a) seeds and (b) $p_{\mathcal{O}}$,$p_{\mathcal{B}}$ distributions.

Algorithm 7: Binary segmentation using Boykov-Jolly model

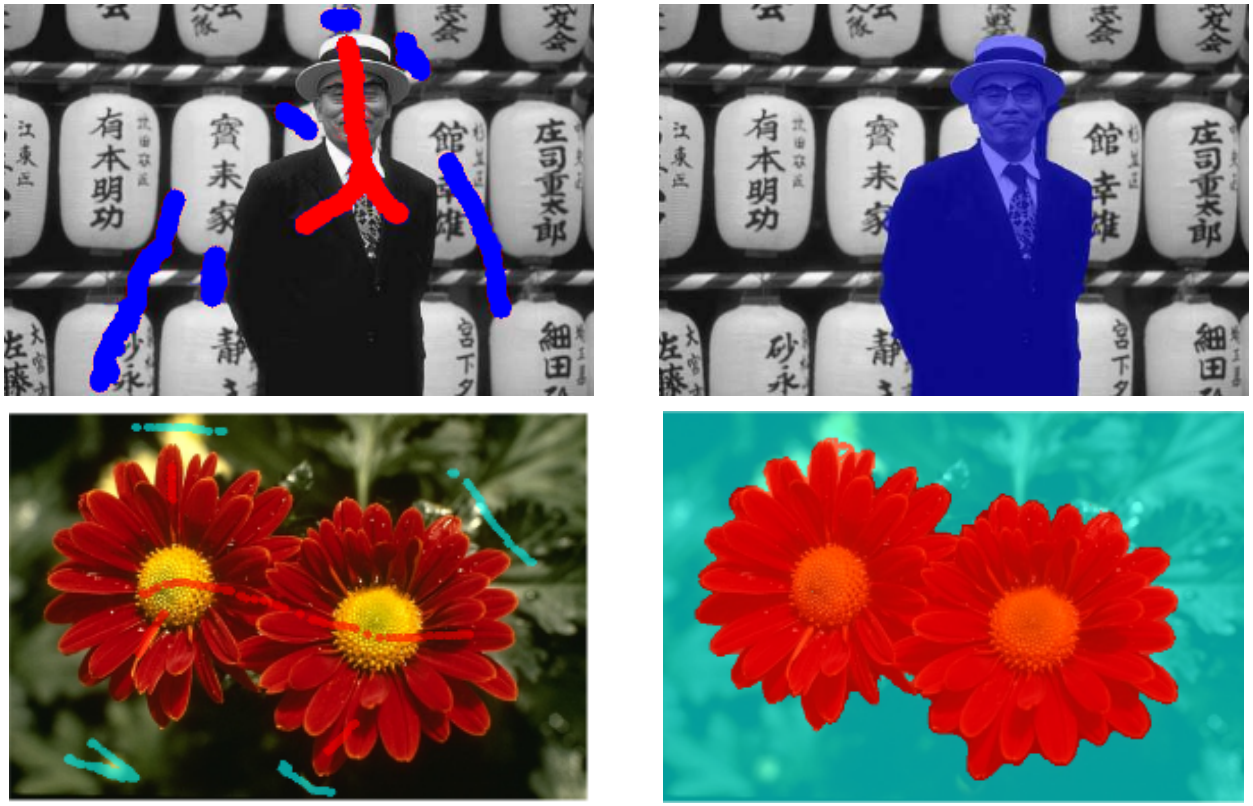
INPUTS: $I \in \mathbb{R}^{\mathcal{P}}$, $\beta \in \mathbb{R}^{+}$, $\epsilon \in \mathbb{R}_{>0}$
OUTPUTS: An approximate minimizer of E

1: Initialize $p^1_{\mathcal{O}}, p^1_{\mathcal{B}}$ *% e.g. using seeds provided by the user*
2: Let $u^0_p = +\infty, \forall p \in \mathcal{P}$
3: **while** $\|u^n - u^{n-1}\|_2^2 > \epsilon \sharp \mathcal{P}$ **do**
4: $\quad u^n \in \operatorname{argmin}_{u \in \{0,1\}^{\mathcal{P}}} E(u, p^n_{\mathcal{O}}, p^n_{\mathcal{B}})$
5: $\quad$ Update $p^{n+1}_{\mathcal{O}}$ and $p^{n+1}_{\mathcal{B}}$ based on u^n *% using the PDF estimation method*
6: **end-while**

Boykov-Jolly model – Some remarks about regularization terms

When $B_{p,q}$ is exponentially decreasing:

- $\frac{1}{\|p-q\|_2}$ aims at making more isotropic the neighborhoods like in [BK03].
- The exponential function assumes that intensity differences are Gaussian-distributed.
- σ controls the alignment of the segmentation boundary on contrasted edges:
  - When σ is small, strong edges are less penalized than weak ones.
  - When σ is large, edges are equally penalized.



(a) (b)

Figure 76: Examples of seeds (a) & segmentations (b) for grayscale (top) and color (bottom image).

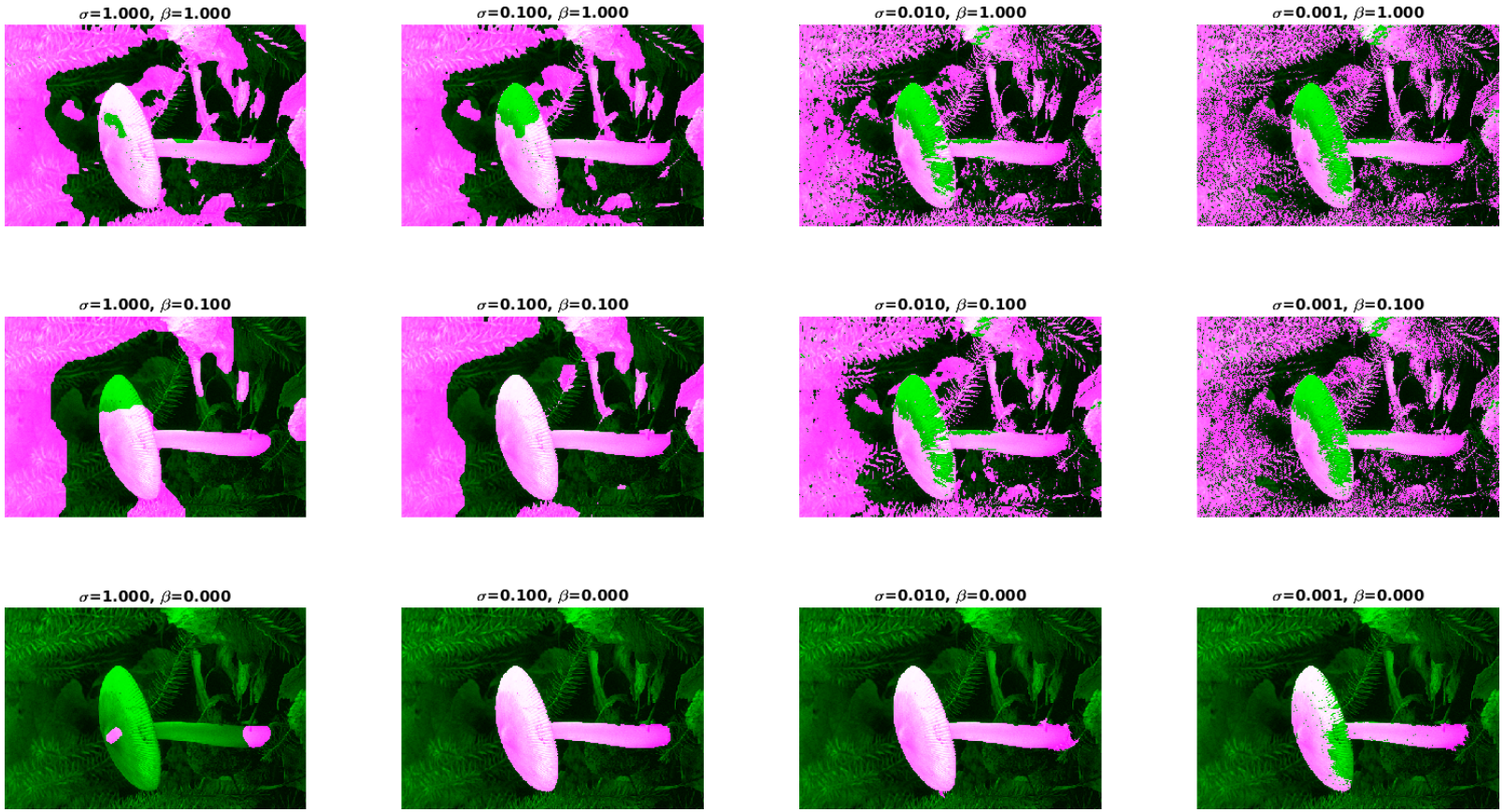


Figure 77: Influence of the parameters β and σ in 8-connectivity.

Outline

1 Motivations

2 Bayes justification

3 Elements of optimization

4 Applications

5 Conclusion

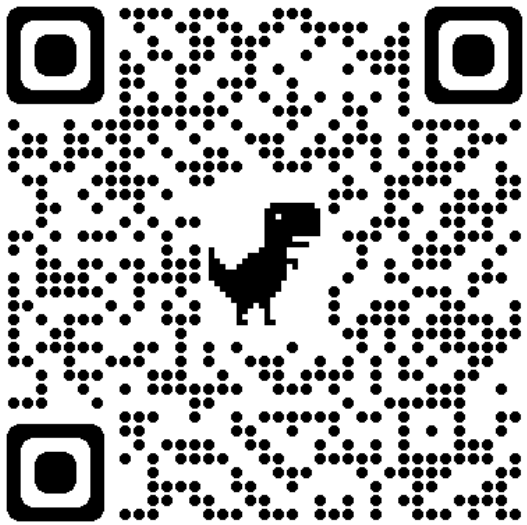
Summary

- Difference between forward and inverse problems.
- A number of vision tasks can be expressed as optimization problems.
- Problems can be either solved in the discrete or continuous domains.
- Optimization algorithms must be selected with regards to the application.

Thank you for your attention!

This course is freely available at (teaching tab)

`https://nicolaslerme.fr/`



References I

[All97] S. Alliney.
A property of the minimum vectors of a regularizing functional defined by means of the absolute norm.
IEEE Transactions on Signal Processing, 45(4):913–917, 1997.

[Ber99] D.P. Bertsekas.
Nonlinear programming.
Athena Scientific, 1999.

[Bes74] J. Besag.
Spatial interaction and the statistical analysis of lattice systems.
Journal of the Royal Statistical Society, Soc. B, 36:192–236, 1974.

[BJ01] Y. Boykov and M.-P. Jolly.
Interactive graph cuts for optimal boundary and region segmentation of objects in N-D images.
In International Conference on Computer Vision (ICCV), volume 1, pages 105–112, 2001.

[BK03] Y. Boykov and V. Kolmogorov.
Computing geodesics and minimal surfaces via graph cuts.
In International Conference on Computer Vision (ICCV), volume 1, pages 26–33, 2003.

[BK04] Y. Boykov and V. Kolmogorov.
An experimental comparison of min-cut/max-flow algorithms for energy minimization in vision.
IEEE Transactions on Pattern Analysis And Machine Intelligence, 26(9):1124–1137, 2004.

References II

[BVZ01] Y. Boykov, O. Veksler, and R. Zabih.
Fast approximate energy minimization via graph cuts.
IEEE Transactions on IEEE Transactions on Pattern Analysis And Machine Intelligence, 23(11), 2001.

[CD09] A. Chambolle and J. Darbon.
On total variation minimization and surface evolution using parametric maximum flows.
International Journal of Computer Vision, 84(3):288, 2009.

[CEPY04] T.F. Chan, S. Esedoglu, F.E. Park, and A.M. Yip.
Recent developments in total variation image restoration.
Technical report, UCLA CAM report, 2004.

[Cha04] A. Chambolle.
An algorithm for total variation minimization and applications.
20(1-2), 2004.

[Cha05] A. Chambolle.
Total variation minimization and a class of binary MRF models.
In Energy Minimization Methods in Computer Vision and Pattern Recognition (EMMCVPR), volume 3757 of LNCS, pages 136–152, 2005.

[CSV00] T.F. Chan, B.Y. Sandberg, and L.A. Vese.
Active contours without edges for vector-valued images.
Journal of Visual Communication and Image Representation, 11(2):130–141, 2000.

References III

[CV99] T.F. Chan and L.A. Vese.
Active contour model without edges.
In Lecture Notes in Computer Science, volume 1682, pages 141–151, 1999.

[Dar05] J. Darbon.
Composants logiciels et algorithmes de minimisation exacte d'Ã©nergies dÃ©diÃ©s au traitement d'images.
PhD thesis, ENST, 2005.

[DF56] G.B. Dantzig and D.R. Fulkerson.
On the max-flow min-cut theorems of networks.
Annals of Mathematics Study, 38:215–221, 1956.

[Din70] E. A. Dinic.
Algorithm for solution of a problem of maximum flow in a network with power estimation.
Soviet Math Doklady, 11:1277–1280, 1970.

[ECS56] P. Elias, D.L. Collins, and C. Shannon.
A note on the maximum flow through a network.
IRE Transactions on Information Theory, 2(4):117–119, 1956.

[EK72] J. Edmonds and R.M. Karp.
Theoretical improvements in algorithmic efficiency for network flow problems.
Journal of the ACM, 19(2):248–264, 1972.

References IV

[EZXSE07] N. El Zehiry, S. Xu, P. Sahoo, and A. Elmaghraby.
Graph cut optimization for the Mumford-Shah model.
In Proceedings of International Conference on Visualization, Imaging and Image Processing (ICVIIP), pages 182–187, 2007.

[FD05] D. Freedman and P. Drineas.
Energy minimization via graph cuts: settling what is possible.
In Conference on Computer Vision and Pattern Recognition (CVPR), volume 2, pages 939–946, 2005.

[Get12] P. Getreuer.
Chan-vese segmentation.
Image Processing On Line, pages 214–224, 2012.

[GY09] D. Goldfarb and W. Yin.
Parametric maximum flow algorithms for fast total variation minimization.
SIAM Journal on Scientific Computing, 31(5):3712–3743, 2009.

[HC71] J.M. Hammersley and P. Clifford.
Markov field on finite graphs and lattices.
unpublished, 1971.

[Ish03] H. Ishikawa.
Exact optimization for markov random fields with convex priors.
IEEE Transactions on Pattern Analysis And Machine Intelligence, 25(10):1333–1336, 2003.

References V

[JB07] O. Juan and Y. Boykov.
Capacity scaling for graph cuts in vision.
In International Conference on Computer Vision (ICCV), pages 1–8, 2007.

[KK62] H.W. Kuhn and R.E. Kuenne.
An efficient algorithm for the numerical solution of the generalized weber problem in spatial economics.
Journal of Regional Science, 4(2):21–34, 1962.

[KS80] R. Kindermann and J.L. Snell.
Markov random fields and their applications, volume 1.
American Mathematical Society, 1980.

[KZ04] V. Kolmogorov and R. Zabih.
What energy functions can be minimized via graph cuts?
IEEE Transactions on Pattern Analysis And Machine Intelligence, 26(2):147–159, 2004.

[LMHT12] N. LermÃ©, F. Malgouyres, D. Hamoir, and E. Thouin.
Bayesian image restoration for mosaic active imaging.
Inverse Problems and Imaging, 8(3), 2012.

[LML10] N. LermÃ©, F. Malgouyres, and L. LÃ©tocart.
Reducing graphs in graph cut segmentation.
In Proceedings of International Conference on Image Processing (ICIP), pages 3045–3048, September 2010.

References VI

[Raj18] S. Raj.
Analysis of convex functions.
Master's thesis, Indian Institute of Technology Hyderabad, 2018.

[RC98] S. Roy and I.J. Cox.
A maximum-flow formulation of the n-camera stereo correspondence problem.
In International Conference on Computer Vision (ICCV), pages 492–499, 1998.

[RKB04] Carsten Rother, Vladimir Kolmogorov, and Andrew Blake.
"grabcut": Interactive foreground extraction using iterated graph cuts.
ACM Transactions on Graphics, 23(3):309–314, 2004.

[RLLHM18] C. Ribal, N. LermÃ©, and S. Le HÃ©garat-Mascle.
Efficient graph cut optimization for shape from focus.
Journal of Visual Communication and Image Representation, 55:529–539, 2018.

[ROF92] L.I. Rudin, S. Osher, and E. Fatemi.
Nonlinear total variation based noise removal algorithms.
Physica D: Nonlinear Phenomena, 60(1):259–268, 1992.

[Ron17] A. Rondepierre.
MÃ©thodes numÃ©riques pour l'optimisation non linÃ©aire dÃ©terministe, 2017.

[SZS+08] R. Szeliski, R. Zabih, D. Scharstein, O. Veksler, V. Kolmogorov, A. Agarwala, M. Tappen, and C. Rother.
A comparative study of energy minimization methods for Markov random fields with smoothness-based priors.
IEEE Transactions on Pattern Analysis And Machine Intelligence, 30(6):1068–1080, 2008.

References VII

[VC02] L.A. Vese and T.F. Chan.
Multiphase level set framework for image segmentation using the Mumford and Shah model.
International Journal of Computer Vision, 50(3):271–293, 2002.

[Vek99] O. Veksler.
Efficient Graph-Based Energy Minimization Methods in Computer Vision.
PhD thesis, Cornell University, 1999.

[WHX10] X.-F. Wang, D.-S. Huang, and H. Xu.
An efficient local Chan-Vese model for image segmentation.
Pattern Recognition, 43(3):603–618, 2010.